

Министерство просвещения РФ
Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Уральский государственный педагогический университет»
Институт математики, физики, информатики и технологий
Кафедра информатики, информационных технологий
и методики обучения информатике

Автоматизированная система управления тепличным хозяйством на базе микроконтроллеров

*Выпускная квалификационная работа
бакалавра по направлению подготовки
09.03.03 – Прикладная информатика в сервисе*

Работа допущена к защите
« ____ » _____ 2021 г.
Зав. кафедрой _____

Руководитель ОПОП

Исполнитель: студент группы ПИ-1601z
Института математики, физики,
информатики и технологий
Кельсин А.А.

Руководитель: Доцент кафедры информатики,
информационных технологий и МО
информатике
Рожина И.В.

Екатеринбург – 2021

РЕФЕРАТ

Кельсин А.А. Выпускная квалификационная работа «Автоматизированная система управления тепличным хозяйством на базе микроконтроллеров» содержит 105 страниц, 48 изображений, 34 использованных источников и 6 приложений.

Ключевые слова: Умная теплица, программно-аппаратный комплекс, веб-сайт, linux.

Предмет разработки – автоматизированная система управления тепличным хозяйством.

Цель разработки – разработать программно-аппаратный комплекс с автоматизированным и ручным управлением теплицами через веб-интерфейс с сохранением данных о климате и их вывод.

В данной работе описан процесс и результаты проектирования и реализации программно-аппаратного комплекса управления тепличным хозяйством, а также веб-сайта, входящего в его состав.

Комплекс реализован на базе клиент-серверной архитектуры, где сервер реализован на базе ОС Raspbian. В комплексе применены программные продукты Arduino IDE, Apache 2.4, Mosquitto, СУБД MariaDB, протокол MQTT, а также языки программирования Python 2.7 с набором библиотек, C, языки разметки HTML5 и CSS3, а также используется микрофреймворк flask [6].

Данный программно-аппаратный комплекс создавался для упрощения выращивания растений и может автоматизировать процесс поддержания температурного режима (открывание окон на проветривание, обогрев) и процесс полива. Также реализовано ручное управление теплицами на сайте и вывод текущей температуры и влажности и сохранения их в базе данных для каждой теплицы. Дополнительно настроена работа освещения для дневного и ночного времени по движению и вручную. Однако, эта конфигурация также подойдет для аквариумов, террариумов и прочих мест, где необходимо поддерживать температурный режим и/или управлять какой-либо нагрузкой.

Реализованный программно-аппаратный комплекс полностью соответствует техническому заданию на разработку.

СОДЕРЖАНИЕ

Реферат	2
Введение	4
I глава. Современные автоматизированные системы	5
1.1 техническое и программное обеспечение интернета вещей.....	5
1.2 сравнение и выбор технических характеристик составляющих	12
1.3 формализованное описание технического задания	28
II глава. Практическая часть.....	33
2.1 разработка и настройка объекта разработки.....	33
2.2 описание продукта (результата разработки).....	60
Заключение.....	65
Список использованных источников.....	66
Приложения.....	69

Введение

Популярность автоматизированных систем и прочих устройств, облегчающих жизнь человека, становится всё больше. И часто возникает потребность автоматизировать различные процессы, такие как выращивание различных растений в теплице. Люди, имеющие дачные участки, не всегда могут быть в нужное время, чтобы открыть окно для проветривания или же полить растения в теплице, да и перспектива добираться до места, чтобы только открыть окно, не всегда радует. Поэтому популярность различных автоматических систем в быту и в промышленности ставится всё больше.

Решено создать автоматизированную систему управления теплицами.

Проект актуален тем, что данный программно-аппаратный комплекс поможет тем, в нашем случае, кто хочет выращивать хороший урожай в теплице с минимальными затратами сил не только дома, но и на производстве.

Предметом разработки является система управления тепличным хозяйством.

Целью разработки является создание программно-аппаратного комплекса с автоматизированным и ручным управлением теплицами через веб-интерфейс с сохранением данных о климате и их вывод.

Задачи:

1. Произвести анализ состояния проблемы и подходов к ее решению.
2. Изучить и обосновать выбор программных и аппаратных технологий реализации.
3. Изучить принципы построения автоматизированных систем.
4. Создать физическую модель теплицы.
5. Создать программно-аппаратный комплекс на базе микроконтроллера и написать веб-интерфейс.
6. Реализовать mqtt-брокер и веб-сервер, на котором находится написанный веб-интерфейс централизованного управления всеми теплицами и отображением данных о них.
7. Реализовать автоматический и ручной режим работы.

I глава. Современные автоматизированные системы

1.1 Техническое и программное обеспечение интернета вещей

В этом быстро развивающемся мире цифровых технологий, в большом количестве производят различные устройства для автоматизации рутинных действий, чтобы освободить время человека, которое можно потратить на более интересные дела. Например, посудомойки, роботы-пылесосы, кофе-машины и многие другие устройства были созданы, чтобы облегчить жизнь людей.

У многих во владении имеются дачные участки, сады и прочее, за которыми необходимо ухаживать и следить, чтобы получить достойный урожай, на что уходит очень много времени и сил, как и в промышленной сфере. Поэтому, автоматизация процессов выращивания, с минимальным участием человека всегда актуальна. В данной ситуации необходимо автоматизировать целый ряд различных процессов, что в свою очередь образует автоматизированную систему (АС), которая представляет собой комплекс технических и программных средств, обеспечивающая централизованное управление объектом в производственной или административной среде. Примерами применения АС являются производства автомобилей, бытовой техники, крупной техники и т.д.

Умные теплицы можно классифицировать следующим образом:

- автономные – все системы работают исключительно на тепловой или солнечной энергии;
- энергозависимые – питание элементов осуществляется от подведенной электросети.

Каждый тип обладает своими достоинствами и преимуществами, о важности, которых споры не утихают и по сей день.

Автоматизированная теплица подразумевает выполнение ряда операций без участия человека, а именно:

- поддержка требуемой температуры внутри, с учетом показаний датчиков;
- проветривание, путем открывания окон;
- автополив растений, с учетом влажности почвы.

Многие автоматизированные системы имеют локальное управление или же не имеют вовсе, полагаясь только на автоматические параметры. Кроме как выполнение ряда действий автоматически, АС может иметь доступ в интернет и возможность управления системой удаленно, без непосредственного присутствия человека вблизи системы. Такие системы называют «интернет вещей».

В современном мире в обиход вошла фраза «интернет вещей». Интернет вещей (Internet of Things, IoT) – это новая стадия развития интернета, когда к нему подключено устройств больше, чем людей. Переход к ней случился в 2008-2009 годах, когда количество устройств в сети обогнало численность населения Земли. Интернет вещей позволяет снижать трудозатраты и автоматизировать процессы компаниям, в электроэнергетике улучшает контролируемость подстанций и линий электропередачи за счет дистанционного мониторинга, а в здравоохранении позволяет перейти на новый уровень диагностики заболеваний — «умные» устройства контролируют показатели здоровья пациента. В сельском хозяйстве «умные» фермы и теплицы сами управляют количеством удобрений и воды. Развитие интернета вещей в логистике позволяет сократить затраты на грузоперевозки и минимизировать влияние человеческого фактора. Также IoT активно внедряют нефтегазовые и горнодобывающие отрасли. Применение углубленной аналитики по буровым скважинам помогает нефтегазовой промышленности увеличить объемы добычи на уже отработанных месторождениях. Интернет вещей в транспорте состоит из самого транспорта, навигаторов, электронных табло, камер наблюдения, систем безопасности.

Развитие интернета вещей стало возможным за счет широкого распространения интернета, смартфонов, беспроводных сетей, удешевления электронных компонентов и обработки данных. На практике IoT-системы обычно состоят из сети умных устройств и облачной платформы, к которой они подключены. К ним примыкают системы хранения, обработки и защиты, собранных датчиками данных.

IoT – состоит из стека технологий. Это и создание датчиков, и множество протоколов их взаимодействия. Объекты могут общаться между собой через wi-fi,

Bluetooth, LPWAN, BLE, Ethernet, RFID, ZigBee и другие виды беспроводной связи. Межмашинное взаимодействие (M2M) является частным случаем интернета вещей.

Архитектура Интернета вещей предполагает наличие следующих функциональных уровней: сеть датчиков, шлюз, управление.

Поскольку нижний уровень состоит из датчиков, сенсоров и реле, то сразу же возникает необходимость в "особенных" протоколах для обеспечения взаимодействия устройств, друг с другом и с верхними уровнями. Стандартные прикладные протоколы не подходят ввиду их неприспособленности к условиям сети интернета вещей. Датчик, обычно миниатюрный, с небольшой памятью, измеряет, физические параметры в режиме реального времени, чаще всего в условиях низкого энергообеспечения. Результаты измерений обрабатываются сенсорным узлом и передаются на сервер. Объем информации, формируемый одним сенсорным узлом, сравнительно небольшой, однако большинство сервисов «Интернета вещей» построено на принципе обработки информации от множества узлов, что принципиально отличается от архитектур, принятых в классических сетях, типа абонент – узел связи для телефонии, клиент-сервер для передачи данных.

В интернете вещей источниками данных являются различные датчики, которые в свою очередь могут подключаться к микроконтроллерам, для удобного сбора и отправки данных для дальнейшего анализа. Часто создаются проекты автоматизированных систем управления на базе микроконтроллеров. Микроконтроллер – это специальная микросхема, предназначенная для управления различными электронными устройствами.

Микроконтроллер представляет собой объединённый процессор, память, ПЗУ и периферия внутри одного корпуса, который внешне похож на обычную микросхему. Производство микроконтроллеров ежегодно во много раз превышает производство процессоров, а потребность в них не снижается.

Происходит по причине, что микроконтроллеры применяются преимущественно во встроенных системах, в игрушках, в станках, в массовой домашней технике, в домашней автоматике, где больше нужен баланс между ценой и достаточной функциональностью, нежели мощность процессора.

Наличие контроллера легко обнаружить в любом мало-мальски интеллектуальном устройстве от детской игрушки до беспроводной гарнитуры сотового телефона, так как способ использования и применения микроконтроллеров в современном мире очень разнообразен.

Микроконтроллер, в процессе работы, считывает команды из памяти или порта ввода и исполняет их. Что означает каждая команда, определяется системой команд микроконтроллера. В архитектуре микроконтроллера заложена система команд, и выполнение кода команды выражается в проведении внутренними элементами микросхемы определенных микроопераций.

В основном память в микроконтроллерах составляет от 2 до 128 Кб, бывает и больше, то используют специальные версии языка Бейсика, Паскаля, но в основном – Си. Если меньше, то писать приходится на ассемблере или Форте. До того как окончательно запрограммировать микроконтроллер, его тестируют в эмуляторах – программных или аппаратных. Микроконтроллер - это уже не процессор, но ещё и не компьютер [3].

Микроконтроллеры для массового потребителя отличаются наличием предварительно прошитого в них загрузчика. С помощью загрузчика пользователь, без использования отдельных аппаратных программаторов, загружает свою программу в микроконтроллер. Загрузчик соединяется с компьютером через интерфейс USB (если он есть на плате) или с помощью отдельного переходника UART-USB. В бесплатную программу Arduino IDE встроена поддержка загрузчика и выполняется в один щелчок мыши [3].

Программисту доступны некоторые специальные возможности портов ввода-вывода микроконтроллеров, например, широтно-импульсная модуляция (ШИМ), аналогово-цифровой преобразователь (АЦП), интерфейсы UART, SPI, I2C. Количество и возможности портов ввода-вывода определяются конкретным вариантом платы.

Помимо портов на платах микроконтроллеров иногда устанавливается периферия в виде интерфейсов USB или Ethernet. Опциональный набор внешней периферии на модулях расширения включает в себя:

- USB Device (чаще всего как виртуальный COM порт через FTDI FT232, имеются также версии с эмуляцией USB HID Class клавиатур и мышек).

- Проводной и беспроводной модули связи как на основной плате, так и на платах расширения.

- Модуль GSM и другие беспроводные интерфейсы.

- Модуль для подключения SD карт.

- Модули управления моторами. Могут подключаться также реле, электромагниты и т.п.

- Графический ЖК индикатор.

- Модуль с макетным полем.

Сторонние производители выпускают широкую гамму датчиков и исполнительных устройств, подключаемых к микроконтроллерам. Например, гироскопы, компасы, манометры, гигрометры, термометры, релейные модули, индикаторы, клавиатуры и т.п [20].

Чаще всего микроконтроллеры, которые управляют различными устройствами вводят в проводную или беспроводную сеть.

Одной из основных частей, в автоматизированной системе управления, является программное обеспечение для управления, сбора, сохранения, анализа и вывода информации. Для этого часто используется веб-сервис, который располагается на веб-сервере.

Веб-сервер (web-server) – это сервер, отвечающий за прием запросов от клиентов к веб-сайту и их обработку. Это может быть какой-нибудь компьютер, который специально выделен из группы персональных компьютеров или рабочая станция, на которых установлено и работает сервисное программное обеспечение. Клиентами обычно выступают веб-браузеры. Веб-сервер выдает клиентам HTTP-ответы, в основном вместе с HTML-страницей, которая может содержать: различные файлы, изображения, медиа-поток или любые другие данные. HTTP (англ. HyperText Transfer Protocol – протокол передачи гипертекста) – это сетевой протокол прикладного уровня передачи данных. Основным принципом протокола

HTTP является технология «клиент-сервер», обеспечивающая взаимодействие сети и пользователя.

Веб-сервер выполняет различные функции исполнения скриптов, написанных на языках программирования, таких как PHP, Python и так далее, которые отвечают за организацию запросов к сетевым службам, базам данных, доступу к файлам, пересылке электронной почты и другим приложениям электронной коммерции.

Все основные и дополнительные функции веб-сервера:

- Прием запросов от веб-браузеров по протоколу стандарта HTTP и HTTPS с использованием сетевых протоколов TCP/IP;
- Запуск прикладных программ на веб-сервере с последующей передачей и возвратом параметров обработки;
- Обслуживание и обработка запросов, типа: mailto, FTP, Telnet и т. п.;
- Выполнение поиска и отсылки файлов с гипертекстом или каких-либо документов в браузер по протоколу HTTP;
- Загрузка Java-приложений;
- Авторизация пользователей и их аутентификация;
- Работа и обслуживание навигационных карт изображений (Image map);
- Администрация и оперативное управление сервером;
- Поддержка страниц, которые генерируются динамически;
- Ведение регистрационного журнала обращений пользователей к различным ресурсам;
- Исполнения скриптов, например, написанных на языках программирования, таких как PHP, Python и т.д.

Одни из самых распространенных веб-серверов: Apache (компания Apache Software Foundation), IIS (компания Microsoft) и iPlanet server (от компаний Sun Microsystems и Netscape Communications Corporation). В данный момент на рынке существует огромный выбор веб-серверов, как коммерческих, так и бесплатных.

Для полного функционирования сайта нужны не только файлы с кодом страниц, но и базы данных. База данных представляет собой определенный набор данных, которые, как правило, связаны объединяющим признаком либо свойствами.

Обилие различных данных, которые могут быть помещены в единую базу, ведет к множеству вариаций того, что может быть записано: личные данные пользователей, записи, даты, заказы и так далее. Это удобно тем, что информацию можно быстро заносить в базу данных и так же быстро ее извлекать при необходимости.

На заре развития web-разработки все необходимые данные нужно было прописывать в коде страницы, сейчас же нужная информация может быть запрошена из базы данных при помощи скриптов. Специальные алгоритмы хранения и поиска информации, которые используются в базах данных, позволяют находить нужные сведения буквально за доли секунд.

Для взаимодействия с базами данных используются системы управления базами данных (СУБД), представляет собой программное обеспечение, которое используется для создания и работы с базами данных. Главной функцией СУБД является управление данными, обязательно поддерживает языки баз данных, а также отвечает за копирование и восстановление данных после каких-либо сбоев. Дополнительно СУБД позволяет быстро обрабатывать клиентские запросы, и выдавать актуальную информацию.

Многие системы «умных» теплиц настроены на определенный функционал с минимальной возможностью его расширения или же направлен на управление только одной теплицей.

Для автоматизации процесса выращивания растений будет реализован проект «умной» теплицы, в основе которого послужит микроконтроллер, а также централизованная система управления.

1.2 Сравнение и выбор технических характеристик составляющих

В мире микроконтроллеров существует огромное количество моделей, подходящих под любые задачи и требования пользователей, от разных производителей и с различными характеристиками. Поэтому необходимо выбрать микроконтроллер, подходящий под наш проект.

Выбор происходил из моделей разных производителей, находящихся в одном сегменте, для выявления необходимого микроконтроллера для реализации проекта. Сравнительная характеристика микроконтроллеров представлена в таблице 1.

Таблица 1. Характеристики NodeMCU (ESP8266)

Характеристика	Значение	Значение	Значение
1	2	3	4
Процессор	Tensilica Xtensa L106, частота 80 МГц (до 160 МГц) 32 разряда с низким энергопотреблением	ATmega328 16 МГц	ARM Cortex-M3 72 МГц 32 разряда
ОЗУ	128 КБ	2 КБ	20 КБ
Хранилище	4 мегабайта Flash-памяти	32 КБ	64 КБ
Wi-Fi	802.11 b/g/n	Нет (доп. модуль)	Нет (доп. модуль)
Сетевые протоколы	IPv4, TCP/UDP/HTTP/FTP	-	-
USB-порты	1 порт MicroUSB	1 x USB	1 x MicroUSB
Периферия	14 портов ввода-вывода	14 (6 из которых могут использоваться как выходы ШИМ + аналоговые входы 6)	14 (6 из которых могут использоваться как выходы ШИМ + аналоговые входы 6)
Питание	5 В, < 0,5-2.5 А через порт micro-USB или GPIO	7-12 В	7-12 В
Размеры	60 мм x 30 мм x 0,8 мм	69 x 53 мм	69 x 53 мм
Поддерживаемые языки	C, Lua, MicroPython, JavaScript, Basic, Lisp	C	C

Для создания проекта выбрана модель NodeMCU (ESP8266). На рисунке 1 изображен NodeMCU (ESP8266).

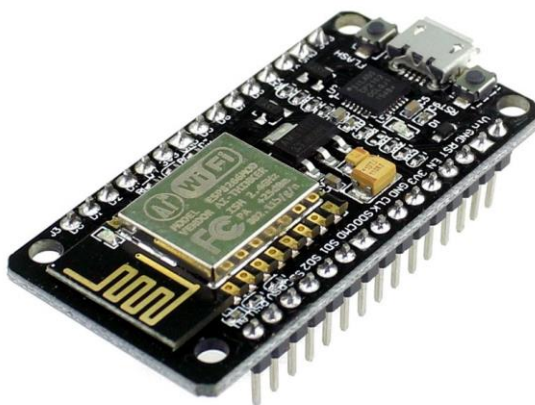


Рисунок 1. NodeMCU (ESP8266)

Данная плата имеет хорошие технические характеристики, функционал, подходящий для будущего улучшения проекта и подключению дополнительных модулей по беспроводной связи.

Микроконтроллер ESP8266 создан для использования в умных розетках, mesh-сетях (Mesh сеть – это распределенная, одноранговая, ячеистая сеть), IP-камерах, беспроводных сенсорах, носимой электронике и так далее. Одним словом, ESP8266 появился на свет, чтобы стать мозгом грядущего «Интернета вещей».

Чип ESP8266 является одним из самых высокоинтегрированных решений для работы с Wi-Fi [20]. Так и в нашем проекте микроконтроллеры и сервер будут соединяться по беспроводной сети Wi-Fi.

Wi-Fi – это аббревиатура, которая произошла от английского словосочетания Wireless Fidelity, что означает «беспроводная передача данных». Это протокол и стандарт на оборудование для широкополосной радиосвязи, предназначенной для организации локальных беспроводных сетей. Данный вид связи хорошо подходит для нашего проекта, когда теплицы находятся на некотором расстоянии от узла управления. Wi-Fi поддерживается огромным количеством устройств, отличающихся простой настройкой и достаточно небольшой ценой.

Внутри чипа ESP8266 уместилась куча всего того, что в конкурирующих решениях часто является частью внешней обвязки.

Предусмотрено два варианта использования чипа. Первый вариант в виде моста UART-WIFI, когда модуль на базе ESP8266 подключается к существующему решению на базе любого другого микроконтроллера и управляется АТ-командами, обеспечивая связь решения с инфраструктурой Wi-Fi. Этот вариант хорошо тем, у кого уже есть в руках готовая схематика и отлаженная прошивка на базе чего-то своего. Второй, реализуя новое решение, использующее сам чип ESP8266 в качестве управляющего микроконтроллера.

В нашем проекте будет использоваться второй вариант в виде платы проекта NodeMCU, где установлен модуль ESP-12E, в котором находится ESP8266 встроенный в 32-битный микроконтроллер (MCU) Tensilica L106, который отличается ультранизким энергопотреблением. Тактовая частота составляет 80 МГц, также она может достичь максимального значения 160 МГц, и внешняя flash-память на 4 МБайта. Модуль ESP-12E показан на рисунке 2 [20]

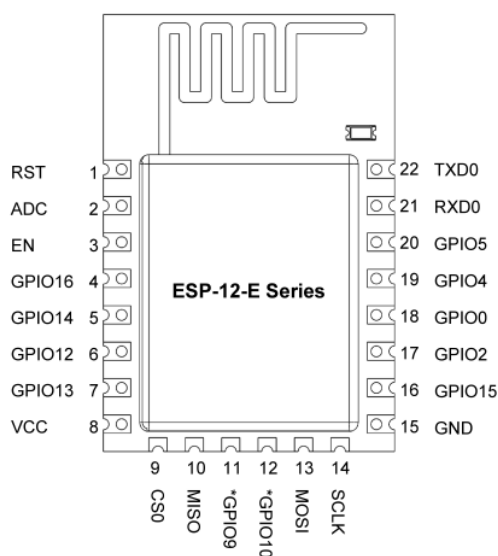


Рисунок 2. ESP-12E

Далее на плате NodeMCU реализован CP2102 – преобразователь интерфейса USB в UART, возможность питания платы от 5 вольт и разъем MicroUSB для подключения к компьютеру.

Модуль поддерживает стандарт IEEE802.11 b/g/n, полный стек TCP/IP протоколов. Разные WiFi режимы: станция, программная точка доступа, программная точка доступа и станция. Безопасность WPA, WPA2 и шифрование WEP, TKIP, AES. Также имеет возможность обновление прошивки через UART,

ОТА (через сеть) или загрузка и запись прошивки через хост. Дополнительно поддерживаются сетевые протоколы: IPv4, TCP, UDP, HTTP, FTP, что будет очень удобно для реализации простого веб-сервера [22].

Размер платы NodeMCU будет в разы меньше, всего 6 сантиметров в длину и 3 в ширину, чем у конкурентов с модулями, выполняющими такой же функционал. Питание платы NodeMCU происходит от 5-20 вольт с последующим понижением до 3.3 вольта для питания модуля ESP-12E.

Узел, отвечающий за сбор и передачу данных, а также выступающий в роли веб-сервера, решено выбрать среди одноплатных миникомпьютеров, чьих мощностей в данном проекте достаточно, чтобы реализовать задуманное.

В нише миникомпьютеров существует огромное количество моделей, подходящих под любые требования пользователей, от разных производителей и с различными характеристиками. Выбор происходил из моделей разных производителей, находящихся в одном сегменте, для выявления необходимого миникомпьютера для реализации проекта. Сравнительная характеристика миникомпьютеров представлена в таблице 2.

Таблица 2. Характеристики миникомпьютеров

Параметр	Значение	Значение	Значение
1	2	3	4
Имя миниПК	Raspberry Pi 3 model B	Orange Pi PC	Banana Pi M2
Система на кристалле (SoC)	Broadcom BCM2837 (CPU + GPU)	Allwinner H3 (CPU + GPU)	Allwinner A20
Процессор	64-битный четырёхъядерный ARMv8 Cortex-A53, 1.2 ГГц	Четырёхъядерный 32-битный процессор ARM Cortex-A7 1.2-1.6 ГГц	ARM Cortex-A7 dual core 1.0 ГГц
Графический процессор	Двухъядерный процессор (GPU) 400 МГц способен кодировать, декодировать и выводить Full HD-видео	Mali400 MP2 600MHz	ARM Mali400 MP2
ОЗУ	1 Гбайт	1 Гбайт	1 Гбайт
Хранилище	слот для карты памяти MicroSDHC, USB Boot Mode	microSD до 64 ГБ (в качестве постоянной памяти и загрузочного диска)	слот для карты памяти MicroSDHC,

1	2	3	4
Ethernet	10/100 Мбит Ethernet RJ45	10/100 Mb/s Ethernet RJ45	2× USB 2.0, 1x USB OTG
Wi-Fi/Bluetooth	Wi-Fi 802.11n и Bluetooth 4.1	нет	Wi-Fi 802.11n и Bluetooth 4.1
Видео вход	1 x CSI-2 для подключения камеры	HDMI с поддержкой HDCP, HDMI CEC, HDMI 30 функций	10/100/1000 Ethernet. Коннектор RJ45
Видео выход	1 x HDMI выход, 1 x видеовыход 3.5 мм разъем 1 x DSI (Display Serial Interface) для подключения штатного дисплея;	1 x HDMI выход, 1 x видеовыход 3.5 мм разъем	1 x HDMI выход, 1 x видеовыход 3.5 мм разъем
USB-порты	4 порта USB 2.0	2 порта USB 2.0	4 порта USB 2.0
Периферия	40 портов ввода-вывода общего назначения (GPIO)	40 портов ввода-вывода общего назначения (GPIO)	40 портов ввода-вывода общего назначения (GPIO)
Питание	5 В, 2-2.5 А через порт micro-USB или GPIO	5 В 2 А	5 В 2 А
Размеры	85.6 мм x 56.5 мм x 17 мм	85 x 55 мм	92 × 60 mm
Поддерживаемые ОС	Ubuntu, Debian, Fedora, Arch Linux, Gentoo, RISC OS, Android, Firefox OS, NetBSD, FreeBSD, Slackware, Tiny Core Linux, Windows 10 IOT	Android, Lubuntu, Debian, Raspbian	Android, Lubuntu

Для создания проекта выбрана модель Raspberry Pi 3. На рисунке 3 изображен Raspberry Pi 3 Model B с подписанными элементами, находящимися на плате.

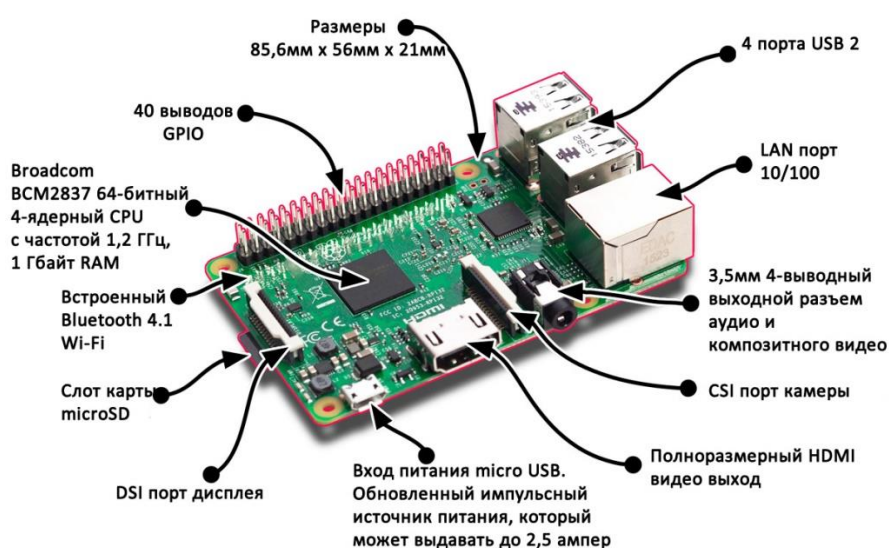


Рисунок 3. Raspberry Pi 3 Model B

Данная модель имеет хорошие технические характеристики, функционал и подходящие, вычислительные мощности для будущего улучшения проекта и подключению дополнительных модулей, также задействование в других проектах одновременно [24].

Питание Raspberry Pi 3 Model B осуществляется от 5-вольтового адаптера через разъём micro-USB или контакты питания на GPIO.

В теплице необходимо измерять как минимум температуру, влажность воздуха и почвы. Для этого было решено использовать электронные датчики способные на измерение необходимых параметров [3].

Для измерения температуры и влажности воздуха необходимо подобрать датчик, подходящий по характеристикам, который будет использоваться в данной работе. Сравнение характеристик датчиков, из которых происходил выбор, представлено в таблице 3.

Таблица 3. Сравнение датчиков

Наименование характеристики	DHT11	DHT22	DHT21
Напряжение питания, В	3-5	3-5	3-5
Температура, мин °C	0 $\pm$ 2	-40 $\pm$ 0,5	-40 $\pm$ 0,5
Температура, макс °C	50 $\pm$ 2	125 $\pm$ 0,5	80 $\pm$ 0,5
Влажность, мин %	20	0	0
Влажность, макс %	80	100	100

Для измерения температуры и влажности воздуха, выбран датчик DHT22 с напряжением питания от 3 до 5 вольт постоянного тока, который имеет большие диапазоны измерения температуры и влажности с меньшими погрешностями в сравнении с DHT11. Также данный датчик имеет меньшие размеры, чем DHT21. Выбранный датчик изображён на рисунке 4.



Рисунок 4. Датчик DHT22

Также одним из главных параметров, который необходимо отслеживать, это влажность почвы, для определения момента полива. Для этого необходимо выбрать подходящий датчик - гигрометр, характеристика которого представлена в таблице 4.

Таблица 4. Сравнительная характеристика датчиков

Наименование характеристики	FC-28	YL-69	ICB510-01
Напряжение питания, В	3.3, 5	3.3 - 5	5 В, 12-24 В
Выход	цифровой и аналоговый	цифровой и аналоговый	аналоговый
Потребляемый ток, мА	35	35	50
Размер, мм	30×60	20×60	50×208

Выбран датчик YL-69, из-за своих размеров, возможности использовать с аналоговым и цифровым входами контроллера и стоимости. Данный датчик изображен на рисунке 5.



Рисунок 5. Датчик влажности почвы YL-69

Данный датчик подключается к плате управления с потенциометром (компаратор), служащим для задания порогового значения при подключении датчика к цифровому контакту.

Далее необходимо определять время суток, для этого используется фоторезистор, производится он разными компаниями, но имеет одинаковые характеристики плюс минус на погрешности, и сравнивать их не имеет смысла. Поэтому берем фоторезистор с компаратором, для удобства задания порогового значения, так как нам необходимо определять лишь день и ночь, то и подключаться он будет к цифровому контакту контроллера. Фоторезистор — это датчик,

электрическое сопротивление которого меняется в зависимости от интенсивности падающего на него света. Чем интенсивней свет, тем меньше сопротивление элемента. На рисунке 6 показан датчик освещенности с компаратором [3].

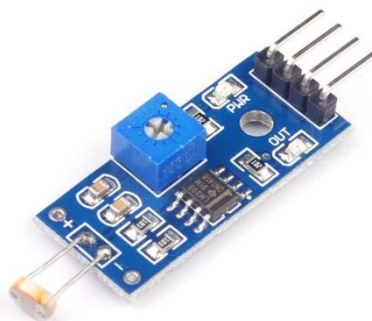


Рисунок 6. Фоторезистор с компаратором

Для того, чтобы понизить температуру внутри теплицы, необходимо выпускать горячий воздух наружу, для этого будет использоваться сервопривод. Поскольку в данном проекте представлена примерная модель теплицы, то для открывания окна будет использоваться небольшой сервопривод. В таблице 5 представлена сравнительная характеристика трех моделей сервопривода [3].

Таблица 5. Сравнительные характеристики сервоприводов

Наименование характеристики	SG-90	Turnigy TSS-10MG	ASMC-04B
Напряжение питания	5В	5В	12 В
Угол поворота ротора, градусы	360	360	300
Крутящий момент	1,6 кг/см	2,2 кг/см	90 кг/см
Внутренний интерфейс	аналоговый	цифровой	аналоговый
Направление	против часовой стрелки	против часовой стрелки	против часовой стрелки
Материал шестерней	нейлон	металл	металл
Материал корпуса	пластик	пластик	металл
Длина проводов	20 см	20 см	30 см
Габариты	23.2 × 12.5 × 22 мм	23.1 x12.0 x24.9 мм	60*95*102 мм
Вес	9 г	15 г	530 г

Для использования в проекте был выбран сервопривод SG-90. Он имеет небольшой вес и рабочее напряжение питания 5 Вольт, также, как и большинство компонентов проекта. Имеет достаточный крутящий момент и небольшой размер. Его будет достаточно, чтобы реализовать необходимый функционал. Выбранный сервопривод представлен на рисунке 7.



Рисунок 7. Сервопривод SG90

При реализации проекта в полевых условиях используются линейные приводы с концевыми выключателями, которые сообщают, что больше не нужно подавать питание на привод. Линейный привод (актуатор) – это устройство, которое преобразует вращательное движение двигателя постоянного тока в линейное перемещение.

В теплице решено реализовать освещение входа теплицы и её внутреннее помещение в темное время суток по движению, для чего необходимо выбрать датчик движения. Выбор происходил из трех моделей модулей, характеристика которых представлена в таблице 6.

Таблица 6. Сравнительные характеристики датчиков движения

Наименование характеристики	E18-D80NK	HC-SR501	HC-SR505
Напряжение питания, В	5	4.5 - 20	4.5 - 20
Угол зоны обнаружения, градусов	< 15	< 140	< 100
Рабочая температура, С	-25...+55	-20...+80	-20...+80
Дальность обнаружения, м	0.03 – 0.8	3-7м (подстраивается)	3
Размеры платы, мм	45x17	32x24x18	10x23

Для использования в проекте выбран датчик движения HC-SR501 - пирозлектрический инфракрасный датчик движения, позволяет обнаруживать движение людей в контролируемой зоне. Представляет из себя модуль, состоящий из ИК сенсора 500BP, линзы Френеля, и управляющей модулем микросхемы BISS0001. На рисунке 8 изображен модуль HC-SR501 [3].

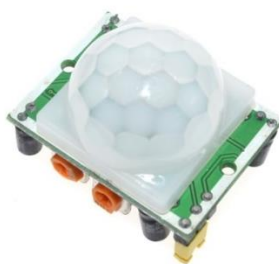


Рисунок 8. Модуль датчика HC-SR501

Для организации полива также необходим насос, но поскольку создается макет теплицы, то и использовать будем небольшой по размеру и мощности AD20P.

Для управления силовой нагрузкой необходимо такое устройство как реле. Реле – коммутационное устройство, соединяющее или разъединяющее цепь электрической или электронной схемы при изменении входных величин тока. В таблице 7 представлена сравнительная характеристика реле.

Таблица 7. Сравнительные характеристики реле

Наименование характеристики	NCR HHG1-1/032F-38-100A	NCR HHG1-1/032F-38-30A	SRD-05VDC-SL-C
Тип	твердотельное	твердотельное	электромагнитное
Управляющее напряжение, В	3...32	3...32	5
Коммутируемое переменное напряжение, В	40...440	40...440	125...250
Максимальный ток нагрузки, А	100	30	10

Для коммутации силовой нагрузки выбрано электромагнитное реле SRD-05VDC-SL-C.. Реле будет в виде модуля для удобства и простоты монтажа и легкой замены нагрузки. Помимо самого реле, модуль содержит еще и оптоэлектронную развязку с транзистором, которая защищает выводы микроконтроллера от скачков напряжения на катушке. На рисунке 9 изображен 4-канальный релейный модуль на основе SRD-05VDC-SL-C.



Рисунок 9. Релейный модуль

К данному модулю с свою очередь можно будет подключать необходимую нагрузку: освещение, обогреватель, насос для полива и прочее [3].

Становление Интернета вещей значительно расширяет возможности сбора, анализа и распределения данных, которые для человека превращаются в информацию, знания и используются для решения специфических задач. Для связи между устройствами необходимо выбрать с помощью какого протокола они будут общаться. Существует множество протоколов для использования в различных задачах, например CoAP, MQTT, Modbus.

MQTT (Message Queuing Telemetry Transport) – это асинхронный протокол передачи сообщений по принципу издатель/подписчик с использованием брокера в сетях с низкой пропускной способностью. Особенностью протокола является его компактность, т.к. величина заголовка пакета данных составляет всего лишь несколько байт. Работает поверх TCP/IP, который обеспечивает простой и надежный поток данных. Основной целью MQTT является удаленный мониторинг данных, собираемых из большого количества устройств, и их телеметрия в IT инфраструктуру. Протокол нацелен на большую сеть небольших устройств, которые необходимо контролировать или управлять из облака. Также он предназначен для «многоадресной передачи» данных для многих приемников. MQTT используется в системах M2M (Машинно-Машинное взаимодействие) и IIoT (Промышленный Интернет вещей).

Протокол CoAP часто сравнивают с MQTT для разработки систем IoT. Они похожи, но есть заметные различия. MQTT – это протокол «многие ко многим», в то время как CoAP – это в основном протокол «один к одному» для связи между сервером и клиентом. В то же время CoAP предоставляет функции метаданных, обнаружения и согласования содержимого, которых нет у MQTT.

Modbus – это открытый коммуникационный протокол для обмена данными между сетевыми устройствами, основанный на архитектуре ведущий-ведомый (master-slave). Преимуществами протокола Modbus являются открытость и массовость использования (множество производителей выпускают различные типы устройств, датчиков, поддерживающих данный протокол). Задачами протокола

Modbus являются контроль и управление сетями, чтение и запись данных в регистры хранения, доступ к файлам, диагностика состояния устройств.

Для реализации передачи данных между сервером и клиентом решено выбрать протокол MQTT. Протокол сделан маленьким и лёгким, с учётом суровых условий эксплуатации, и идеален для устройств с небольшой вычислительной мощностью и ограниченным временем автономной работы. К их числу сейчас относятся и смартфоны, и постоянно растущее число датчиков и подключённых устройств и др.

Одно из главных особенностей протокола является то, что инициаторами подключения выступают клиенты, это может оказаться полезным, когда необходимо собирать данные из разных территориально распределённых источников. При этом брокер-сервер имеет белый ip-адрес, что позволяет использовать любой хостинг, а клиенты, которые подключаются к брокеру, имеют просто выход в интернет.

MQTT обладает следующими особенностями:

1. Асинхронный протокол;
2. Работа в условиях нестабильной связи на линии передачи данных;
3. Идеально подходит для распределённых коммуникаций «один ко многим» и разъединённых приложений;
4. Нейтрален к содержимому сообщения;
5. Полагается на TCP/IP для базовых задач связи;
6. Оснащён функцией LWT (Last Will and Testament, «последняя воля и завещание») для уведомления сторон об аномальном отключении клиента;
7. Поддержка нескольких уровней качества обслуживания (QoS): «максимум один раз», «минимум один раз» и «ровно один раз».

Все эти моменты сделали MQTT протоколом для потоковой передачи данных между устройствами с ограниченной мощностью CPU и/или временем автономной работы, а также для сетей с низкой пропускной способностью, непредсказуемой стабильностью или высокой задержкой. Поэтому MQTT известен как идеальный транспорт для IoT. Он построен на протоколе TCP/IP, но есть ответвление MQTT-SN для работы по Bluetooth, UDP, ZigBee и в других сетях IoT, отличных от TCP/IP.

В протоколе MQTT обмен сообщениями осуществляется между клиентом (client), который может быть издателем или подписчиком (publisher/subscriber) сообщений, и брокером (broker) сообщений.

Издатель отправляет данные на MQTT брокер, указывая в сообщении определенную тему, топик (topic). Подписчики могут получать разные данные от множества издателей в зависимости от подписки на соответствующие топики. Участник системы MQTT может взять на себя роль издателя, потребителя или сразу обе роли

Издатель не требует каких-либо настроек по количеству или расположению подписчиков, получающих сообщения. Кроме того, подписчикам не требуется настройка на конкретного издателя. В системе может быть несколько брокеров, распространяющих сообщения.

Устройства MQTT используют определенные типы сообщений для взаимодействия с брокером, ниже представлены основные:

- Connect – установить соединение с брокером;
- Disconnect – разорвать соединение с брокером;
- Publish – опубликовать данные в топик на брокере;
- Subscribe – подписаться на топик на брокере;
- Unsubscribe – отписаться от топики.

Схема простого взаимодействия между подписчиком, издателем и брокером показана на рисунке 10.



Рисунок 10. Схема взаимодействия протокола mqtt

Топики представляют собой символы с кодировкой UTF-8. Иерархическая структура топиков имеет формат «дерева», что упрощает их организацию и доступ к данным. Топики состоят из одного или нескольких уровней, которые разделены

между собой символом «/». Всякий раз, когда у издателя есть новые данные для распространения среди клиентов, сообщение сопровождается примечанием контроля доставки. Клиенты более высокого уровня могут получать каждое сообщение, в то время как клиенты более низкого уровня могут получать сообщения, относящиеся только к одному или двум базовым каналам, «ответвляющимся» в нижней части иерархии. Это облегчает обмен информацией размером от двух байт до 256 мегабайт.

Пример топика, в который датчик температуры, расположенный в кухне, публикует данные брокеру: `/home/kitchen/kitchen1/temperature` .

Подписчик может так же получать данные сразу с нескольких топиков, для этого существуют подстановочные знаки (wildcard). Они бывают двух типов: одноуровневые и многоуровневые.

Одноуровневый wildcard, для его использования применяется символ «+», который мы можем использовать вместо указания имени для любого уровня в фильтре темы.

К примеру, нам необходимо получить данные о температуры во всех спальнях комнатах:

`/home/room-space+/temperature`

В результате получаем данные с топиков:

`/home/room-space/kitchen1/temperature`

`/home/room -space/kitchen2/temperature`

Многоуровневый wildcard, для его использования применяется символ «#», который мы можем использовать только в конце фильтра темы в качестве последнего уровня, что соответствует любой теме, первые уровни которой совпадают с уровнями тем, указанными в левой части от специального знака «#».

К примеру, чтобы получить данные с различных датчиков всех кухонь в доме:

`/home/room-space/#`

В результате получаем данные с топиков:

`/home/room-space/kitchen1/temperature`

`/home/room -space/kitchen2/light1`

MQTT поддерживает три уровня качества обслуживания (QoS – quality of service) при передаче сообщений.

QoS 0. На этом уровне издатель один раз отправляет сообщение брокеру и не ждет подтверждения от него, то есть отправил и забыл.

QoS 1. Этот уровень гарантирует, что сообщение точно будет доставлено брокеру, но есть вероятность дублирования сообщений от издателя.

QoS 2. На этом уровне гарантируется доставка сообщений подписчику и исключается возможное дублирование отправленных сообщений.

Для работы с Raspberry Pi использовать будем Raspbian – бесплатная операционная система на основе Debian, который специально оптимизирован для оборудования Raspberry Pi [5].

Для удобства написания пользовательского интерфейса решено использовать фреймворк, написанный на языке Python. Python – высокоуровневый язык программирования общего назначения, ориентированный на повышение производительности разработчика и читаемости кода. Синтаксис Python’a минималистичен, в то же время стандартная библиотека включает большой объем полезных функций. Фреймворк – это программная среда специального назначения, своеобразный каркас, используемый для того, чтобы существенно облегчить процесс объединения определенных компонентов при создании программ. База, на которой можно сформировать веб-сервис любого назначения достаточно быстро и без особых затруднений.

Выбор фреймворка происходил из трех вариантов: Flask, Django, TurboGears.

TurboGears – open-source full-stack фреймворк для веб-приложений. Он позволяет разработчику быстро создавать расширяемые веб-приложения, управляемые данными. TurboGears поставляется с удобными шаблонами и мощным и гибким ORM. Наиболее важные функции фреймворка: поддержка различных баз данных, поддержка SQLAlchemy и SQLAlchemy, валидация для FormEncode, Pylons как веб-сервер, инструменты командной строки.

Flask – это микрофреймворк, который предоставляется по лицензии BSD. Его разработчики вдохновлялись фреймворком Sinatra Ruby. Он зависит от

инструментария Werkzeug WSGI и шаблона Jinja2. У Flask модульный дизайн, который позволяет адаптировать его для выполнения многих задач. «Из коробки» разработчик получает следующие функции: встроенный сервер и дебаггер, шаблоны Jinja2, поддержка безопасных куки, WSGI 1.0, Unicode, возможность подключения к любой ORM.

Создавался фреймворк для поддержки разработчиков веб-приложений, которые получили возможность выбирать расширения по вкусу.

Django – свободный open-source full-stack фреймворк. Он позволяет добавить большинство стандартных функций единым пакетом вместо поиска отдельных библиотек. Среди них – такие востребованные, как аутентификация, URL-маршрутизация, миграция схемы данных и т.п. Django использует ORM для сопоставления объектов с таблицами баз данных. Один и тот же код работает с разными базами данных, так что переход из одной БД в другую становится простой задачей. Основными базами, с которыми работает Django, являются PostgreSQL, MySQL, SQLite и Oracle. Можно добавить и другие, но для этого понадобятся решения сторонних разработчиков.

Решено выбрать микрофреймворк Flask, который наиболее подходит за счет своей модульности и масштабируемости. Изначально имеет базовый набор инструментов, что позволяет подключать и использовать только те модули, которые необходимы в проекте.

На данном этапе завершен выбор аппаратных и программных компонентов проекта. Выбраны микроконтроллер NodeMCU (esp8266), Raspberry Pi 3 в качестве веб-сервера, датчики для получения данных о температуре и влажности воздуха и почвы внутри теплицы, и датчик движения.

1.3 Формализованное описание технического задания

на разработку системы управления
«Автоматизированная система управления тепличным хозяйством
на базе микроконтроллеров»

Составлен на основе ГОСТ 34.602-89 «Техническое задание на создание
автоматизированной системы» [20].

1. Общие сведения.

1.1. Название организации-заказчика.

ФГБОУ ВО «УрГПУ»

1.2. Название продукта разработки (проектирования).

«Автоматизированная система управления тепличным хозяйством на базе
микроконтроллеров»

1.3. Назначение продукта.

Автоматизация частной и промышленной сельскохозяйственной деятельности.

1.4. Плановые сроки начала и окончания работ.

В соответствии с планом выполнения ВКР (01.09.2019 – 19.05.2020).

2. Характеристика области применения продукта.

2.1. Процессы и структуры, в которых предполагается использование продукта разработки.

Частные или промышленные организации, занимающиеся
сельскохозяйственной деятельностью. Обеспечение автоматизации процессов
выращивания и централизованное управление теплицами.

2.2. Характеристика персонала (количество, квалификация, степень готовности)

Разработчик:

- знания html, css, python, flask, sql, mqtt библиотеки для python, язык C;
- умения работать с микроконтроллерами.

Техник-установщик:

- умения пайки электроэлементов;
- базовые знания электротехники.

Системный администратор:

- знания операционных систем UNIX-подобных систем;
- умение настраивать веб-сервер Apache;
- умение настраивать mqtt-брокер mosquitto.

Пользователь:

- базовые навыки работы с компьютером.

3. Требования к продукту разработки.

3.1. Требования к продукту в целом.

Требуется спроектировать и разработать автоматизированную систему управления на основе клиент-серверной архитектуры, включающую в себя:

- Программное обеспечение для автоматического управления проветриванием, обогревом и поливом;
- База данных для сохранения данных о температуре и влажности;
- Веб-сайт, с возможностью централизованного управления теплицами, с выводом данных о температуре и влажности для каждой теплицы, а также возможность резервного способа управления.

3.2. Аппаратные требования.

Серверная часть (минимальные требования):

- Система на кристалле (SoC) Broadcom BCM2837;
- Процессор 64-битный четырёхъядерный ARMv8 Cortex-A53 процессор с тактовой частотой 1.2 ГГц;
- Графический процессор: Двухъядерный процессор (GPU) 400 МГц способен кодировать, декодировать и выводить Full HD-видео;
- 1 Гбайт ОЗУ;
- 8 ГБ дискового пространства;
- Ethernet 100 Мбит Ethernet RJ45;
- Wi-Fi 802.11n.

Рабочая станция (минимальные требования):

- 64 или 32-разрядный процессор, 2 ядра или более, тактовой частотой не менее 2.3ГГц;

- Ethernet 10/100 Мбит Ethernet RJ45;

- Wi-Fi/Bluetooth Wi-Fi 802.11n;

- Браузер Google Chrome 68 и выше, Opera 34 и выше,

3.3. Указание системного программного обеспечения (операционные системы, браузеры, программные платформы и т.п.).

- Браузеры: Google Chrome, Яндекс-браузер, Mozilla Firefox, Opera.

3.4. Указание программного обеспечения, используемого для реализации.

- Поддерживаемые операционные системы: Ubuntu, Debian, Fedora, Arch Linux, Gentoo, RISC OS;

- Putty -удаленное управление сервером на базе Linux по протолку SSH;

- Язык программирования Python 2.7;

- Языки разметки HTML5, CSS3;

- Веб-сервер Apache 2.4;

- Arduino IDE;

- Notepad ++;

- mqtt-брокер Mosquitto;

- СУБД MariaDB.

3.5. Для сетевых систем – особенности реализации серверной и клиентской частей.

Серверная часть системы может располагаться либо на выделенном сервере непосредственно на месте развертывания системы, так и находиться на арендованном в датацентре выделенном или виртуальном сервере.

Рабочая станция может располагаться в любой точке мира, откуда возможно произвести подключение к серверу.

В качестве программного обеспечения для серверной части используются имеющиеся на рынке продукты, необходимо правильно установить и сконфигурировать их в соответствии с требованиями, а также обеспечить их слаженную работу и взаимодействие.

Веб-сайт разрабатывается с нуля и предполагает общее управления теплицами.

3.6. Форматы входных и выходных данных

Форматы входных и выходных данных в рамках mqtt-протокола состоят из строковых значений.

Ввод со стороны пользователя осуществляется путем нажатия кнопок и задания фильтров.

3.7. Источники данных и порядок их ввода в систему (программу), порядок вывода, хранения.

Источниками данных являются микроконтроллеры, которые передают данные серверу для хранения в базе данных.

Хранение осуществляется в трех несвязанных таблицах: температура, влажность воздуха и наименования микроконтроллеров.

Вывод данных происходит путем выбора на сайте соответствующей теплицы и определённых фильтров: даты и количество данных.

3.8. Порядок взаимодействия с другими системами, возможности обмена информацией.

Не предусмотрено.

3.9. Меры защиты информации.

Не предусмотрено.

4. Требования к пользовательскому интерфейсу.

4.1. Общая характеристика пользовательского интерфейса.

Пользовательский интерфейс централизованного управления имеет:

- элементы управления нагрузкой теплиц;
- меню для перехода по страницам;
- информация о текущей температуре и влажности;
- информация о температуре и влажности с фильтром по дате и количеству данных;
- информация о состоянии нагрузки (включен ли обогрев, свет и т.д.).

Пользовательский интерфейс управления одной теплицей:

- элементы управления нагрузкой теплицы;
- информация о состоянии нагрузки (включен ли обогрев, свет и т.д.);
- информация о текущей температуре и влажности.

4.2. Размещение информации на экране, дизайн экрана.

Сверху окна располагается шапка. Основная информация располагается по центру. Слева присутствует меню для перехода по страницам. Снизу блок с дополнительной информацией. Эскиз сайта и цветовая гамма представлена на рисунках 11 и 12.

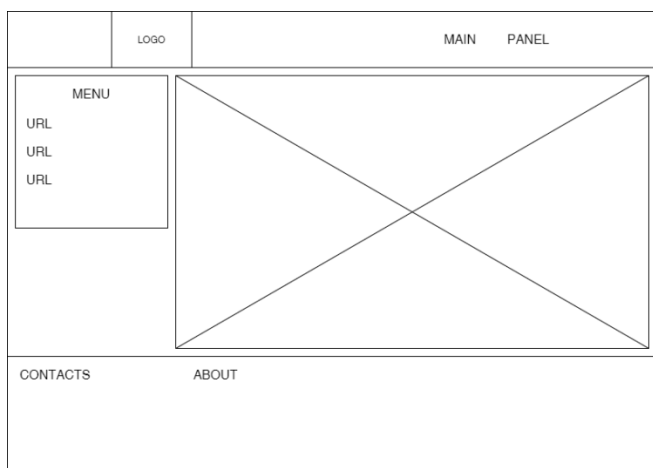


Рисунок 11. Эскиз сайта схематично

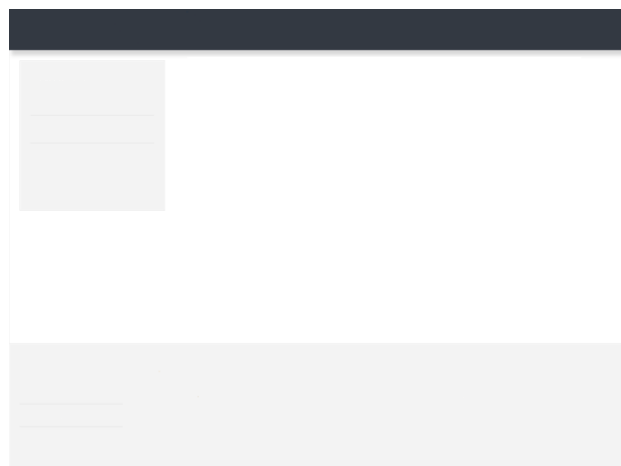


Рисунок 12. Цветовой палитры сайта

4.3. Особенности ввода информации пользователем, представление выходных данных.

Ввод данных пользователем не предусмотрен. Выходные данные представлены в табличном виде, по каждой теплице.

5. Требования к документированию.

5.1. Перечень сопроводительной документации.

Не предусмотрено.

5.2. Требования к содержанию отдельных документов.

Не предусмотрено.

6. Порядок сдачи-приемки продукта.

В соответствии с планом выполнения ВКР.

II глава. Практическая часть

2.1 Разработка и настройка объекта разработки

Главной особенностью микроконтроллеров является контакты GPIO. Интерфейс ввода/вывода общего назначения (англ. general-purpose input/output, GPIO) – интерфейс для связи между компонентами системы, к примеру, микропроцессором и различными периферийными устройствами. Контакты GPIO могут выступать как в роли входа, так и в роли выхода – это, как правило, конфигурируется.

GPIO контакты не имеют специального назначения и, как правило, остаются неиспользованными. Идея состоит в том, что иногда для построения полной системы, использующей тот либо иной чип, может оказаться полезным иметь несколько дополнительных линий цифрового управления. Это даёт возможность организовать дополнительные схемы, не создавая их с нуля. На рисунке 13 показаны подписанные GPIO контакты.

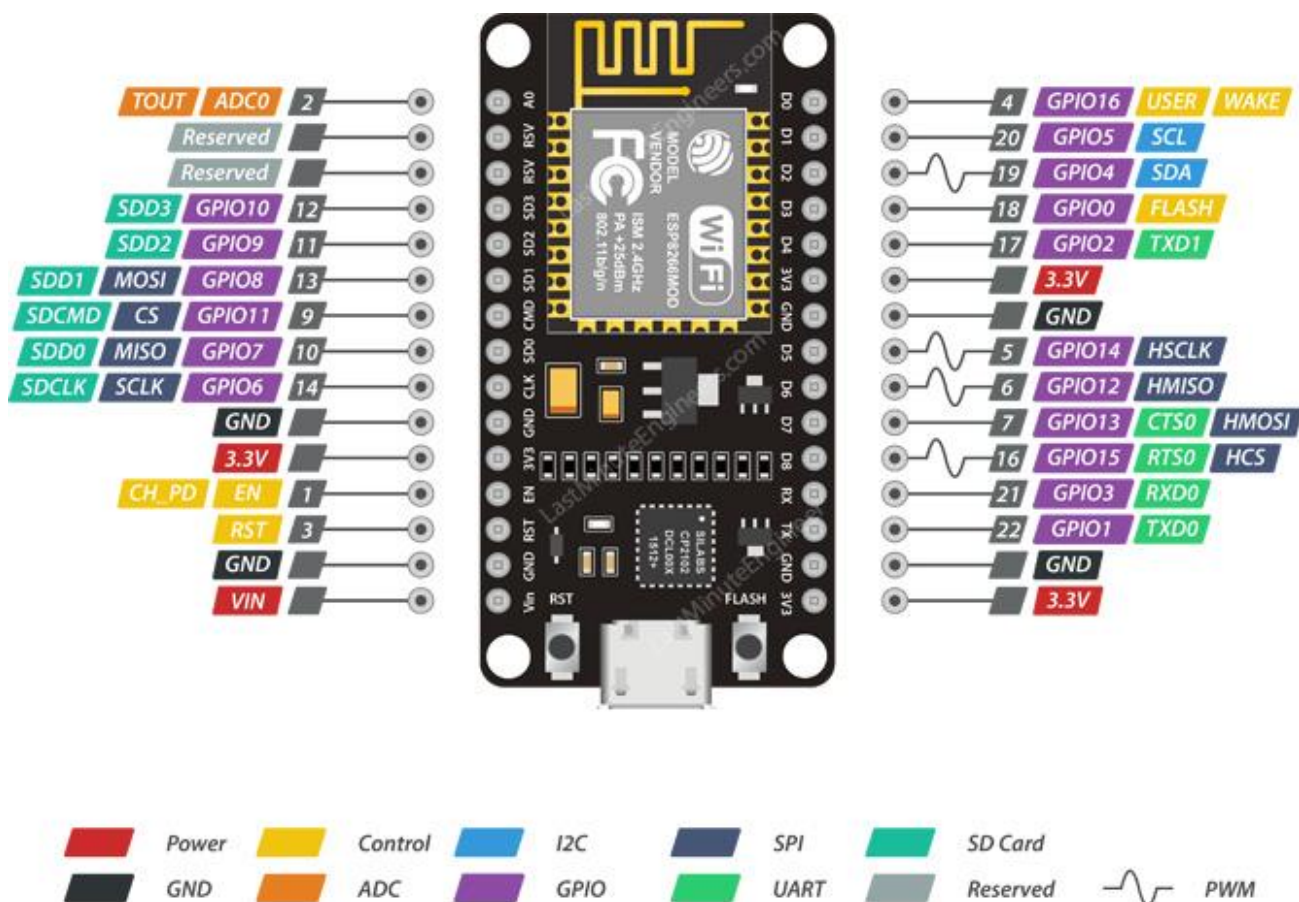


Рисунок 13. Пронумерованные контакты GPIO

Для подключения датчиков необходимо 8 контактов. Контакты под номерами D0, D1, D2 будут использоваться для управления реле с нагрузкой, а контакты D4, D5, D7, A0 для подключения датчиков, также к контакту D8 подключен управляющий провод сервопривода. Схема подключения к контактам GPIO показана на рисунке 14, а также данная схема представлена в большем размере в приложении 1.

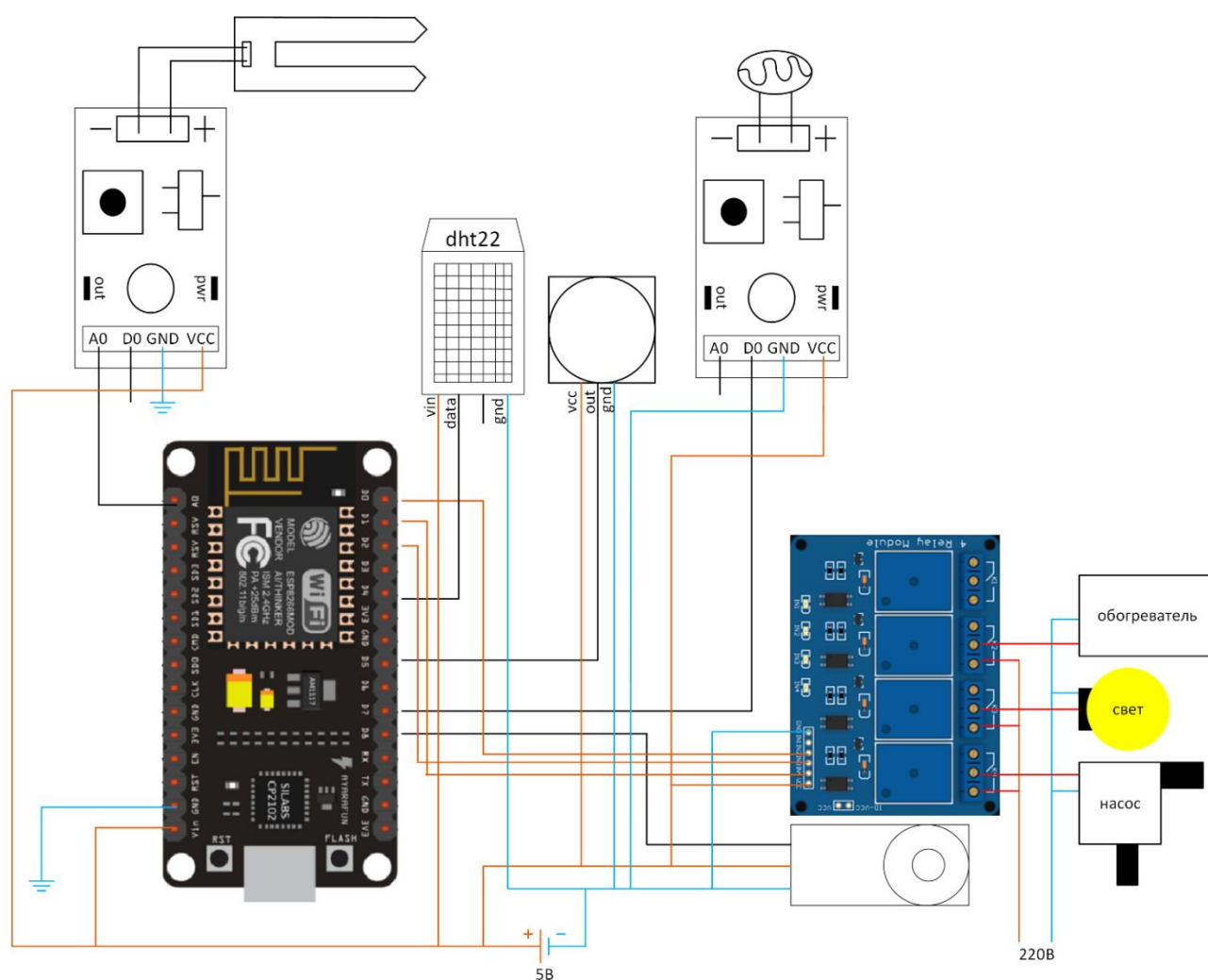


Рисунок 14. Схема соединения необходимых компонентов

Собранная схема помещена в корпус, который располагается в теплице. На рисунках 15 – 18 показан макет теплицы.



Рисунок 15 Передняя часть макета



Рисунок 16. Макет, общий вид

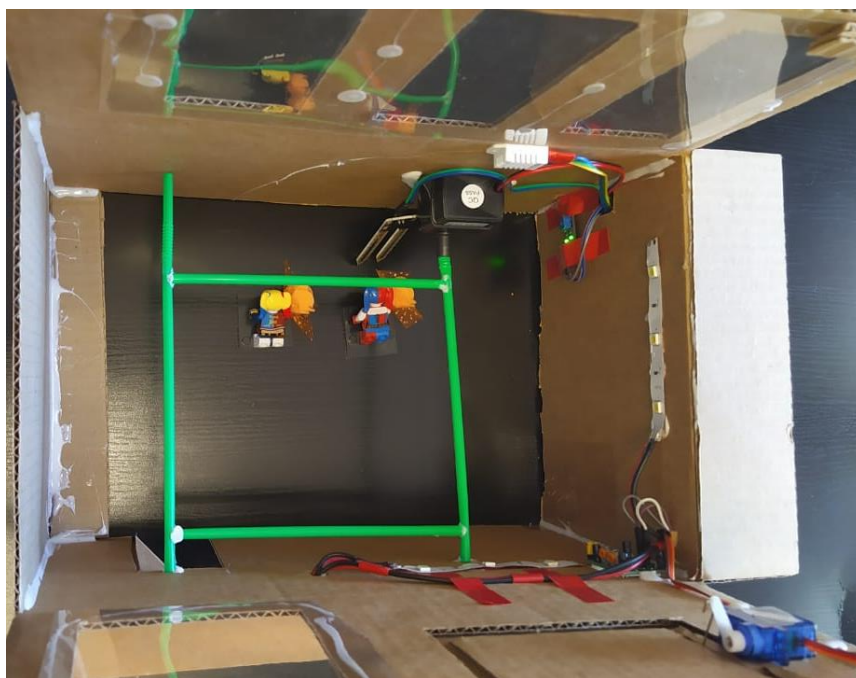


Рисунок 17. Макет, вид сверху

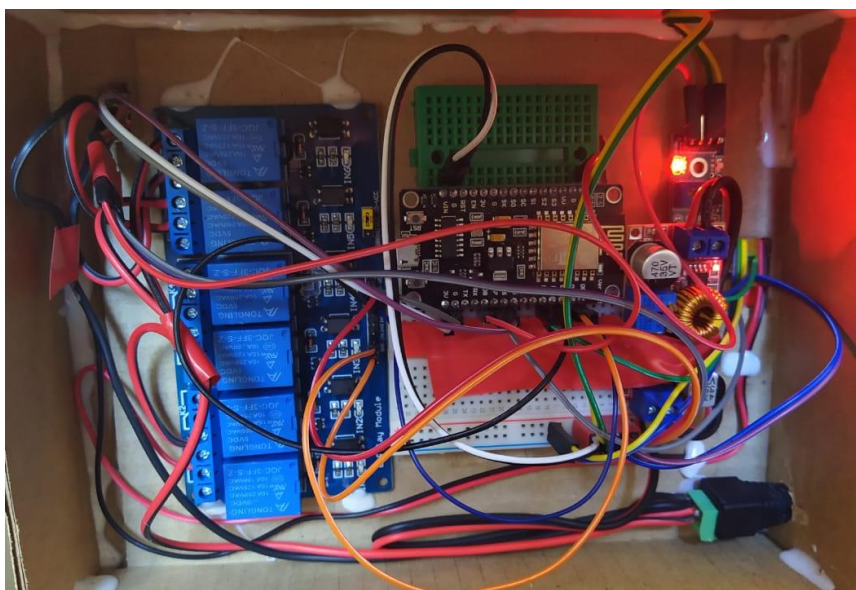


Рисунок 18. Схема внутри макета

Произведена сборка макета теплицы. Блок управления помещен в небольшую коробочку, которая устанавливается в теплице. Разведены провода для датчиков. В макет установлены все необходимые компоненты. Далее необходимо написать код и загрузить его в микроконтроллер.

Плата NodeMCU поддерживает несколько языков программирования, такие как C, Lua, MicroPython, JavaScript, Basic, Lisp для работы с ESP8266. В проекте используется C в программной среде Arduino IDE. Она является удобным инструментом, как для новичков, так и для тех, кто уже имеет опыт в данной программе.

Для начала необходимо установить среду разработки Arduino IDE, для этого переходим на сайт [arduino.cc](https://arduino.cc/en/main/software), он показан на рисунке 19, скачиваем инсталлятор и производим установку программы.



Рисунок 19. Официальный сайт проекта Arduino

Запускаем среду разработки и переходим в меню *Файл – Настройки* и вставляем в строку «Дополнительные ссылки для Менеджера плат» ссылку

http://arduino.esp8266.com/stable/package_esp8266com_index.json.

Настройки

показаны на рисунке 20.

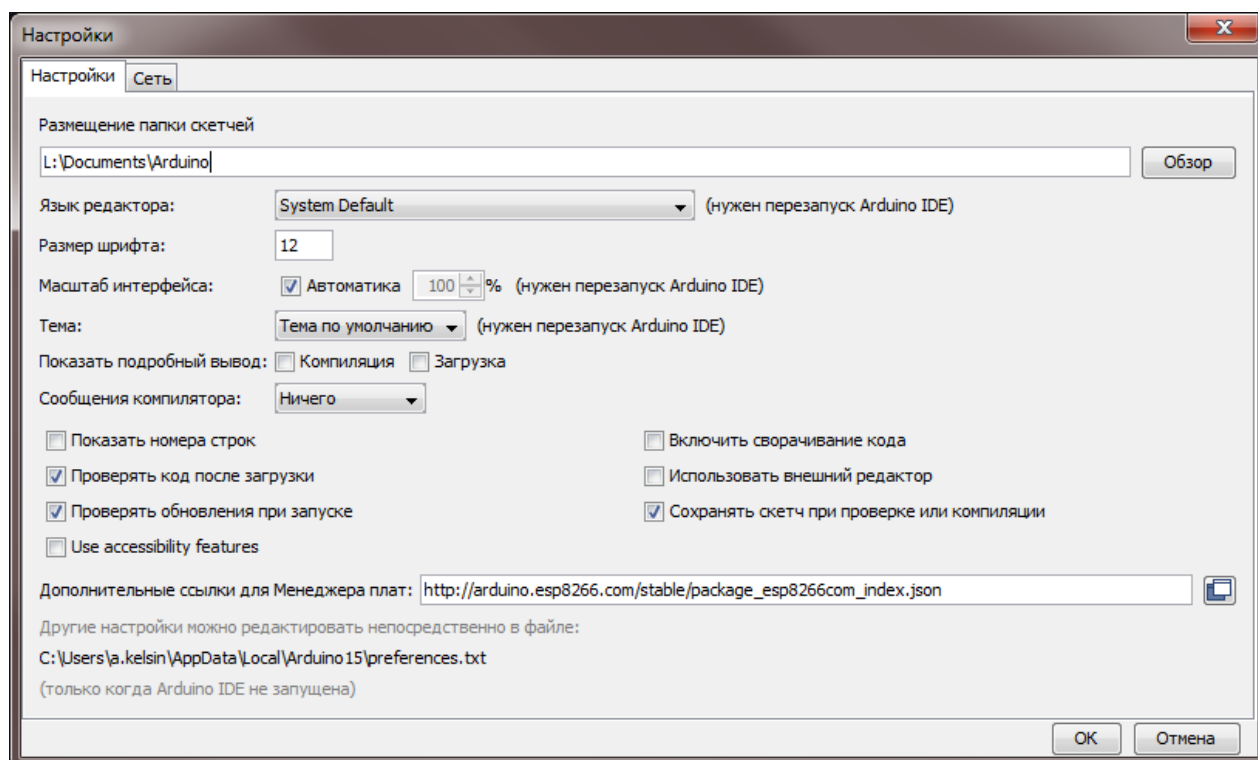


Рисунок 20. Настройки Arduino IDE

Далее переходим в менеджер плат и вводим в строку поиска esp8266, на выбираем пункт «esp8266 by ESP8266 Community» версии 2.6.2, и нажимаем «установка». Далее окно должно выглядеть так, как показано на рисунке 21.

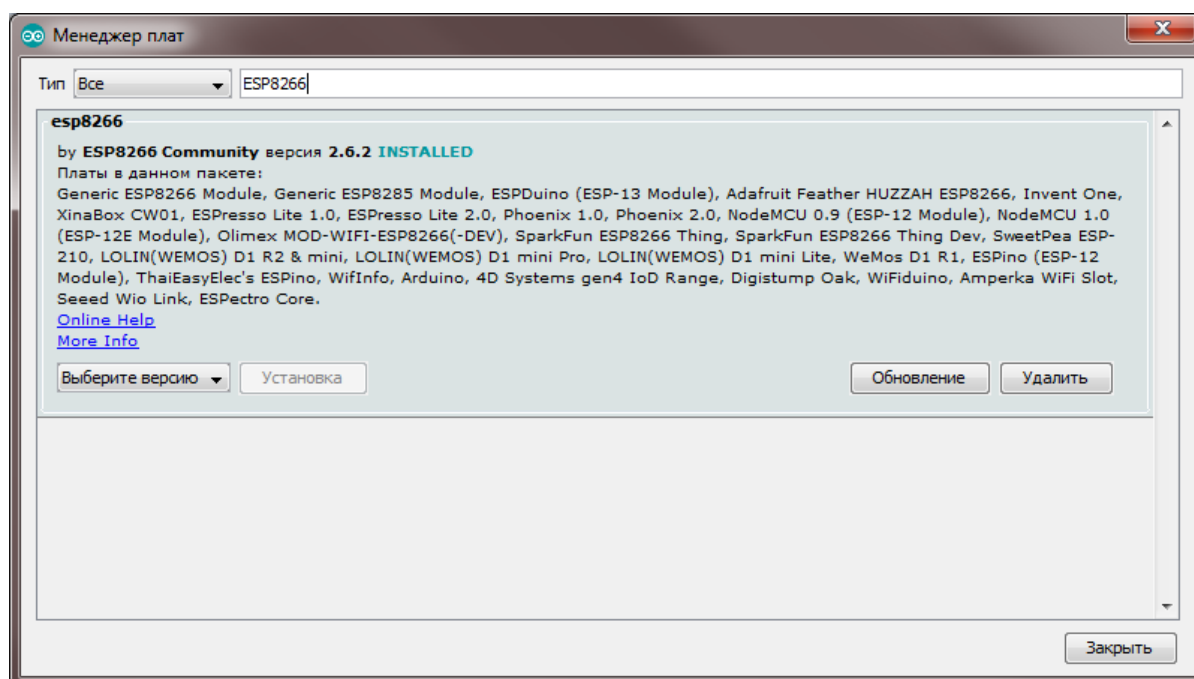


Рисунок 21. Менеджер плат

Далее переходим в *Инструменты* и выбираем нашу плату «NodeMCU». Настройки платы NodeMCU показаны на рисунке 22.

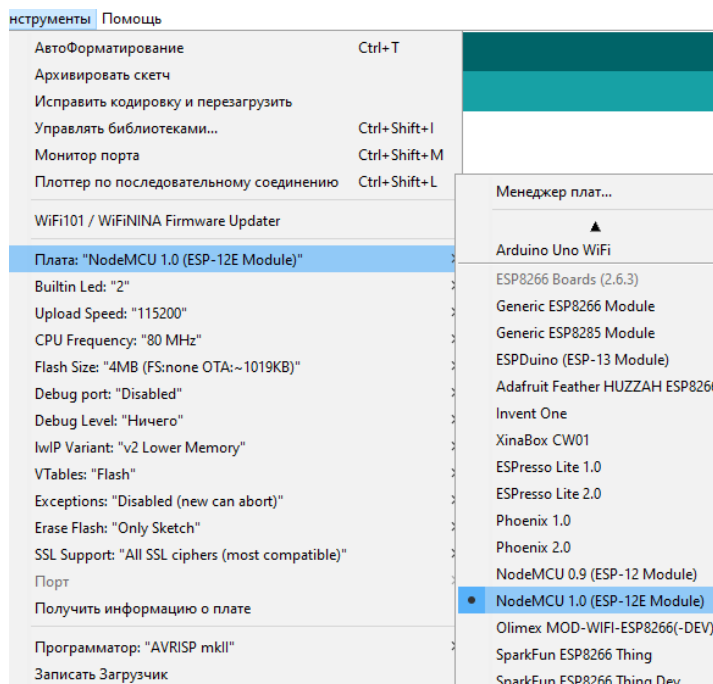


Рисунок 22. Настройки платы NodeMCU

Далее загружаем необходимые для работы библиотеки, заходим в программу Arduino IDE, далее нажимаем *Скетч - Подключить библиотеку - Добавить .ZIP библиотеку....* и далее выбираем необходимую библиотеку, рисунок 23. Также это можно сделать вручную, поместив все библиотеки в папку *libraries* в папке с установленной программой или же в документах пользователя.

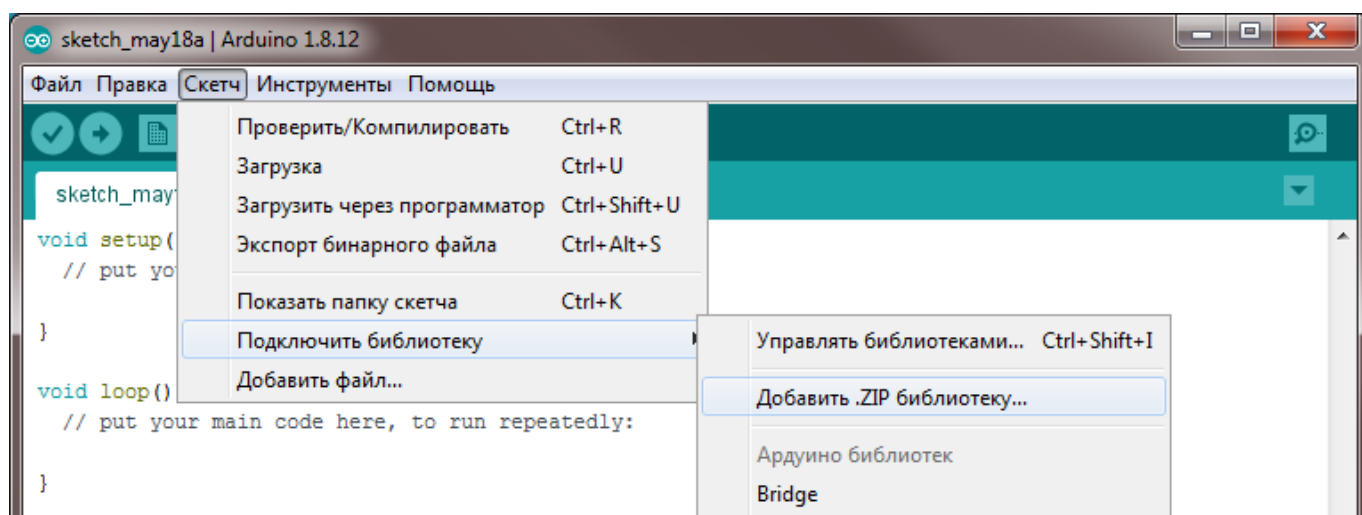


Рисунок 23. Подключение библиотеки

Для анализа данных поступающих с датчиков и управления нагрузкой в зависимости от значений этих данных пишем код, который будет следовать по

алгоритму. Алгоритм действий заключается в том, что будет выбор ручного и автоматического режимов.

В ручном режиме пользователь сам решает, что включить, а что выключить. В автоматическом режиме происходит определение дня и ночи, в зависимости от которых происходит выполнение определённого сценария действий. Также снимаются показатели температуры, влажности почвы и влажности воздуха, что в свою очередь тоже приводит к определенным действиям (включить обогреватель, открыть форточку, полить, включить свет).

Если для автоматической работы теплицы достаточно только включить плату, то для управления в ручном режиме необходим веб-интерфейс, с помощью которого пользователь будет взаимодействовать с теплицей, также это необходимо в случае обрыва связи и недоступности основного узла управления, для подключения напрямую без использования протокола MQTT.

На ESP8266 есть возможность настроить небольшой веб-сервер, который мы можем использовать, чтобы организовать связь между пользователем и теплицей, для этого напишем код, но есть пару нюансов.

К примеру, нам необходимо включить обогреватель на 15 минут, мы можем использовать оператор `delay()`, но он задерживает выполнение программы на указанный промежуток времени, и пока это время не истечёт, следующие команды программы не будут выполняться, и доступ к веб-интерфейсу в этот момент не получим. Из-за этого мы не можем задать разную длительность выполнения для каждой задачи в цикле `loop()` программы.

Так как на микроконтроллере нет возможности параллельного выполнения задач, и все они выполняются последовательно друг за другом, что не всегда бывает удобно. В нашем случае веб-сервер должен быть всегда готов ответить на запрос данных, и чтобы нагрузка не сходила с ума с бесконечным включением и выключением, при пороговых значениях, полученных с датчиков, будем использовать псевдопараллельное потоковое выполнение задач, что позволит задать промежуток времени, через который будет выполняться конкретная задача.

Во всех микроконтроллерах имеется таймер, включая и NodeMCU. Таймер – это устройство, которое умеет точно отсчитывать время. Запускается и обнуляется он каждый раз во время подачи питания на контроллер. Чтобы узнать, сколько времени прошло на таймере с момента запуска, используется функция `millis`. Суть в проверке значения `millis`, если значение больше, чем время, через которое должна выполняться функция, то записать новое время в переменную, прибавить к нему время через, сколько должна выполняться функция и выполнить функцию. Если значение `millis` меньше заданного периода, то просто записать новое время в переменную. Но данный способ подразумевает писать под каждую задачу проверку, код которой занимает значительный объём, поэтому будем использовать библиотеку `Thread.h`, которая позволяет значительно сократить код, используя псевдопараллельные потоки [7].

В теле программы создаем 4 псевдопараллельных потока для основной функции, которая поддерживает температуру в помещении, для функции управления полива, функции проверки подключения к серверу и функция, отвечающая за считывание данных с датчиков и их публикацию.

```
Thread mainnightThread = Thread();
```

```
Thread nasosThread = Thread();
```

```
Thread reconnectThread = Thread();
```

```
Thread datchikiThread = Thread();
```

Назначаем главному потоку функцию, которую он будет вызывать, и задаем интервал срабатывания в мсек. Аналогично задается для остальных потоков.

```
mainnightThread.onRun(mainnight);
```

```
mainnightThread.setInterval(10000);
```

Далее с помощью строчек ниже подписываемся или переподписываемся на топики, можно подписаться не только на один, а на несколько сразу. Получая данные с данных топики, от mqtt-брокера, осуществляется управление нагрузкой, с помощью централизованного управления.

```
client.subscribe("espmain/auto_main");
```

```
client.subscribe("espmain/svet_main");
```



```
client.subscribe("espmain/nasos_main");  
client.subscribe("espmain/okno_main");  
client.subscribe("espmain/obogrev_main");
```

Чтобы опубликовать данные, полученные от датчиков, используется конструкция показанная ниже.

```
client.publish("/espmain/temperature", temperatureTemp);  
client.publish("/espmain/humidity", humidityTemp);
```

Более подробно с листингом кода для микроконтроллера можно ознакомиться в приложении 5. В приложении 3 представлена блок-схема, которая показывает алгоритм автоматического режима работы.

Далее загружаем наисанный код в плату NodeMCU. После окончания загрузки плата подключается к Wi-Fi сети, которая указана в коде, и получает свой ip-адрес, если настроена автоматическая выдача сетевых реквизитов на роутере, по которому необходимо пройти в любом браузере с любого доступного устройства и получить доступ к панели управления нашей платы NodeMCU. Также на плате можно настроить режим «точки доступа», где выдачей сетевых реквизитов будет распоряжаться уже сама плата, то есть подключение будет осуществляться без посредников.

Далее наша задача состоит из трех уровней. На нижнем уровне находятся датчики и контроллеры. Середина – это место брокера MQTT, причем он должен быть расположен локально, чтобы проблемы с доступом к интернет можно было частично решить внутренним оборудованием. А верхний уровень – облако с интерфейсом. В такой концепции очень подходит энергоэффективный и бюджетный микро-сервер на основе ARM.

В нашем случае используется Raspberry Pi, которая будет совмещать в себе второй и третий уровень, то есть содержать в себе брокер и приложение для управления теплицами, сбора и хранения данных.

Согласно техническому заданию, серверная часть будет разворачиваться на физическом сервере, на базе операционной системы Raspbian. Также дополнительно потребуется установить и сконфигурировать следующее программное обеспечение:

- Apache2.4, Python 2.7 с дополнительными библиотеками, Flask, MariaDB, phpMyAdmin – компоненты веб-сервера;

- Mosquitto – mqtt-брокер сообщений;

- OpenSSH – сервер удаленного доступа по протоколу ssh.

На миникомпьютере необходимо установить операционную систему и провести настройку необходимого программного обеспечения.

Для установки операционной системы на microSD-карту, необходимо скачать образ Raspbian с графическим интерфейсом с официального сайта. Распаковываем архив с файлом, имеющий расширение img. Вставляем microSD-карту в компьютер. Для загрузки образа необходимо использовать бесплатную программу Win32 Disk Imager. Для этого загружаем с официального сайта программу, устанавливаем и запускаем ее. Далее в строке «Image File» указываем путь до образа, в «Device» выбираем нужное устройство и жмем «Write». Процесс записи образа показан на рисунке 24.

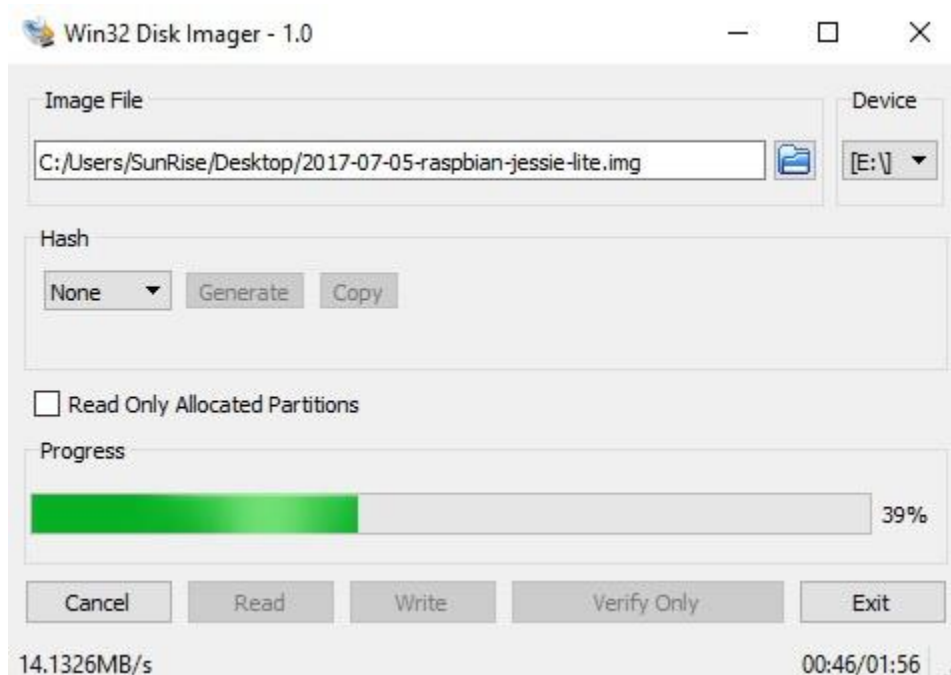


Рисунок 24. Интерфейс программы

После успешной записи образа устанавливаем microSD-карту в Raspberry Pi, подключаем необходимые кабели [12]. Raspberry Pi включается автоматически, как только на нее подается напряжение питания. На этом установка системы завершена.

По умолчанию логин - pi и пароль - raspberry. Рабочий стол Raspbian показан на рисунке 25.

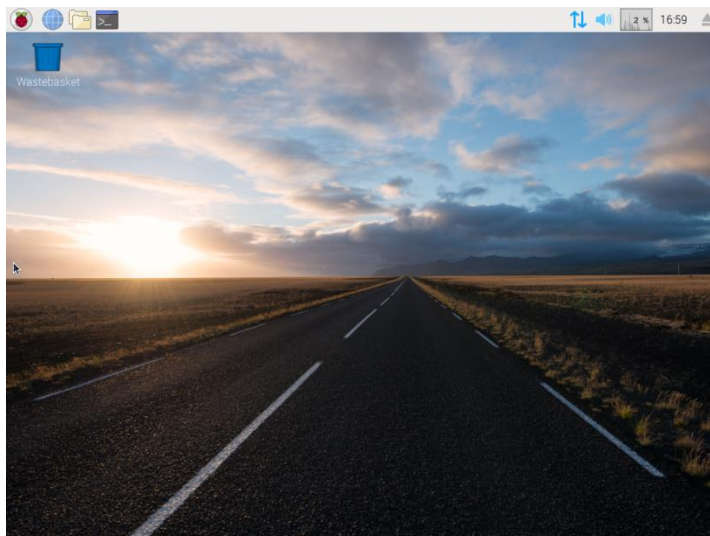


Рисунок 25. Рабочий стол Raspbian

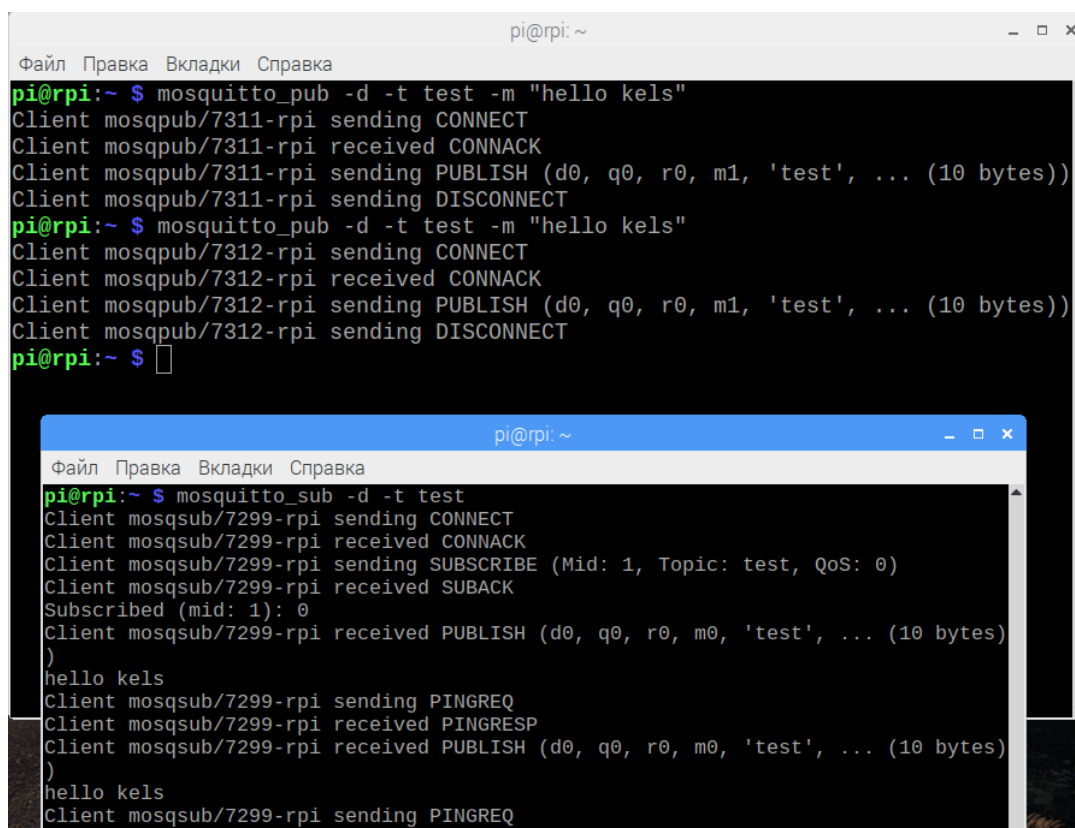
Далее производим установку Mosquitto – это брокер сообщений с открытым исходным кодом, который реализует протоколы MQTT версий 5.0, 3.1.1 и 3.1, рисунок 26. Mosquitto лёгкий и подходит для использования на всех устройствах: от маломощных одноплатных компьютеров до полноценных серверов.

```
pi@pi: ~  
Файл Правка Вкладки Справка  
pi@pi:~$ sudo apt-get install -y mosquitto mosquitto-clients  
Чтение списков пакетов... Готово  
Построение дерева зависимостей  
Чтение информации о состоянии... Готово  
Следующий пакет устанавливался автоматически и больше не требуется:  
  realpath  
Для его удаления используйте «sudo apt autoremove».  
Будут установлены следующие дополнительные пакеты:  
  libev4 libmosquitto1 libwebsockets8  
Предлагаемые пакеты:  
  arparmor  
НОВЫЕ пакеты, которые будут установлены:  
  libev4 libmosquitto1 libwebsockets8 mosquitto mosquitto-clients  
обновлено 0, установлено 5 новых пакетов, для удаления отмечено 0 пакетов, и 351  
пакетов не обновлено.  
Необходимо скачать 341 кБ архивов.  
После данной операции, объём занятого дискового пространства возрастёт на 765 кБ  
.  
Пол:1 http://archive.raspberrypi.org/debian stretch/main armhf libwebsockets8 ar  
mhf 2.0.3-2+b1-rpt1 [85,2 kB]  
Пол:2 http://mirror.yandex.ru/mirrors/raspbian/raspbian stretch/main armhf libev  
4 armhf 1:4.22-1 [34,0 kB]  
Пол:3 http://mirror.yandex.ru/mirrors/raspbian/raspbian stretch/main armhf libmo  
squitto1 armhf 1.4.10-3+deb9u4 [47,2 kB]
```

Рисунок 26. Установка брокера Mosquitto

Проведем проверку работы брокера, для этого откроем два окна терминала. В первом окне подписываемся на топик *test*, с помощью команды *mosquitto_sub -d -t*

test. Во втором окне публикуем данные в топик *test* командой *mosquitto_pub -d -t test -m "hello kels"*. Данные успешно получены, что показано на рисунке 27.



The image shows two overlapping terminal windows on a Raspberry Pi. The top window shows the execution of `mosquitto_pub` twice, each time sending a message to the `test` topic. The bottom window shows the execution of `mosquitto_sub`, which subscribes to the `test` topic and receives the messages sent by the publisher. The messages received are `hello kels`.

```
pi@rpi: ~  
Файл Правка Вкладки Справка  
pi@rpi:~ $ mosquitto_pub -d -t test -m "hello kels"  
Client mosqpub/7311-rpi sending CONNECT  
Client mosqpub/7311-rpi received CONNACK  
Client mosqpub/7311-rpi sending PUBLISH (d0, q0, r0, m1, 'test', ... (10 bytes))  
Client mosqpub/7311-rpi sending DISCONNECT  
pi@rpi:~ $ mosquitto_pub -d -t test -m "hello kels"  
Client mosqpub/7312-rpi sending CONNECT  
Client mosqpub/7312-rpi received CONNACK  
Client mosqpub/7312-rpi sending PUBLISH (d0, q0, r0, m1, 'test', ... (10 bytes))  
Client mosqpub/7312-rpi sending DISCONNECT  
pi@rpi:~ $  
  
pi@rpi: ~  
Файл Правка Вкладки Справка  
pi@rpi:~ $ mosquitto_sub -d -t test  
Client mosqsub/7299-rpi sending CONNECT  
Client mosqsub/7299-rpi received CONNACK  
Client mosqsub/7299-rpi sending SUBSCRIBE (Mid: 1, Topic: test, QoS: 0)  
Client mosqsub/7299-rpi received SUBACK  
Subscribed (mid: 1): 0  
Client mosqsub/7299-rpi received PUBLISH (d0, q0, r0, m0, 'test', ... (10 bytes))  
)  
hello kels  
Client mosqsub/7299-rpi sending PINGREQ  
Client mosqsub/7299-rpi received PINGRESP  
Client mosqsub/7299-rpi received PUBLISH (d0, q0, r0, m0, 'test', ... (10 bytes))  
)  
hello kels  
Client mosqsub/7299-rpi sending PINGREQ
```

Рисунок 27. Проверка работы брокера Mosquitto

Для работы сайта необходимо установить выбранный нами веб-сервер, СУБД и программу для удобного администрирования баз данных через графический интерфейс, а не в терминале.

Сначала мы установим все необходимые компоненты, а затем будем настраивать их и проверять правильность работы по мере необходимости. Но сначала нужно обновить систему до последней версии:

```
sudo apt-get update && sudo apt-get upgrade
```

Далее установим веб-сервер и интерпретатор языка программирования PHP:

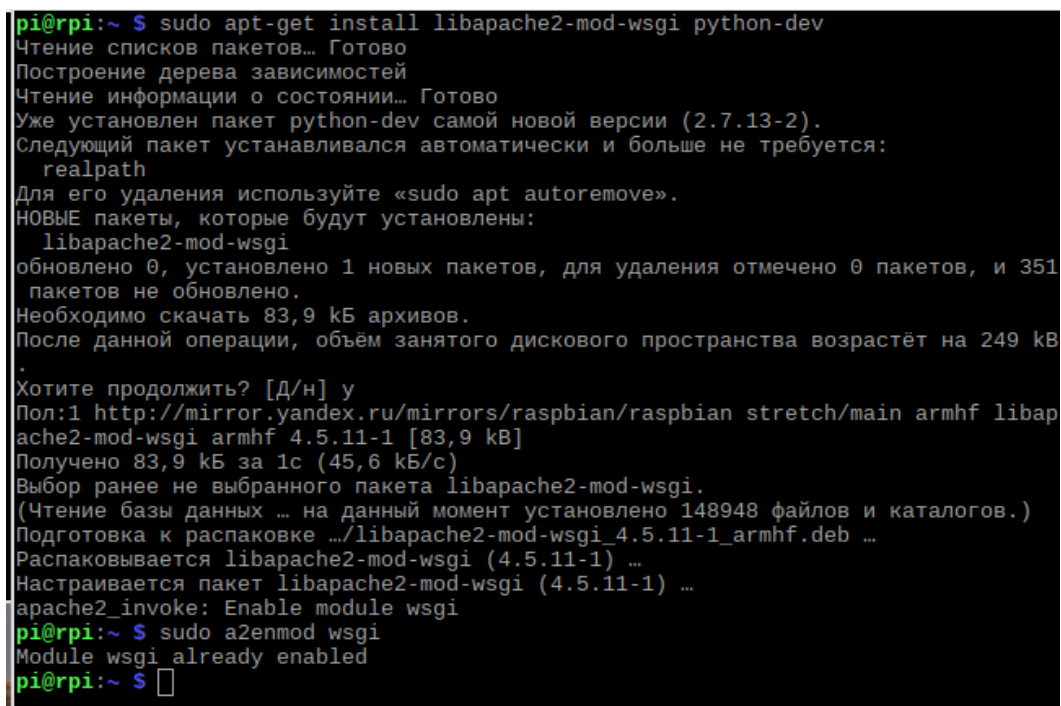
```
sudo apt-get install apache2 php php-mysql libapache2-mod-php php-mbstring php-zip php-gd
```

Произведена установка не только веб-сервера, но и основных библиотек PHP, которые могут понадобиться для работы с phpMyAdmin. Пакет `libapache2-php` отвечает за интеграцию поддержки php для веб-сервера Apache.

Следующим установим систему управления базами данных – MariaDB. Для ее установки необходимо выполнить команду:

```
sudo apt-get install mariadb-server mariadb-client mariadb-common
```

В качестве основного языка программирования, на котором будет написан сайт, выбран Python, уже встроенный в Raspbian. Использоваться будет версия Python 2.7, поэтому устанавливаем `mod_wsgi` – модуль для веб-сервера Apache, который предоставляет WSGI-совместимый (англ. Web Server Gateway Interface) интерфейс для работы с web-приложениями, написанными на языке Python. Совместим с версиями языка Python 2 и Python 3. Позволяет запускать web-приложения в двух режимах: режиме встраивания, при котором используются одинаковые идентификаторы пользователя и группы (`uid` и `gid` соответственно) для всех использующих модуль приложений, а также в режиме демона (доступен для Apache 2.x) – где можно задать пользователя, от имени которого будет запускаться каждое приложение, что повышает безопасность в случае общего хостинга для разных пользователей. В нашем случае используем второй вариант. Установка данного модуля показана на рисунке 28 [14].

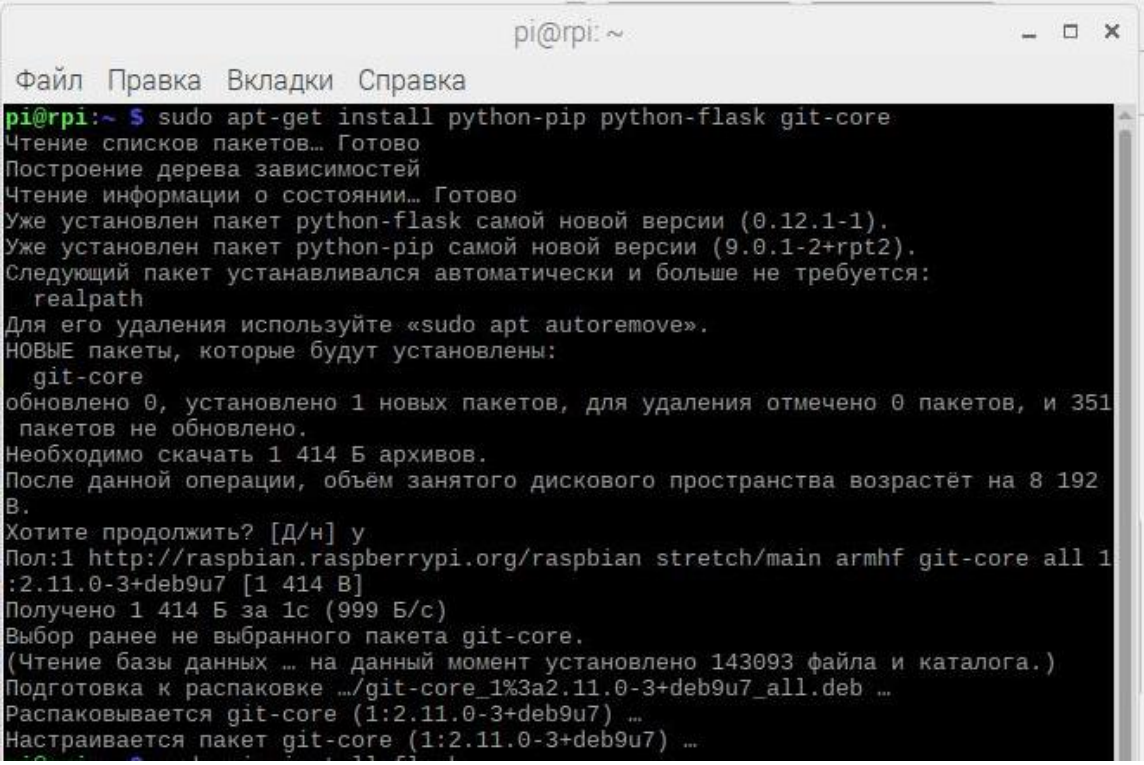


```
pi@rpi:~ $ sudo apt-get install libapache2-mod-wsgi python-dev
Чтение списков пакетов... Готово
Построение дерева зависимостей
Чтение информации о состоянии... Готово
Уже установлен пакет python-dev самой новой версии (2.7.13-2).
Следующий пакет устанавливался автоматически и больше не требуется:
  realpath
Для его удаления используйте «sudo apt autoremove».
НОВЫЕ пакеты, которые будут установлены:
  libapache2-mod-wsgi
обновлено 0, установлено 1 новых пакетов, для удаления отмечено 0 пакетов, и 351
пакетов не обновлено.
Необходимо скачать 83,9 кБ архивов.
После данной операции, объём занятого дискового пространства возрастёт на 249 кБ
.
Хотите продолжить? [Д/н] y
Пол:1 http://mirror.yandex.ru/mirrors/raspbian/raspbian stretch/main armhf libap
ache2-mod-wsgi armhf 4.5.11-1 [83,9 kB]
Получено 83,9 кБ за 1с (45,6 кБ/с)
Выбор ранее не выбранного пакета libapache2-mod-wsgi.
(Чтение базы данных ... на данный момент установлено 148948 файлов и каталогов.)
Подготовка к распаковке .../libapache2-mod-wsgi_4.5.11-1_armhf.deb ...
Распаковывается libapache2-mod-wsgi (4.5.11-1) ...
Настраивается пакет libapache2-mod-wsgi (4.5.11-1) ...
apache2_invoke: Enable module wsgi
pi@rpi:~ $ sudo a2enmod wsgi
Module wsgi already enabled
pi@rpi:~ $
```

Рисунок 28. Установка `mod_wsgi`

Для реализации веб-интерфейса ранее выбран микрофреймворк Flask, установка которого показана на рисунке 29. Использовалась команда `sudo apt-get`

install python-pip python-flask git-core. Также установлен менеджер пакетов PIP – система управления пакетами, которая используется для установки и управления программными пакетами, написанными на Python.



```
pi@rpi: ~  
Файл Правка Вкладки Справка  
pi@rpi:~$ sudo apt-get install python-pip python-flask git-core  
Чтение списков пакетов... Готово  
Построение дерева зависимостей  
Чтение информации о состоянии... Готово  
Уже установлен пакет python-flask самой новой версии (0.12.1-1).  
Уже установлен пакет python-pip самой новой версии (9.0.1-2+rpt2).  
Следующий пакет устанавливался автоматически и больше не требуется:  
  realpath  
Для его удаления используйте «sudo apt autoremove».  
НОВЫЕ пакеты, которые будут установлены:  
  git-core  
обновлено 0, установлено 1 новых пакетов, для удаления отмечено 0 пакетов, и 351  
пакетов не обновлено.  
Необходимо скачать 1 414 Б архивов.  
После данной операции, объем занятого дискового пространства возрастет на 8 192  
Б.  
Хотите продолжить? [Д/н] у  
Пол:1 http://raspbian.raspberrypi.org/raspbian stretch/main armhf git-core all 1  
:2.11.0-3+deb9u7 [1 414 Б]  
Получено 1 414 Б за 1с (999 Б/с)  
Выбор ранее не выбранного пакета git-core.  
(Чтение базы данных ... на данный момент установлено 143093 файла и каталога.)  
Подготовка к распаковке .../git-core_1%3a2.11.0-3+deb9u7_all.deb ...  
Распаковывается git-core (1:2.11.0-3+deb9u7) ...  
Настраивается пакет git-core (1:2.11.0-3+deb9u7) ...
```

Рисунок 29. Установка Flask

Производим установку библиотеки *raho-mqtt*, необходимую для работы python с протоколом mqtt, из репозитория *pip*, с помощью команды *sudo pip install raho-mqtt*.

Далее для того, чтобы данные о среде в теплице и время их получения обновлялись на сайте в режиме реального времени, решено установить Socket.IO – JavaScript-библиотека для веб-приложений командой *sudo pip install flask-socketio*.

Так как для сохранения данных о среде будет использоваться СУБД, то было решено использовать библиотеку SQLAlchemy – это библиотека на языке Python для работы с реляционными СУБД с применением технологии ORM (Object-Relational Mapping), которая связывает базы данных с концепциями объектно-ориентированных языков программирования, таких как Python. SQLAlchemy позволяет описывать структуры баз данных и способы взаимодействия с ними прямо на языке Python. Установка Flask-SQLAlchemy-2.4.4 и SQLAlchemy версии

1.3.20 показана на рисунке 30, также устанавливаем дополнительную библиотеку PyMySQL для работы SQLAlchemy с СУБД MariaDB, рисунок 31.

```
pi@rpi:~/web-server $ sudo pip install Flask-SQLAlchemy
Collecting Flask-SQLAlchemy
  Downloading https://files.pythonhosted.org/packages/af/4b/3dd83c3e2c6e3034b804d971952e0120606506f153990cc56d2d33173b72/Flask_SQLAlchemy-2.4.4-py2.py3-none-any.whl
Collecting SQLAlchemy>=0.8.0 (from Flask-SQLAlchemy)
  Downloading https://files.pythonhosted.org/packages/69/ef/6d18860e18db68b8f25e0d268635f2f8cefa7a1cbf6d9d9f90214555a364/SQLAlchemy-1.3.20.tar.gz (6.3MB)
    100% |████████████████████████████████████████| 6.3MB 12kB/s
Requirement already satisfied: Flask>=0.10 in /usr/lib/python2.7/dist-packages (from Flask-SQLAlchemy)
Building wheels for collected packages: SQLAlchemy
  Running setup.py bdist_wheel for SQLAlchemy ... done
  Stored in directory: /root/.cache/pip/wheels/c2/8b/1a/a6f5fe6e4e63d156a1d32ee8a137292c3f2fea3f480bcd4c7
Successfully built SQLAlchemy
Installing collected packages: SQLAlchemy, Flask-SQLAlchemy
Successfully installed Flask-SQLAlchemy-2.4.4 SQLAlchemy-1.3.20
pi@rpi:~/web-server $
```

Рисунок 30. Установка SQLAlchemy

```
pi@rpi:~ $ pip install PyMySQL
Collecting PyMySQL
  Downloading https://files.pythonhosted.org/packages/1a/ea/dd9c81e2d85efd03cfbf808736dd055bd9ea1a78aea9968888b1055c3263/PyMySQL-0.10.1-py2.py3-none-any.whl (47kB)
    100% |████████████████████████████████████████| 51kB 395kB/s
Installing collected packages: PyMySQL
Successfully installed PyMySQL-0.10.1
pi@rpi:~ $
```

Рисунок 31. Установка PyMySQL

Далее создаем папку, где будет размещен сайт командой:

```
sudo mkdir /var/www/teplica.org/
```

Создаем пользователя flask, без домашнего каталога, и добавляем в группу, связанную с веб-сервером, www-data с помощью команды:

```
sudo nano useradd flask -M -g www-data -p password
```

Задаем виртуальный доменный адрес, по которому мы будем получать доступ к нашему сайту. Конфигурационные файлы виртуальных хостов хранятся в каталоге /etc/apache2/sites-available/ создаем новую конфигурацию *teplica.conf* командой:

```
sudo nano /etc/apache2/sites-available/teplica.conf
```

Теперь в полученном файле прописываем директивы, необходимые для работы сайта.

Задаем на каком порту будет работать сайт.

```
<VirtualHost *:80>
```

Задаем имя виртуального домена, по которому будет доступен сайт.

ServerName teplica.org

Прописываем путь до папки с сайтом.

DocumentRoot /var/www/teplica.org

Можно задать любой электронный адрес для отображения в ошибках.

ServerAdmin root@localhost

Далее задаем от какого пользователя, какой группы будет запускаться процесс с нашим приложением, а также количество потоков и домашний каталог.

*WSGIDaemonProcess teplica.org user=flask group=www-data threads
home=/var/www/teplica.org*

- threads – определяет кол-во потоков для обработки запросов по каждому из процессов демона внутри группы процессов, определенных директивой *WSGIProcessGroup*.

Задаем местоположение скрипта, обрабатывающего запросы к приложению.

WSGIScriptAlias / /var/www/teplica.org/teplica.py

Далее задаем параметры для директории корня сайта.

<Directory /var/www/teplica.org>

Группируем процессы демона:

WSGIProcessGroup teplica.org

Каждый сайт виртуального хоста Flask должен использовать отдельную группу процессов демона, поэтому группа приложений *%{GLOBAL}* вынуждает использовать контекст основного интерпретатора в соответствующих группах процессов.

WSGIApplicationGroup %{GLOBAL}

Чтобы облегчить работу инструментов установки, включаем поддержку автоматической перезагрузки, когда wsgi-скрипт изменится, *mod_wsgi* перезагрузит для нас все процессы демона.

WSGIScriptReloading On

Задаем разрешение на доступ к сайту.

Order allow,deny

Allow from all

</Directory>

Задаем путь до каталога static.

Alias /static /var/www/teplica.org/teplica/static

Задаем разрешения каталога static.

<Directory /var/www/teplica.org/teplica/static>

Order allow,deny

Allow from all

</Directory>

Далее прописываем, куда должны записываться ошибки и системные сообщения.

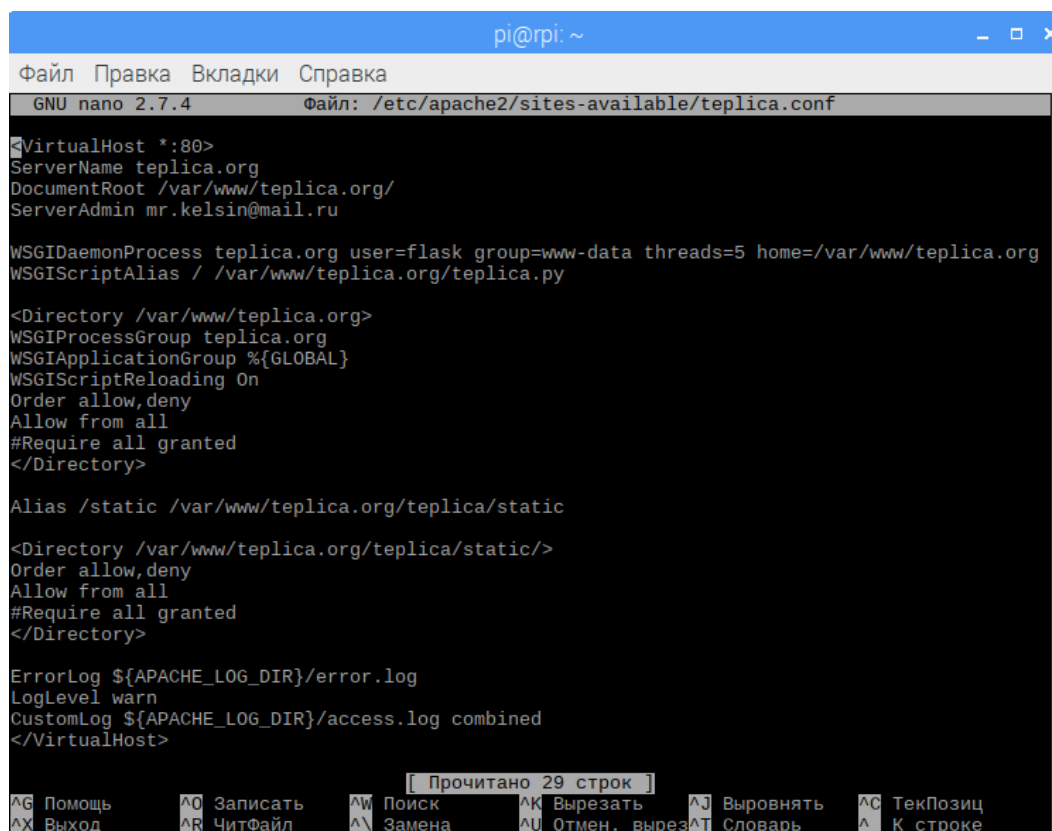
ErrorLog \${APACHE_LOG_DIR}/error.log

LogLevel warn

CustomLog \${APACHE_LOG_DIR}/access.log combined

</VirtualHost>

На рисунке 32 показан конфигурационный файл сайта teplica.org.



```
pi@pi: ~
Файл Правка Вкладки Справка
GNU nano 2.7.4 Файл: /etc/apache2/sites-available/teplica.conf
VirtualHost *:80>
ServerName teplica.org
DocumentRoot /var/www/teplica.org/
ServerAdmin mr.kelsin@mail.ru

WSGIDaemonProcess teplica.org user=flask group=www-data threads=5 home=/var/www/teplica.org
WSGIScriptAlias / /var/www/teplica.org/teplica.py

<Directory /var/www/teplica.org>
WSGIProcessGroup teplica.org
WSGIApplicationGroup %{GLOBAL}
WSGIScriptReloading On
Order allow,deny
Allow from all
#Require all granted
</Directory>

Alias /static /var/www/teplica.org/teplica/static

<Directory /var/www/teplica.org/teplica/static/>
Order allow,deny
Allow from all
#Require all granted
</Directory>

ErrorLog ${APACHE_LOG_DIR}/error.log
LogLevel warn
CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>

[ Прочитано 29 строк ]
^G Помощь ^O Записать ^W Поиск ^K Вырезать ^J Выводить ^C ТекПозиц
^X Выход ^R ЧитФайл ^\ Замена ^U Отмен. вырез ^T Словарь ^_ К строке
```

Рисунок 32. Конфигурационный файл

Затем создаем WSGI-скрипт в корневой папке сайта, с помощью команды.

```
sudo nano /var/www/teplica.org/teplica.py
```

В созданном файле прописываем параметры, указанные ниже.

Указываем, что используется python.

```
#!/usr/bin/python
```

Импортируем необходимые библиотеки.

```
import os, sys, flask
```

```
import logging
```

Указываем, что для записи логов, будет использоваться основной поток.

```
logging.basicConfig(stream=sys.stderr)
```

Указываем, что первым делом необходимо импортировать папку, в которой будут находиться основные файлы приложения.

```
sys.path.insert(0, "/var/www/teplica.org/teplica")
```

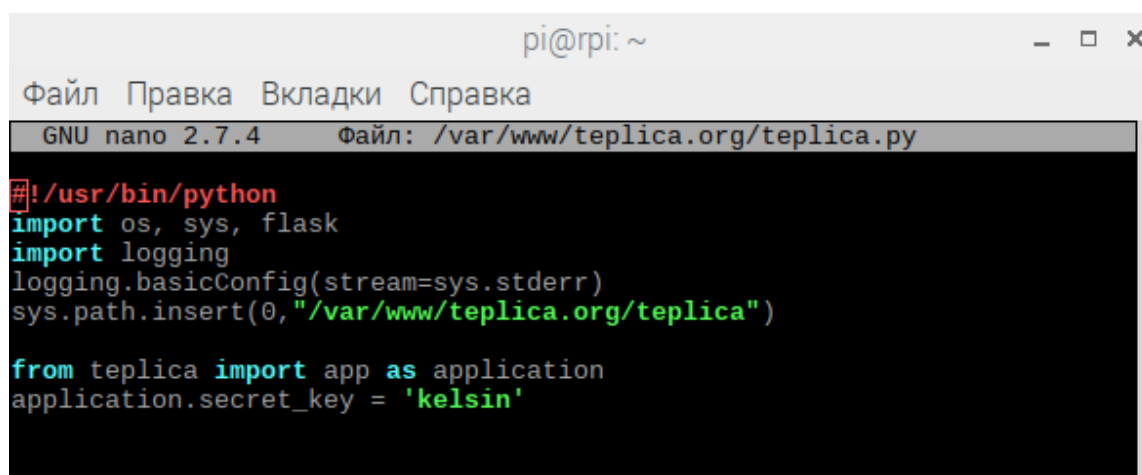
Следом указываем, что необходимо импортировать наш сайт как приложение.

```
from teplica import app as application
```

Задаем для приложения секретный ключ.

```
application.secret_key = 'kelsin'
```

На рисунке 33 показан wsgi-скрипт сайта teplica.org.



```
pi@pi: ~
Файл  Правка  Вкладки  Справка
GNU nano 2.7.4  Файл: /var/www/teplica.org/teplica.py
#!/usr/bin/python
import os, sys, flask
import logging
logging.basicConfig(stream=sys.stderr)
sys.path.insert(0, "/var/www/teplica.org/teplica")

from teplica import app as application
application.secret_key = 'kelsin'
```

Рисунок 33. Файл teplica.py

После проделанных действий, создаем в корне сайта каталог teplica, в котором в свою очередь помещаем каталоги templates, где создаем пустые файлы html страничек, и static, внутри которого помещаем статические файлы, такие как

картинки и стили страничек, а также файл `__init__.py`, в котором будет прописана логика приложения. На рисунке 34 показано получившееся дерево каталогов и файлов.

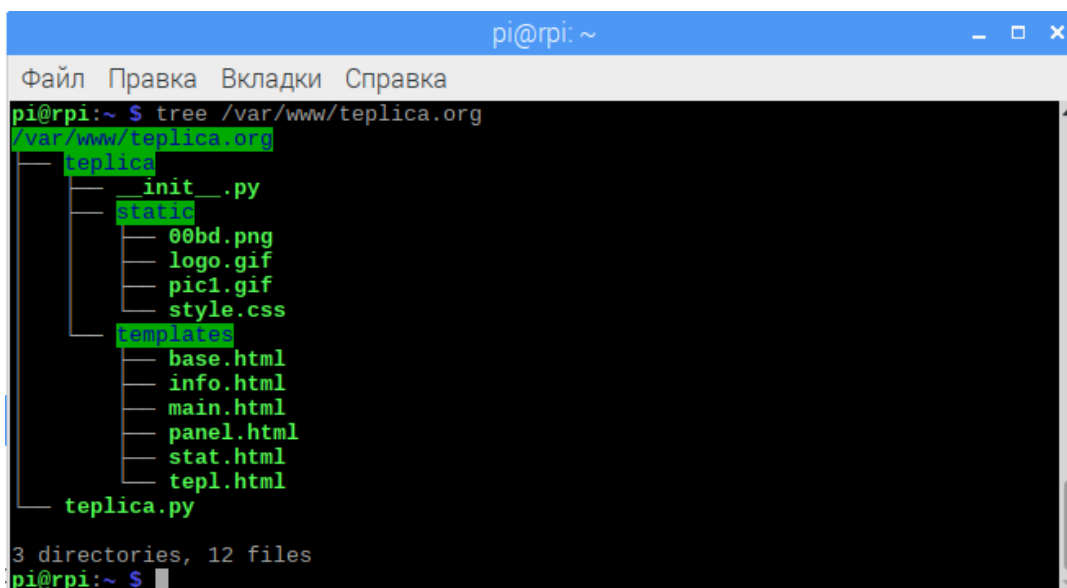


Рисунок 34. Дерево каталогов и файлов

Далее необходимо прописать код в файле `__init__.py`, который загружается самым первым и содержит логику, по которой приложение будет работать.

В файле подключаем все нужные библиотеки. Создаем подклассы, обращающиеся к объектам классов для управления данными в таблицах БД, с помощью SQLAlchemy, поэтому писать запрос на чистом sql не нужно, что упрощает написание и читабельность кода [34].

```
class Temper(db.Model):
```

Указываем с какой таблицей работать.

```
__tablename__ = 'temperatura'
```

Указываем поля в таблице.

```
id_temp = db.Column(db.Integer(), primary_key=True)
```

```
board = db.Column(db.Text, nullable=False)
```

```
date = db.Column(db.DateTime(), default=datetime.utcnow)
```

```
temp = db.Column(db.Float, nullable=False)
```

`__repr__` же вызывается при операторе print, для удобства отображения.

```
def __repr__(self):
```

```
    return "<{}:{},{}:{}>".format(self.id_temp, self.date, self.board, self.temp)
```

Далее написана функция, которая принимает имя микроконтроллера и температуру и загружает их в базу.

```
def insert_temp(board, temperature):
```

Использована конструкция try-except, которая пытается выполнить код, и в случае неудачи, выполняет код после except.

```
try:
```

Задаем переменную с простым запросом, так как используется SQLAlchemy.

```
slqinsert = Temper(board=board, temp=temperature)
```

Далее запрос добавляется в сессию работы с базами данных. SQLAlchemy автоматически производит действия с сессия

```
db.session.add(slqinsert)
```

Применяем внесенные изменения в базы данных.

```
db.session.commit()
```

Задаем переменную и выводим, что всё хорошо.

```
message = "OK mysql"
```

```
print message
```

Задаем переменную и выводим, если произойдет ошибка.

```
except:
```

```
message = "Problems with mysql in temp"
```

```
print message
```

В созданной функции on_connect подписываемся на топики, данные которых будут поступать с микроконтроллеров ESP8266. Далее функция on_message принимает данные с топиков, на которые подписан клиент и отвечает за их обработку. Следующим задали параметры для работы MQTT клиента и задаем, что ранее созданные функции отвечают за обработку данных.

Современные веб-приложения используют «красивые» URL, с помощью маршрутизации, поэтому в приложении используется декоратор route(), который необходим для привязки функции к URL. Это помогает людям запомнить эти URL, чтобы пользователь мог перейти сразу на желаемую страницу, без предварительного посещения начальной страницы. Задаем маршрутизацию для главной страницы.

Внутри Flask уже автоматически преднастроен шаблонизатор Jinja2. Поэтому для визуализации шаблона используется метод `render_template()`. Всё, что необходимо - это указать имя шаблона, а также переменные в виде именованных аргументов, которые необходимо передать движку обработки шаблонов.

```
@app.route("/")
def main():
    return render_template('main.html')
```

Задаем маршрутизацию для страницы с данными о температуре и влажности конкретной теплицы. Чтобы добавлять к адресу URL переменные части, можно эти особые части выделить в `<variable_name>`. Затем подобные части передаются в функцию в качестве аргумента – в виде ключевого слова.

```
@app.route("/stat/<board>", methods=['post','get'])
```

Поскольку на странице, с данными о температуре и влажности, реализована форма, с помощью которой можно задавать параметры вывода данных, то добавляем проверку, поступали ли данные из формы или нет. Параметры можно задавать по одному, благодаря реализованной проверке.

```
if request.method == 'POST':
    if request.form.get('rows') is None:
        rows = 100
    else:
```

Возводим в модуль и округляем количество строк.

```
rows = abs(int(request.form.get('rows')))
calendar = request.form.get('calendar')
```

Далее выполняем два запроса в соответствии с заданными параметрами из формы.

```
sqltemp =
db.session.query(Temper).order_by(db.desc(Temper.date)).filter(Temper.board==board,
Temper.date.startswith(calendar)).limit(rows).all()
```

```
sqlhum =
```

```
db.session.query(Humidity).order_by(db.desc(Humidity.date)).filter(Humidity.board==board, Humidity.date.startswith(calendar)).limit(rows).all()
```

Задаем маршрутизацию для страницы с панелью управления нагрузкой теплиц, при нажатии на кнопку.

```
@app.route("/<board>/<changePin>/<action>")
```

```
def action(board, changePin, action):
```

Переводим переменную в строковый тип данных.

```
changePin = str(changePin)
```

Присваиваем имя платы из словаря в соответствии с принятым changePin.

```
devicePin = pins[changePin]['name']
```

Далее производим проверку, на какой плате и в какое состояние перевести нагрузку.

```
if action == "1" and board == 'espmain' or action == "1" and board == 'espone':
```

Публикуем данные в нужный топик.

```
mqttc.publish(pins[changePin]['topic'], "1")
```

Меняем значение о состоянии в словаре pins.

```
pins[changePin]['state'] = 'True'
```

```
if action == "0" and board == 'espmain' or action == "0" and board == 'espone':
```

```
mqttc.publish(pins[changePin]['topic'], "0")
```

```
pins[changePin]['state'] = 'False'
```

```
templateData = {
```

```
    'pins' : pins
```

```
}
```

```
return render_template('panel.html', async_mode=socketio.async_mode, **templateData)
```

Полный листинг файла `__init__.py` с комментариями представлен в приложении 4. Далее создаем файлы шаблонов сайта. Файл `base.html` – это основной

шаблон сайта, который с помощью микрофреймворка flask будет совмещаться с другими страничками сайта и отдаваться пользователю как полноценная страница. Параметры `{% block content %}{% endblock %}` указывают, что в данном месте будет находиться содержимое другой html-страницы. В других же файлах содержится следующая структура:

```
{% extends 'base.html' %}  
{% block content %}  
    html код  
{% endblock content %},
```

что указывает на то, что необходимо добавить содержимое файла `base.html`. Данный способ очень эффективен, при редактировании, а также уменьшает количество файлов и строк кода. Полный листинг основных файлов сайта можно увидеть в приложении 6 [6].

Дальше сохраняем изменения и активируем сайт командой:

```
sudo a2ensite teplica.conf
```

Теперь осталось перезапустить Apache с помощью команды:

```
sudo service apache2 restart
```

В качестве СУБД используем MariaDB. Чтобы начать ее настройку выполняем в командной строке:

```
mysql_secure_installation
```

Далее будет предложено задать новый пароль для СУБД, затем удалить анонимных пользователей, удалить тестовую базу данных, перезагрузить все права на доступ к базе данных – соглашаемся со всем.

После того как процесс первоначальной настройки завершится, проверяем работоспособность СУБД с ранее созданным паролем:

```
sudo mysql -u root -p
```

На рисунке 35 показан успешный результат входа.

```
pi@rpi:~ $ sudo mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 175
Server version: 10.1.37-MariaDB-0+deb9u1 Raspbian 9.0

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> █
```

Рисунок 35. Результат входа

Для удобства администрирования базы данных произведем установку phpMyAdmin – это свободное программное обеспечение, предназначенное для работы с MySQL (MariaDB) в сети. Данная программа позволяет использовать удобный внешний интерфейс для MySQL (MariaDB). Можно загрузить с сайта разработчика пакет данного программного обеспечения, но поскольку данный пакет добавлен в репозиторий нашей операционной системы, воспользуемся командой:

```
sudo apt-get install phpmyadmin
```

Во время установки phpMyAdmin необходимо выполнить базовую конфигурацию. Когда процесс установки запущен, выполняем следующие действия:

- в качестве сервера выбираем Apache2;
- на вопрос, нужно ли настроить базу данных для phpmyadmin с помощью dbconfig-common, отвечаем YES;
- вводим пароль, ранее созданный при настройке MariaDB;
- устанавливаем пароль для входа в phpmyadmin.

По завершении установки вносим phpMyAdmin в конфигурацию Apache:

```
sudo nano /etc/apache2/apache2.conf
```

Дописываем в конфигурационный файл phpMyAdmin:

```
Include /etc/phpmyadmin/apache.conf
```

Затем перезапускаем Apache:

```
sudo service apache2 restart
```

Теперь открываем phpMyAdmin, вводим в строку адреса браузера teplica.org/phpmyadmin/index.php, это выведет страницу приветствия phpMyAdmin, которая выглядит так, как показано на рисунке 36.

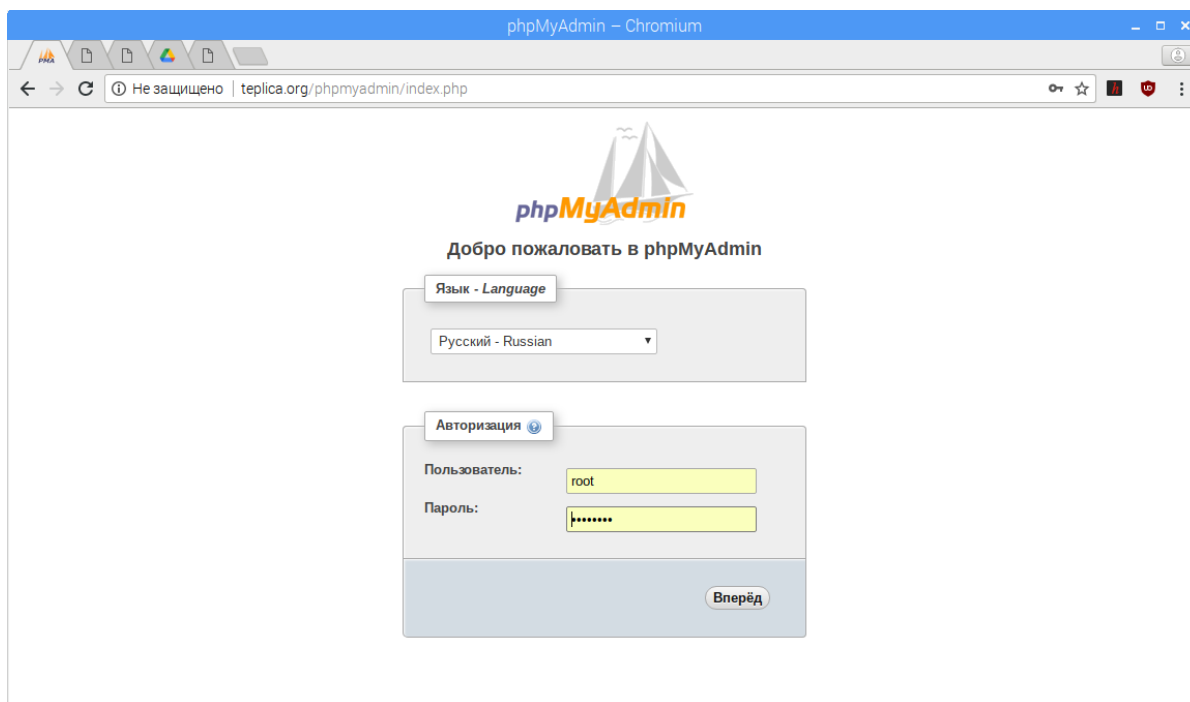


Рисунок 36. phpMyAdmin

Вводим логин root и пароль, который задавали ранее и попадаем в интерфейс программы, в левой части нажимаем на «Создать БД», указываем имя нашей базы данных «teplica» и нажимаем «Создать». Данный момент показан на рисунке 37.

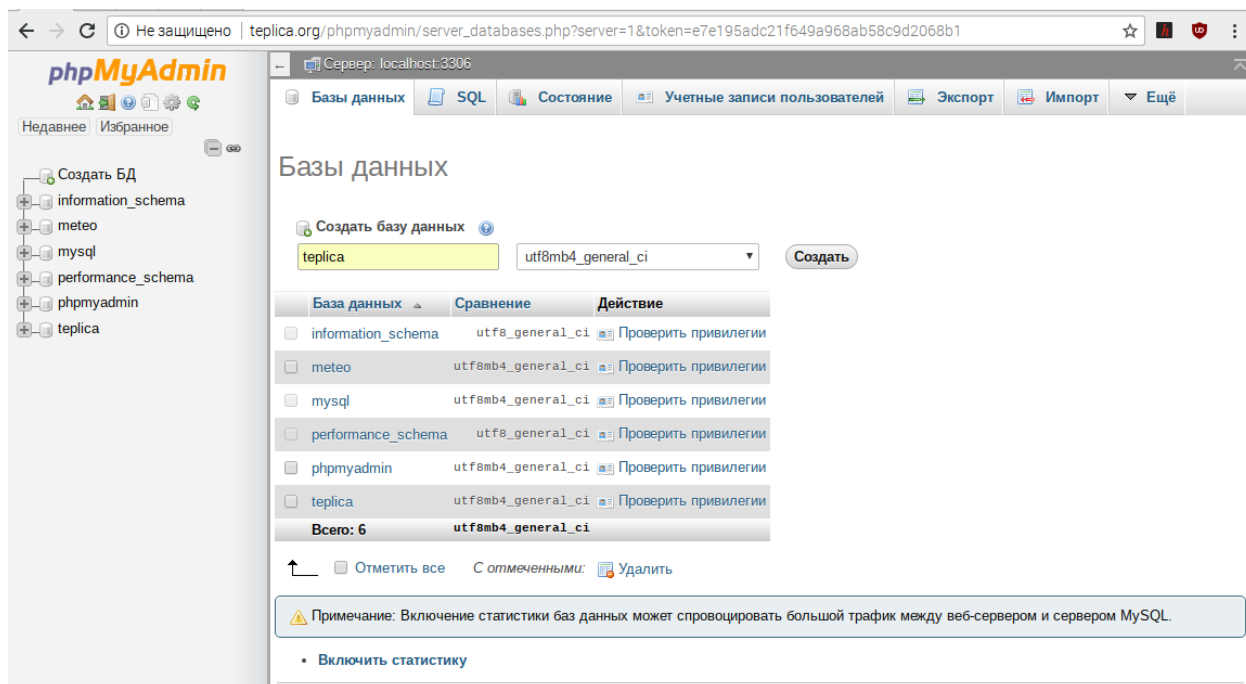


Рисунок 37. Создание БД

Далее необходимо создать таблицу, для данных, поступающих с датчика влажности. Нажимаем в разделе базы данных teplica «Новая таблица» и задаем название и тип полей. У поля «id_temp» ставим галочку параметра A_I, что значит

автоматическое прибавление значения на 1, в поле «board» будет записываться имя платы, с которой приходят данные, поэтому он текстовый тип данных. Поле «date» имеет тип даты и время, также в значении по умолчанию задан параметр для получения, текущего времени получения данных. Последнее поле имеет тип вещественного числа, куда будет записана температура в единичный промежуток времени. На рисунке 38 показаны поля таблицы для сохранения данных температуры.

The screenshot shows the phpMyAdmin interface for the 'teplica' database. The 'Table structure' view for the 'temperatura' table is displayed. The columns are defined as follows:

Имя	Тип	Длина/Значения	По умолчанию	Сравнение	Атрибуты	Null
id_temp	INT	11	Нет			☒
board	TEXT		Нет	utf8mb4_general_		☐
date	DATETIME		CURRENT_TIME()		on update	☒
temp	FLOAT		Нет			☐

Buttons at the bottom: Предпросмотр SQL, Сохранить.

Рисунок 38. Создание таблицы с данным о температуре

Далее сохраняем таблицу, и создаем аналогичную для сохранения данных о влажности воздуха. Далее создаем таблицу с наименованиями плат nodemcu. рисунок 39.

The screenshot shows the phpMyAdmin interface for the 'teplica' database. The 'Table structure' view for the 'nodemcu' table is displayed. The columns are defined as follows:

Имя	Тип	Длина/Значения	По умолчанию	Сравнение	Атрибуты	Null
id_esp	INT	11	Нет			☒
name_teplica	TEXT		Нет	utf8mb4_general_		☒
name_esp	TEXT		Нет	utf8mb4_general_		☒

Buttons at the bottom: Предпросмотр SQL, Сохранить.

Рисунок 39. Таблица в базе данных

На рисунке 40 показана итоговая структура таблиц в базе данных.

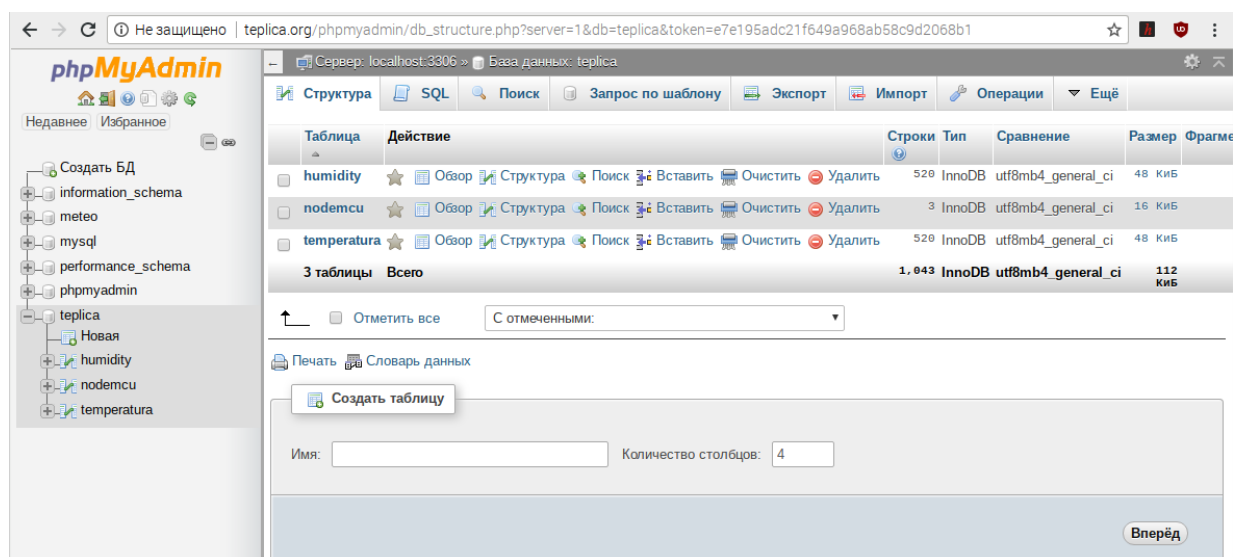


Рисунок 40. Структура таблицы со всеми таблицами

В итоге создали три таблицы, не связанные между собой, сделано это для последующей расширяемости и то, что датчики собирают разные данные и приходят они в разные промежутки времени. Данные с однотипных датчиков будут сохраняться в одну таблицу.

Так как у нас локальный сайт, то необходимо прописать ассоциацию *127.0.0.1 teplica.org* в файле */etc/hosts*.

Для того чтобы администрировать Raspberry Pi удаленно, для применения или изменения каких-либо параметров программы, активируем доступ по SSH (англ. Secure Shell – «безопасная оболочка») – это сетевой протокол прикладного уровня, который позволяет выполнять удалённое управление операционной системой и туннелирование соединений, шифрует весь трафик и пароли. SSH допускает выбор различных алгоритмов шифрования. Производим включение с помощью команды `sudo systemctl enable ssh` `sudo systemctl start ssh` [15].

На этом разработка программно-аппаратного комплекса для управления тепличным хозяйством завершена.

2.2 Описание продукта (результата разработки)

Согласно требованиям технического задания был реализован программно-аппаратный комплекс автоматизированной системы управления тепличным хозяйством.

Реализована модель теплицы, в которой установлены настроенный микроконтроллер с датчиками. Реализован автоматический и ручной режим управления подключенной нагрузкой. Для ручного управления на микроконтроллере реализован веб-сервер с интерфейсом. Данное решение необходимо для резервного управления теплицей в случае потери связи с центральным сервером. На странице отображается состояние режима работы, полива, обогрева, открыто или закрыто окно и включен ли свет, а также данные о температуре и влажности воздуха. На рисунках 41-43 изображен web-интерфейс для управления теплицей в режиме «AUTO» и в ручном режиме.



Рисунок 41. Включен режим «AUTO»

ESP8266 Web Server

Режим AUTO - off

AUTO ON

Температура = 28.50 C Влажность = 24.90 %

Полив Off | Обогрев Off | Окно Off | Свет Off

Полив - State off Обогреватель - State off

ON**ON**

Свет - State off

Окно - State off

ON**ON**

Рисунок 42. Ручной режим управления

ESP8266 Web Server

Режим AUTO - off

AUTO ON

Температура = 28.50 C Влажность = 24.90 %

Полив Off | Обогрев Off | Окно Off | Свет On

Полив - State off Обогреватель - State off

ON**ON**

Свет - State on

Окно - State off

OFF**ON**

Рисунок 43. Включено освещение снаружи и внутри

На сервере реализовано централизованное управление теплицами. Установлен и настроен mqtt-брокер, для обработки публикуемых данных от микроконтроллеров. Написан веб-интерфейс для удобного управления теплицами и отображения сохраненных данных, для перехода в браузере прописывается адрес terpica.org. Веб-интерфейс показан на рисунках 44-46.

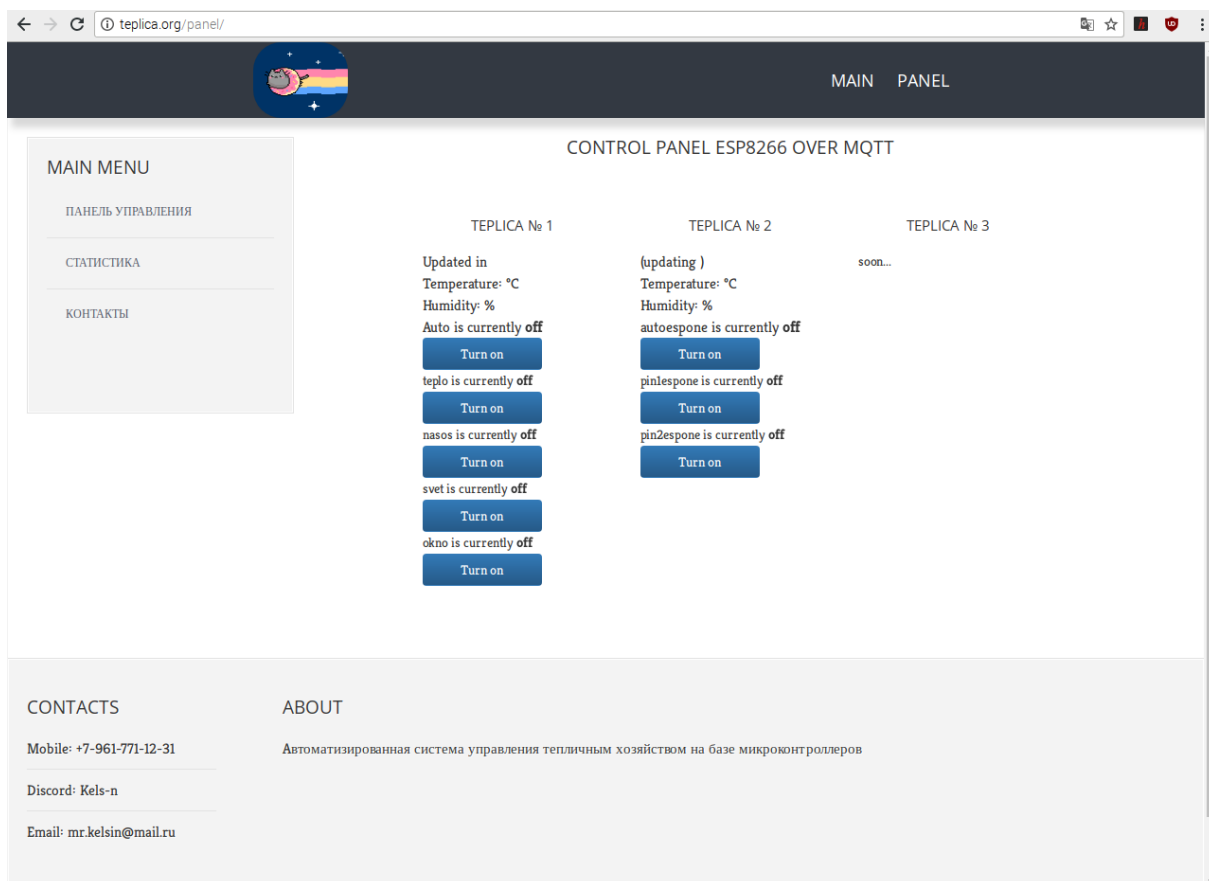


Рисунок 44. Страница управления теплицами

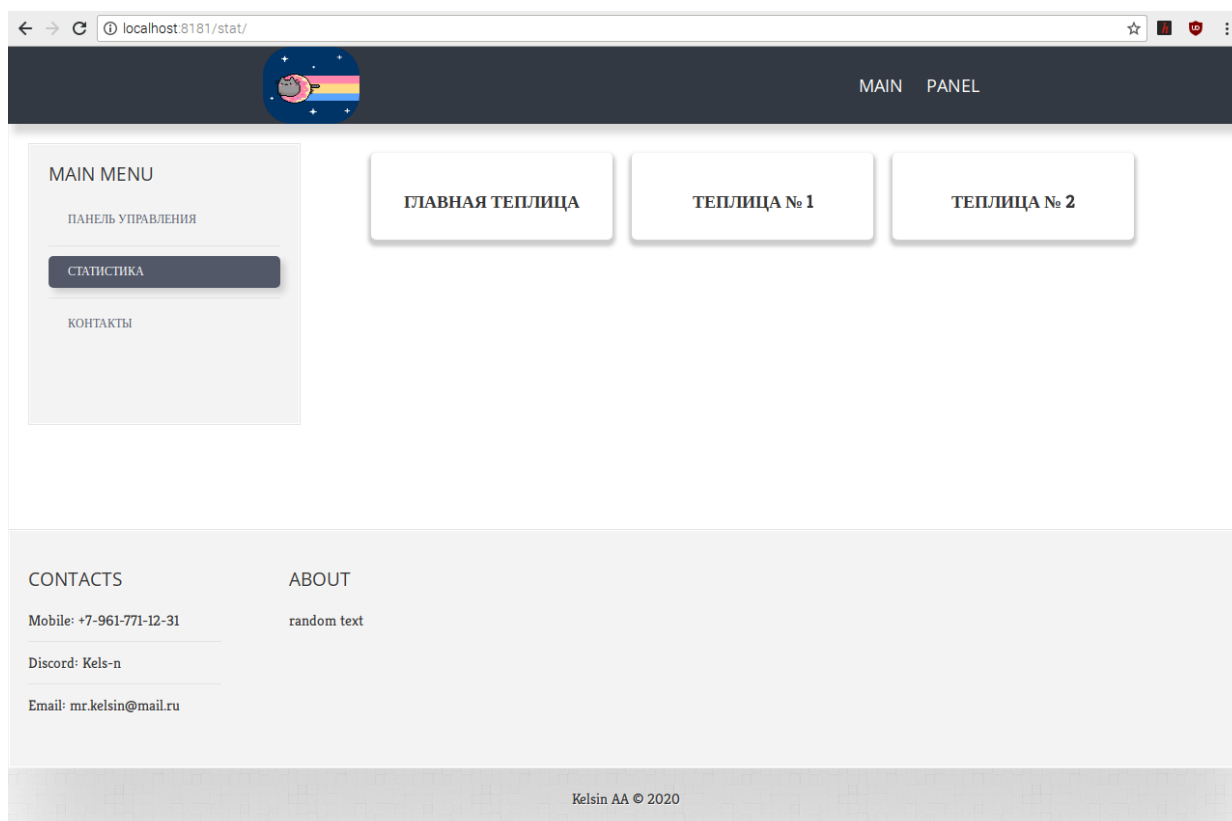


Рисунок 45. Страница со всеми теплицами

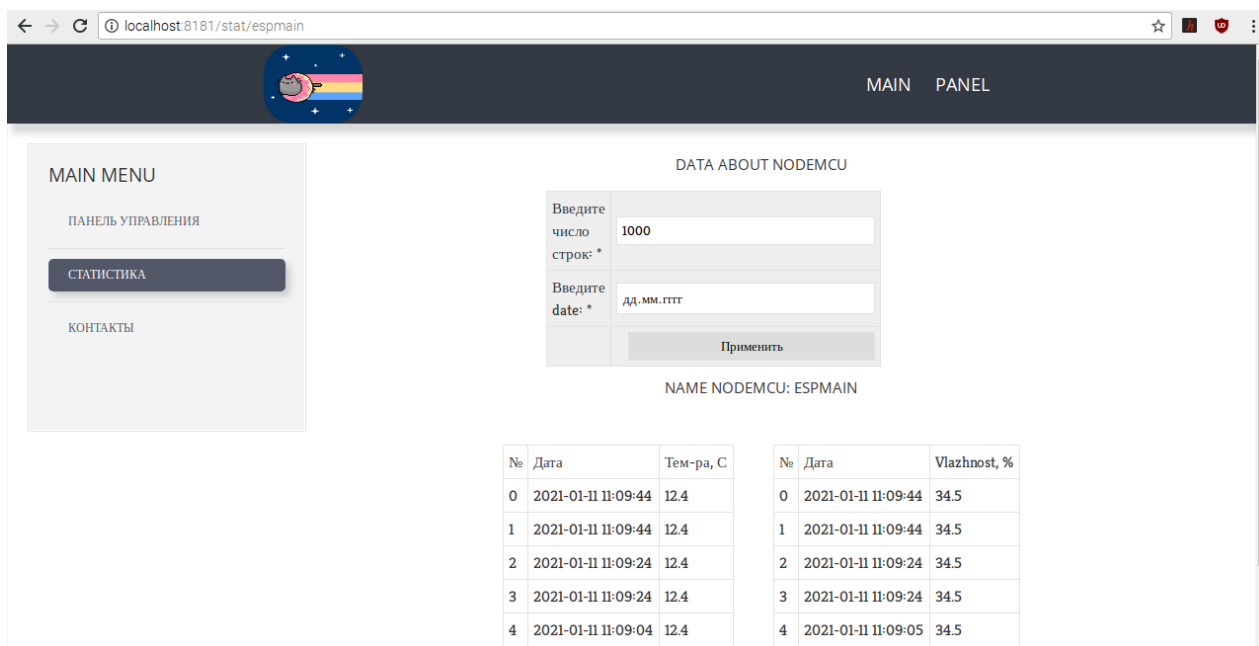


Рисунок 46. Страница с данными теплицы ESPMAIN

Для подключения к операционной системе по SSH возможно использовать программу бесплатную программу PuTTY. Для этого вводим ip-адрес нашей машинки и порт 22 , по которому работает SSH. На рисунках 47 показан интерфейс программы PuTTY, а на рисунке 48 терминал, в котором показана открытая сессия [32].

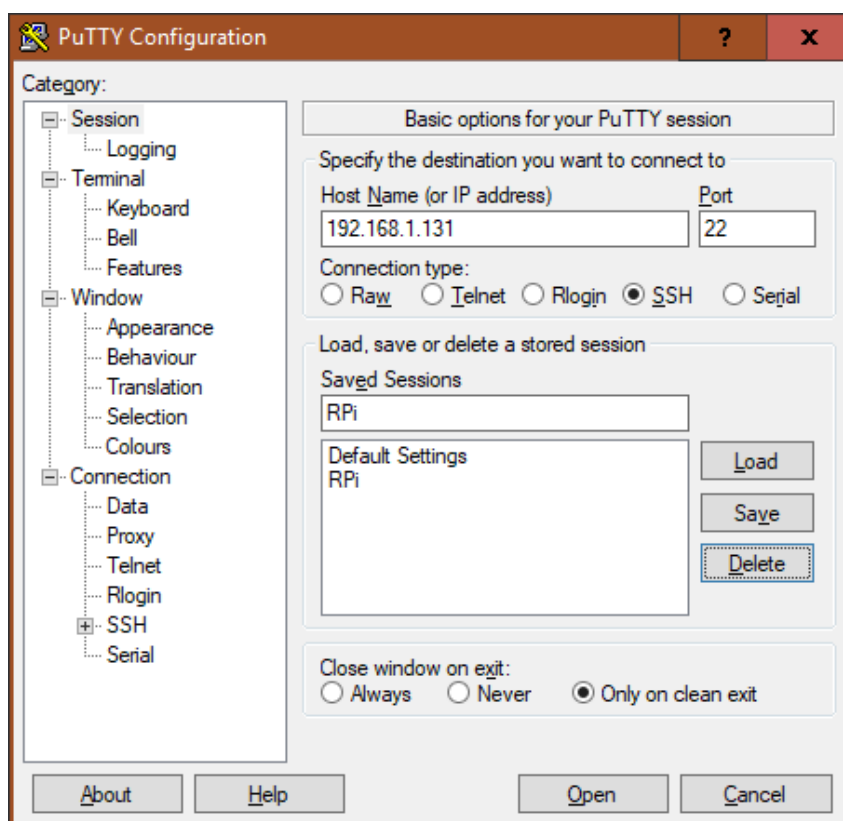


Рисунок 47. Программа PuTTY

```
pi@rpi: ~
login as: pi
pi@192.168.1.132's password:
Linux rpi 4.14.79-v7+ #1159 SMP Sun Nov 4 17:50:20 GMT 2018 armv7l

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sun Apr  7 12:11:59 2019 from 192.168.1.2

SSH is enabled and the default password for the 'pi' user has not been changed.
This is a security risk - please login as the 'pi' user and type 'passwd' to set
a new password.

pi@rpi:~ $
```

Рисунок 48. Открытая сессия с Raspberry Pi

Также имеется возможность подключаться по протоколу SFTP для замены файлов или расширения возможностей. SFTP (англ.SSH File Transfer Protocol) – это протокол прикладного уровня, предназначенный для работы с файлами поверх надёжного и безопасного соединения. Подключение можно осуществлять с помощью программы WinSCP – свободный графический клиент протокола SFTP. Обеспечивает защищённое копирование файлов между компьютером и серверами, поддерживающими эти протоколы.

В данной главе представлен макет теплицы, написан код, выполняющий определённый алгоритм действий, который зависит от времени суток и погодных условий, также реализован веб-интерфейс для взаимодействия между системой и пользователем, также присутствует вывод информации.

Заключение

В ходе выпускной квалификационной работы был полностью реализован программно-аппаратный комплекс автоматизированной системы управления тепличным хозяйством на базе микроконтроллеров.

В процессе работы над проектом были выполнены все поставленные задачи: были изучены принципы построения автоматизированных систем, произведен анализ и выбор программного и аппаратного обеспечения для реализации проекта. Была создана физическая модель теплицы. Показаны используемые датчики и элементы, микроконтроллер и серверная часть проекта, а также произведена настройка всех компонентов. Создан программно-аппаратный комплекс на базе микроконтроллера платы NodeMCU (ESP8266). Написан веб-интерфейс на каждом микроконтроллере. Теплица с автоматической регулировкой температуры и поливом, датчиком движения для включения света в темное время суток. Также реализован mqtt-брокер и веб-сервер, на котором находится написанный веб-интерфейс централизованного управления всеми теплицами, с отображением текущего состояния нагрузки (освещение, окно, обогреватель, насос для полива), а также температура и влажность воздуха. Реализованы автоматический и ручной режим работы. В ручном режиме управляет нагрузкой пользователь сам, это будет удобно для тех, кто хочет самостоятельно контролировать процессы.

Данный программно-аппаратный комплекс поможет тем, в нашем случае, кто хочет выращивать хороший урожай в теплице. Также можно изменить автоматический алгоритм, изменить нагрузку и установить в аквариум или террариум, чтобы поддерживать благоприятные условия в замкнутом пространстве.

Разработанный проект соответствует техническому заданию и полностью готов к эксплуатации.

Список использованных источников

1. Арнольд М. Администрирование Apache / Арнольд М., Алмейда Дж., Миллер К. – Лори, 2016. – 418 с.
2. Банахан, М. The C Book / М. Банахан. – БХВ-Петербург. – Москва, 2016. – 359 с.
3. Блум, Д. Изучаем Arduino / Д. Блум. БХВ-Петербург. – Москва, 2016. – 544 с.
4. ГОСТ 34.602-89. Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы : межгосударственный стандарт : издание официальное : дата введения 01.01.1990. – Москва : Стандартинформ, 2009. – 12 с. – Текст: непосредственный.
5. Граннеман С. Linux. Карманный справочник / С. Граннеман. – Диалектика, 2019. – 464 с.
6. Гринберг М. Разработка веб-приложений с использованием Flask на языке Python / М. Гринберг. – ДМК, 2016. – 272 с.
7. Густедт, Д. Modern C / Д. Густедт. – Вильямс, 2015. – 496 с.
8. Дакетт Дж. HTML и CSS. Разработка и создание веб-сайтов / Дж. Дакетт. – Эксмо. – Москва, 2019. – 480 с.
9. Дауни А. Изучение сложных систем с помощью Python / А. Дауни. – ДМК Пресс. – Москва, 2019. – 160 с.
10. Доусон М. Програмируем на Python / М. Доусон. – Питер СПб. – Санкт-Петербург, 2019. – 416 с.
11. Дронов В. PHP, MySQL, HTML5 и CSS 3. Разработка современных динамических Web-сайтов / В. Дронов. – БХВ-Петербург, 2016. – 688 с.
12. Дронов, В. HTML 5, CSS 3 и Web 2.0. Разработка современных Web-сайтов / В. Дронов. – БХВ-Петербург – Москва, 2011. – 416 с.
13. Кириченко А. HTML5 + CSS3. Основы современного WEB-дизайна / А. Кириченко, А. Хрусталева. – Наука и техника СПб. – Санкт-Петербург, 2018. – 352 с.
14. Клинтон Д. Linux в действии / Д. Клинтон. – Прогресс книга, 2019. – 416 с.

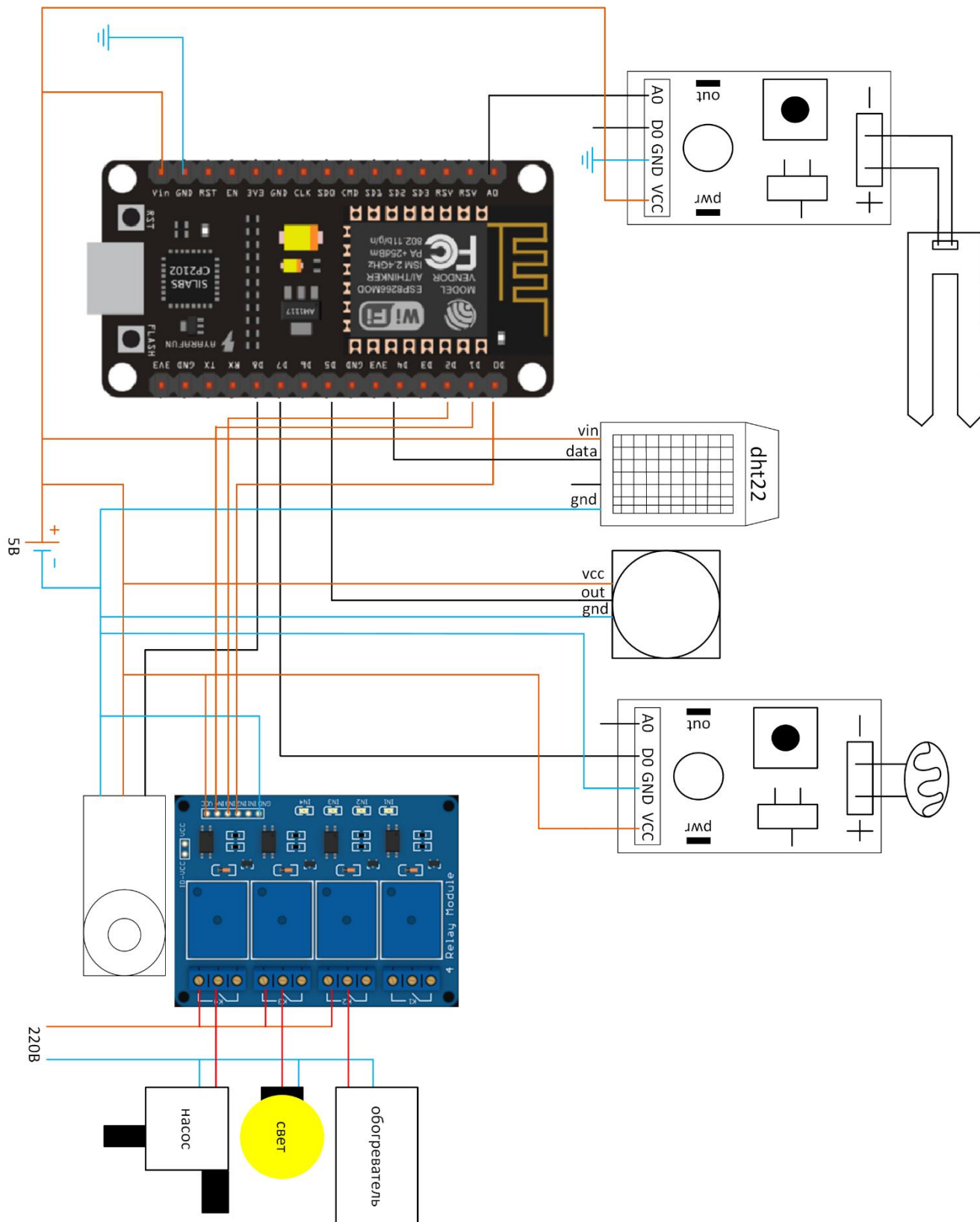
15. Колисниченко Д. Linux. От новичка к профессионалу Д. Колисниченко. – БХВ-Петербург. – Москва, 2020. – 672 с.
16. Колисниченко Д. LINUX. Полное руководство по работе и администрированию / Д. Колисниченко. – Наука и техника СПб. – Санкт-Петербург, 2021. – 480 с.
17. МакГрат, М. Программирование на С для начинающих / М. МакГрат. – Эксмо. – Москва, 2016. – 196 с.
18. Монк, С. Програмируем Arduino / С. Монк. – БХВ-Петербург, 2017. – 176 с.
19. Начало работы с ESP8266 NodeMcu v3 Lua с WiFi. – Текст: электронный. // ArduinoMaster: [сайт]. – URL: <https://arduinomaster.ru/platy-arduino/esp8266-nodemcu-v3-lua/> (дата обращения: 11.02.2021).
20. Обзор платы NodeMcu v2 на ESP8266-12E. – Текст: электронный // robotchip: [сайт]. – URL: <https://robotchip.ru/obzor-platy-nodemcu-v2-na-esp8266-12e/> (дата обращения: 28.01.2021).
21. Олифер, В. Компьютерные сети. Принципы, технологии, протоколы / В. Олифер, Н. Олифер. – 5-е изд. – СПб.: Питер, 2016. – 992 с. – Текст: непосредственный.
22. Официальный сайт Amperka. – Текст: электронный // Amperka: [сайт]. – URL: <https://amperka.ru/en/Main/Software> (дата обращения: 22.01.2021).
23. Перри, Г. Программирование на С для начинающих / Г. Перри, Д. Миллер. – 3-е изд. – Эксмо. – Москва, 2015. – 368 с.
24. Петин, В.А. Arduino и Raspberry Pi в проектах Internet of Things / В.А. Петин. – БХВ-Петербург. – Москва, 2019. – 432 с.
25. Петин, В.А. Практическая энциклопедия Arduino / В.А. Петин. – 2-е изд. – ДМК Пресс, 2020. – 166 с.
26. Петин, В.А. Проекты с использованием контроллера Arduino / В.А. Петин – 2-е изд. – БХВ-Петербург 2021. – 560 с.
27. Ревич, Ю. Занимательная электроника / Ю. Ревич. – БХВ-Петербург. – Москва, 2018. – 672 с.

28. Рейтц К. Автостопом по Python / К. Рейтц, Т. Шлюссер. – Питер СПб. – Санкт-Петербург, 2017. – 336 с.
29. Саммерфилд М. Python на практике / М. Саммерфилд. – ДМК. – Москва, 2016. – 338 с.
30. Сергеев, А.П. Основы локальных компьютерных сетей / А.П. Сергеев. – Лань, 2021. – 184 с.
31. Соммер, У. Программирование микроконтроллерных плат Arduino/Freeduino / У.Соммер. – СПб.: БХВ-Петербург, 2017. – 238 с.
32. Таненбаум, Э.С. Компьютерные сети / Э.С. Таненбаум, Д, Э, Уэзеролл. – Питер СПб, 2015. – 960 с.
33. Широтно-импульсная модуляция. – Текст: электронный // URL: <http://electricalschool.info/electronica/1759-shirotno-impulsnaja-moduljacija.html>, (дата обращения: 10.02.2021).
34. Шолле Ф. Глубокое обучение на Python / Ф. Шолле. – Питер СПб. – Санкт-Петербург, 2018. – 400 с.

Приложения

Приложение 1.

Схема подключения



Веб-интерфейс микроконтроллера для взаимодействия с теплицей

ESP8266 Web Server

Режим AUTO - on

AUTO OFF

Температура = 28.50 C Влажность = 24.80 %

Полив Off | Обогрев Off | Окно Off | Свет Off

ESP8266 Web Server

Режим AUTO - off

AUTO ON

Температура = 28.50 C Влажность = 24.90 %

Полив Off | Обогрев Off | Окно Off | Свет On

Полив - State off Обогреватель - State off

ON

ON

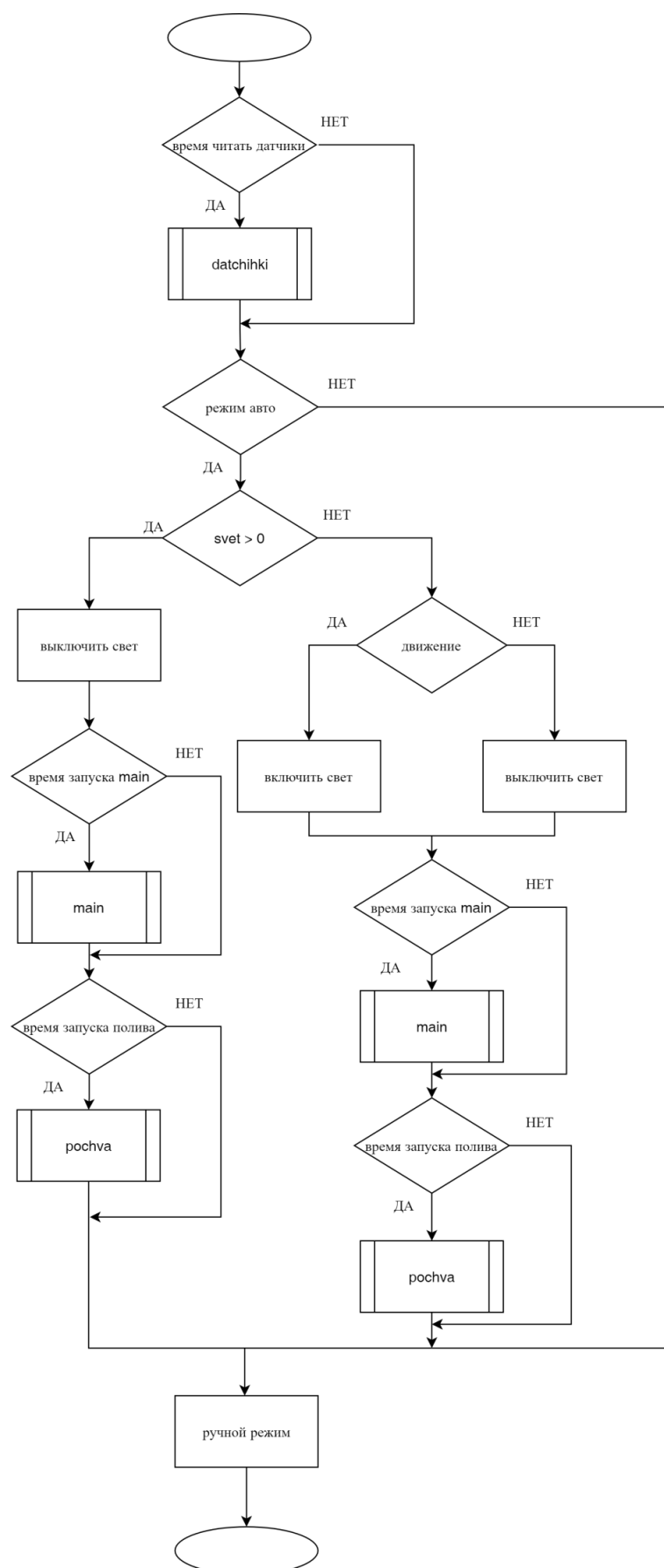
Свет - State on

Окно - State off

OFF

ON

Алгоритм работы автоматического режима работы



Листинг файла `__init__.py`

Указываем, что используется python.

```
#!/usr/bin/python
```

Кодировка файла utf-8.

```
# -*- coding: utf-8
```

Импортируем в приложение необходимы библиотеки, для работы с операционной системой.

```
import os, sys
```

Для работы со строковым типом данных.

```
import string
```

Также импортируем необходимую для работы языка python с протоколом MQTT, библиотеку.

```
import paho.mqtt.client as mqtt
```

Далее импортируем библиотеки необходимы для реализации сайта на Flask, также библиотеку для работы с базами данных и прочее.

```
from flask import Flask, render_template, request, redirect, flash, session
```

```
from flask_socketio import SocketIO, emit
```

```
from datetime import datetime
```

```
from flask_sqlalchemy import SQLAlchemy
```

```
sys.path.append(os.path.join(os.path.dirname(__file__), '..'))
```

Указываем путь до каталогов, которых размещены файлы сайта.

```
TEMPLATE_DIR = os.path.abspath('./templates')
```

```
STATIC_DIR = os.path.abspath('./static')
```

Задаем переменную, которая будет содержать наше приложение.

```
app = Flask(__name__)
```

Указываем секретный ключ для приложения.

```
app.config['SECRET_KEY'] = 'secret!'
```

Прописываем данные для подключения к базе данных.

```
app.config['SQLALCHEMY_DATABASE_URI'] = 'mysql://root:password@localhost/teplica'
```

Указываем, что приложение будет использовать библиотеку socketio.

```
socketio = SocketIO(app)
```


Указываем, что приложение будет использовать библиотеку SQLAlchemy для работы с базами данных и их таблицами.

```
db = SQLAlchemy(app)
```

Следующим шагом будет создание класса с логикой объектов, унаследованного из базовой библиотеки SQLAlchemy, для хранения информации о микроконтроллерах.

```
class Nodemcu(db.Model):
```

Указываем с какой таблицей работать.

```
    __tablename__ = 'nodemcu'
```

Указываем поля в таблице.

```
    id_esp = db.Column(db.Integer(), primary_key=True)
```

```
    name_teplica = db.Column(db.Text, nullable=False)
```

```
    name_esp = db.Column(db.Text, nullable=False)
```

`__repr__` же вызывается при операторе `print`, для удобства отображения.

```
    def __repr__(self):
```

```
        return "<{}: {}: {}>".format(self.id_esp, self.name_teplica, self.name_esp)
```

Аналогично создаем класс для хранения данных температуры.

```
class Temper(db.Model):
```

```
    __tablename__ = 'temperatura'
```

```
    id_temp = db.Column(db.Integer(), primary_key=True)
```

```
    board = db.Column(db.Text, nullable=False)
```

```
    date = db.Column(db.DateTime(), default=datetime.utcnow)
```

```
    temp = db.Column(db.Float, nullable=False)
```

```
    def __repr__(self):
```

```
        return "<{}: {}: {}>".format(self.id_temp, self.date, self.board, self.temp)
```

Также создали класс для хранения данных о влажности.

```
class Humidity(db.Model):
```

```
    __tablename__ = 'humidity'
```

```
    id_hum = db.Column(db.Integer(), primary_key=True)
```

```
    board = db.Column(db.Text, nullable=False)
```

```
    date = db.Column(db.DateTime(), default=datetime.utcnow)
```

```
    humidity = db.Column(db.Float, nullable=False)
```

```
def __repr__(self):
    return "<{}: {}: {}>".format(self.id_hum, self.date, self.board, self.humidity)
```

Далее написана функция, которая принимает имя микроконтроллера и температуру и загружает их в базу.

```
def insert_temp(board, temperature):
```

Использована конструкция try-ехсепт, которая пытается выполнить код, и в случае неудачи, выполняет код после ехсепт.

```
try:
```

Задаем переменную с запросом. Так как используется SQLAlchemy, то писать запрос на чистом sql не нужно, что упрощает написание и читабельность кода.

```
    slqinsert = Temper(board=board, temp=temperature)
```

Далее запрос добавляется в сессию работы с базами данных. SQLAlchemy автоматически производит действия с сессия

```
    db.session.add(slqinsert)
```

Применяем внесенные изменения в базы данных.

```
    db.session.commit()
```

Задаем переменную и выводим, что всё хорошо.

```
    message = "_____OK mysql_____"
```

```
    print message
```

```
except:
```

```
    message = "_____Problems with mysql in temp_____"
```

```
    print message
```

Аналогично создали функцию для сохранения данных о влажности.

```
def insert_hum(board, humidity):
```

```
try:
```

```
    slqinsert = Humidity(board=board, humidity=humidity)
```

```
    db.session.add(slqinsert)
```

```
    db.session.commit()
```

```
    message = "_____OK mysql_____"
```

```
    print message
```

```
except:
```

```
    message = "_____Problems with mysql in hum_____"
```

```
print message
```

В созданной функции `on_connect` подписываемся на топики, данные которых будут поступать с микроконтроллеров ESP8266.

```
def on_connect(client, userdata, flags, rc):  
    print("Connected with result code "+str(rc))  
    client.subscribe("/espmain/temperature")  
    client.subscribe("/espmain/humidity")  
    client.subscribe("/espone/pin1espone")  
    client.subscribe("/espone/pin2espone")
```

Функция `on_message` принимает данные с топиков, на которые подписан клиент и отвечает за их обработку.

```
def on_message(client, userdata, message):  
  
    Выводим полученное сообщение в консоль.  
  
    print("Received message '" + str(message.payload) + "' on topic '"  
        + message.topic + "' with QoS " + str(message.qos))
```

Далее производим проверку с какого топика пришли данные.

```
    if message.topic == "/espmain/temperature":  
        print("temperature update")
```

Далее с помощью библиотеки `socketio` выводим данные на сайт.

```
        socketio.emit('dht_temperature', {'data': message.payload})
```

Задаем переменную с именем платы.

```
        board = "espmain"
```

Задаем переменную с полученным сообщением.

```
        temp_espmain = float(message.payload)
```

Вызываем функцию, для сохранения данных в базу данных.

```
        temp_espmain = insert_temp(board, temp_espmain)  
        print temp_espmain
```

Далее аналогично производим проверку топиков.

```
    if message.topic == "/espmain/humidity":  
        print("humidity update")  
        socketio.emit('dht_humidity', {'data': message.payload})  
        board = "espmain"
```

```

humidity_espmain = float(message.payload)
humidity_espmain = insert_hum(board, humidity_espmain)
print humidity_espmain

```

```

if message.topic == "/espone/pin1espone":
    print("pin1espone update")
    socketio.emit('pin1espone', {'data': message.payload})

```

```

if message.topic == "/espone/pin2espone":
    print("pin2espone update")
    socketio.emit('pin2espone', {'data': message.payload})

```

Задаем клиента для протокола MQTT.

```
mqttc=mqtt.Client()
```

Задаем функцию, отвечающую за подпись на топики.

```
mqttc.on_connect = on_connect
```

Задаем функцию, отвечающую за обработку данных с топики.

```
mqttc.on_message = on_message
```

Задаем значения для подключения к брокеру.

```
mqttc.connect("localhost",1883,60)
```

Запускаем бесконечную функцию для взаимодействия с mqtt-брокером.

```
mqttc.loop_start()
```

Далее создаем словарь словарей, где будут храниться данные о платах и текущее состояние их топики.

```

pins = {
    'auto_main' : {'name' : 'Auto', 'board' : 'espmain', 'topic' : 'espmain/auto_main', 'state' :
'False'},
    'svet_main' : {'name' : 'svet', 'board' : 'espmain', 'topic' : 'espmain/svet_main', 'state' : 'False'},
    'nasos_main' : {'name' : 'nasos', 'board' : 'espmain', 'topic' : 'espmain/nasos_main', 'state' :
'False'},
    'okno_main' : {'name' : 'okno', 'board' : 'espmain', 'topic' : 'espmain/okno_main', 'state' :
'False'},
    'obogrev_main' : {'name' : 'teplo', 'board' : 'espmain', 'topic' : 'espmain/obogrev_main', 'state' :
'False'},

```

```

        'autoesponse' : {'name' : 'autoesponse', 'board' : 'esponse', 'topic' : 'esponse/autoesponse', 'state' :
'False'},
        'pin1esponse' : {'name' : 'pin1esponse', 'board' : 'esponse', 'topic' : 'esponse/pin1esponse', 'state' :
'False'},
        'pin2esponse' : {'name' : 'pin2esponse', 'board' : 'esponse', 'topic' : 'esponse/pin2esponse', 'state' :
'False'}
    }

```

Созданный словарь pins поместили в следующий словарь, для удобства передачи данных на страницу сайта, когда будет необходимо добавить еще данных, нужно только дописать их в этот словарь.

```

templateData = {
    'pins' : pins
}

```

Современные веб-приложения используют «красивые» URL, с помощью маршрутизации, поэтому в приложении используется декоратор route(), который необходим для привязки функции к URL. Это помогает людям запомнить эти URL, чтобы пользователь мог перейти сразу на желаемую страницу, без предварительного посещения начальной страницы. Задаем маршрутизацию для главной страницы.

```

@app.route("/")
def main():

```

Внутри Flask уже автоматически преднастроен шаблонизатор Jinja2. Поэтому для визуализации шаблона используется метод render_template(). Всё, что необходимо - это указать имя шаблона, а также переменные в виде именованных аргументов, которые необходимо передать движку обработки шаблонов.

```

    return render_template('main.html')

```

Задаем маршрутизацию для страницы с именами теплиц.

```

@app.route("/stat")
def stat():

```

Выполняем запрос всех плат из базы данных.

```

    sqlnodemcu = db.session.query(Nodemcu).all()

```

Задаем переменную с количеством полученных строк.

```

    lenodemcu = len(sqlnodemcu)

```

Создаем словарь с именованными аргументами.

```
info = {  
    'sqlnodemcu':sqlnodemcu,  
    'lenodemcu':lenodemcu,  
}
```

Выводим страницу с данными с помощью шаблонизатора.

```
return render_template('templ.html', **info)
```

Задаем маршрутизацию для страницы с данными о температуре и влажности конкретной теплицы. Чтобы добавлять к адресу URL переменные части, можно эти особые части выделить в `<variable_name>`. Затем подобные части передаются в функцию в качестве аргумента – в виде ключевого слова.

```
@app.route("/stat/<board>", methods=['post','get'])
```

```
def statboard(board):
```

Задаем количество строк для вывода по умолчанию.

```
rows = 10 #лимит запроса данных
```

Далее выполняем два запроса из базы данных о температуре и влажности воздуха с фильтрацией по имени теплицы, с сортировкой по дате и ограничением в количестве строк.

```
sqltemp =
```

```
db.session.query(Temper).order_by(db.desc(Temper.date)).filter(Temper.board==board).limit(rows).all(  
)
```

```
sqlhum =
```

```
db.session.query(Humidity).order_by(db.desc(Humidity.date)).filter(Humidity.board==board).limit(rows  
)
```

Поскольку на странице, с данными о температуре и влажности, реализована форма, с помощью которой можно задавать параметры вывода данных, то добавляем проверку, поступали ли данные из формы или нет. Параметры можно задавать по одному, благодаря реализованной проверке.

```
if request.method == 'POST':
```

```
    if request.form.get('rows') is None:
```

```
        rows = 100
```

```
    else:
```

Возводим в модуль и округляем.

```
        rows = abs(int(request.form.get('rows')))
```

```
        calendar = request.form.get('calendar')
```

Далее выполняем два запроса в соответствии с заданными параметрами из формы.

```
sqltemp =  
db.session.query(Temper).order_by(db.desc(Temper.date)).filter(Temper.board==board,  
Temper.date.startswith(calendar)).limit(rows).all()  
  
sqlhum =  
db.session.query(Humidity).order_by(db.desc(Humidity.date)).filter(Humidity.board==board,  
Humidity.date.startswith(calendar)).limit(rows).all()
```

Задаем переменные с количеством строк в запросах, так как заданное ранее количество строк может превышать существующие.

```
lentemp = len(sqltemp)  
lenhum = len(sqlhum)
```

Добавляем необходимые переменные в виде именованных аргументов словаря.

```
info = {  
    'sqltemp':sqltemp,  
    'sqlhum':sqlhum,  
    'lentemp':lentemp,  
    'lenhum':lenhum,  
    'namesp':board,  
}
```

```
return render_template('stat.html', **info)
```

Задаем маршрутизацию для страницы информации.

```
@app.route("/info/")  
def info():  
    return render_template('info.html')
```

Задаем маршрутизацию для страницы с панелью управления нагрузкой теплиц.

```
@app.route("/panel/")  
def panel():  
    return render_template('panel.html',async_mode=socketio.async_mode)
```

Задаем маршрутизацию для страницы с панелью управления нагрузкой теплиц, при нажатии на кнопку.

```
@app.route("/<board>/<changePin>/<action>")
```

```
def action(board, changePin, action):
```

Переводим переменную в строковый тип данных.

```
changePin = str(changePin)
```

Присваиваем имя платы из словаря в соответствии с принятым changePin.

```
devicePin = pins[changePin]['name']
```

Далее производим проверку, на какой плате и в какое состояние перевести нагрузку.

```
if action == "1" and board == 'espmain' or action == "1" and board == 'espone':
```

Публикуем данные в нужный топик.

```
mqttc.publish(pins[changePin]['topic'], "1")
```

Меняем значение о состоянии в словаре pins.

```
pins[changePin]['state'] = 'True'
```

```
if action == "0" and board == 'espmain' or action == "0" and board == 'espone':
```

```
mqttc.publish(pins[changePin]['topic'], "0")
```

```
pins[changePin]['state'] = 'False'
```

```
templateData = {
```

```
    'pins': pins
```

```
}
```

```
return render_template('panel.html', async_mode=socketio.async_mode, **templateData)
```

Благодаря конструкции ниже можно быть уверенным, что сервер запустится только при непосредственном вызове скрипта из интерпретатора Python, а не при его импортировании в качестве модуля.

```
if __name__ == "__main__":
```

```
    socketio.run(app, host='0.0.0.0', port=8181)
```


Листинг программы для микроконтроллера ESP8266

Подключаем все библиотеки

```
#include <ESP8266WiFi.h> //библиотека для подключения к WiFi
#include <WiFiClient.h> //библиотека для работы платы в режиме клиента
#include <ESP8266mDNS.h>
#include <PubSubClient.h>
#include <SimpleDHT.h> //для работы с датчиком DHT22
#include <Servo.h> //используем библиотеку для работы с сервоприводом
#include <Thread.h> // библиотека для работы с псевдопараллельными потоками
```

Объявляем переменную servo типа Servo.

```
Servo servo;
```

Создаем переменные для имени сети и пароля, которые будут использоваться для подключения к существующей сети.

```
const char* ssid = "имя_WiFi_cemu";
const char* password = "password";
```

Создаем переменные, которые будут использоваться для режима точки доступа.

```
const char* ssid_ap = "имя_WiFi_cemu_точки_доступа";
const char* password_ap = "password";
```

Создаем сетевые реквизиты, которые будут использоваться для режима точки доступа.

```
IPAddress local_ip(192,168,1,1);
IPAddress gateway(192,168,1,1);
IPAddress subnet(255,255,255,240);
```

IP-адрес Raspberry Pi, чтобы она могла подключиться к брокеру MQTT.

```
const char* mqtt_server = "192.168.43.235";
```

Задаем переменную для веб-сервера.

```
WiFiServer server(80);
```

Инициализируем espClient для подключения к mqtt брокеру:

```
WiFiClient espClient;  
PubSubClient client(espClient);
```

Далее идут переменные, которые будут использоваться в программе.

Датчик освещенности подключен к цифровому контакту D7 платы, так как нам необходимо определять лишь день и ночь.

```
#define PIN_PHOTO_SENSOR 13;
```

Переменная, в которой хранится значение о режиме работы.

```
String tmp_auto = "on";
```

Переменная, в которой хранится web-страница.

```
String header;
```

Переменные, которые хранят в себе статус нагрузки.

```
String output4State = "off"; //статус свет  
String output5State = "off"; //статус насос  
String output15State = "off"; //статус окно  
String output16State = "off"; //статус обогреватель
```

Задаем постоянные переменные с номерами контактов платы.

```
const int output5 = 5; //насос  
const int output4 = 4; //свет  
const int output16 = 16; //обогреватель
```

Задаем переменную с номером контакта, к которому подключен датчик DHT22.

```
int pinDHT22 = D4;
```

Вызов функции для DHT22.

```
SimpleDHT22 dht22(pinDHT22);
```

Задаем переменные аналогового входа A0 и для хранения данных датчика влажности почвы.

```
int pochva_pin = A0;  
int tmp_pochva = 0;
```

Переменные для хранения данных о температуре и влажности воздуха полученных от датчика DHT22.

```
float temperature = 0;  
float humidity = 0;
```

Создаем 4 псевдопараллельных потока для основной функции, которая поддерживает температуру в помещении, и для функции управления полива.

```
Thread mainnightThread = Thread(); // создаём основной поток управления  
Thread nasosThread = Thread(); // создаём поток управления насосом  
Thread reconnectThread = Thread(); // создаём поток для переподключения  
Thread datchikiThread = Thread(); // создаём поток для считывания датчиков
```

```
static char temperatureTemp[7]; // для хранения данных о температуре в char для публикации  
static char humidityTemp[7]; // для хранения данных о влажности в char для публикации
```

Пишем функцию для подключения платы к сети Wi-Fi/

```
void setup_wifi() {  
    delay(10);
```

Ждем соединения и выводим ip-адрес в терминал.

```
    Serial.println();  
    WiFi.begin(ssid, password);  
    Serial.print("WiFi connecting.");  
    while (WiFi.status() != WL_CONNECTED) {  
        delay(500);  
        Serial.print(".");  
    }  
    Serial.println("Connected to "); // "Подключились к "  
    Serial.println(ssid);  
    Serial.println("");  
    Serial.print("WiFi connected - ESP IP address: ");  
    // "Подключение к WiFi выполнено – IP-адрес ESP8266: "
```

```

Serial.println(WiFi.localIP());

if (mdns.begin("esp8266", WiFi.localIP())) {
  Serial.println("MDNS responder started");
  // "Занущен MDNSresponder"

}
}

```

Эта функция выполняется, когда какой-то девайс публикует сообщение в топик, на который подписан ESP8266.

```

void callback(String topic, byte* message, unsigned int length) {
  Serial.print("Message arrived on topic: ");
  "Сообщение прибыло в топик: "
  Serial.print(topic);
  Serial.print(". Message: "); // ". Сообщение: "
  String messageTemp;

```

Далее выводим пришедшее сообщение.

```

for (int i = 0; i < length; i++) {
  Serial.print((char)message[i]);
  messageTemp += (char)message[i];
}
Serial.println();

```

если топик *espmain/auto_main* получил сообщение, смотрим, что это за сообщение – «0» или «1» и переключаем GPIO-контакт в соответствии с присланным значением.

Если топик *espmain/auto_main* получил «1» т.е. включить, то игнорируем остальные топики, если «0», то смотрим остальные топики

```

if(topic=="espmain/auto_main"){
  Serial.print("Changing AUTO mode to ");
  // "Смена состояния режима АВТО на "

```

Далее условие, если в сообщении единица, то включаем режим Автоматической работы

```

if(messageTemp == "1"){

```

```

    tmp_auto = "on"; //меняем значение в переменной
    Serial.print("On");
}
else if(messageTemp == "0"){
    tmp_auto = "off";
    Serial.print("Off");
}
}

```

Условие, что если режим Авто выключен, то проверяем в какой топик пришло сообщение.

```
if(tmp_auto == "off"){
```

Проверяем пришло ли сообщение в топик *espmain/svet_main*.

```

if(topic=="espmain/svet_main"){
    Serial.print("Changing GPIO 4 (svet) to ");
    // "Смена состояния GPIO-контакта 4 на "

```

Далее проверяем какое сообщение пришло в топик.

```
if(messageTemp == "1"){
```

Меняем состояние контакта.

```
digitalWrite(output4, HIGH);
```

Меняем значение переменной, в которой хранится состояние включено.

```

    output4State = "on";
    Serial.print("On");
}
else if(messageTemp == "0"){
    digitalWrite(output4, LOW);
    output4State = "off";
    Serial.print("Off");
}
}

```

Далее аналогично с остальными топиками.

```

if(topic=="espmain/nasos_main"){
    Serial.print("Changing GPIO 5 (nasos) to ");
    if(messageTemp == "1"){

```

```

    digitalWrite(output5, HIGH); // включаем насос
    output5State = "on";
    Serial.print("On");
}
else if(messageTemp == "0"){
    digitalWrite(output5, LOW); // выключаем насос
    output5State = "off";
    Serial.print("Off");
}
}

if(topic=="espmain/obogrev_main"){
    Serial.print("Changing GPIO 16 (obogrev) to ");
    if(messageTemp == "1"){
        digitalWrite(output16, HIGH); //включить реле - обогреватель
        output16State = "on";
        Serial.print("On");
    }
    else if(messageTemp == "0"){
        digitalWrite(output16, LOW); //включить реле - обогреватель
        output16State = "off";
        Serial.print("Off");
    }
}

if(topic=="espmain/okno_main"){
    Serial.print("Changing GPIO 16 (okno) to ");
    if(messageTemp == "1"){
        servo.write(75); //ставим вал под 75 градусов
        output15State = "on";
        Serial.print("On");
    }
    else if(messageTemp == "0"){
        servo.write(0); //ставим вал под 0 градусов
        output15State = "off";
    }
}

```

```

        Serial.print("Off");
    }
}
}

Serial.println();
}

```

Эта функция переключает ESP8266 к MQTT-брокеру, и подписывается на заданные топики.

```

void reconnect() {
    Если клиент не подключен, то переключаемся.
    if (!client.connected()) {
        Serial.print("Attempting MQTT connection...");
        // "Попытка подключиться к MQTT-брокеру... "
        if (client.connect("espmain")) {
            Serial.println("connected"); // "подключен"

```

Подписываемся или переподписываемся на топик, можно подписаться не только на один, а на несколько топиков сразу.

```

        client.subscribe("espmain/auto_main");
        client.subscribe("espmain/svet_main");
        client.subscribe("espmain/nasos_main");
        client.subscribe("espmain/okno_main");
        client.subscribe("espmain/obogrev_main");

    } else {
        Serial.print("failed, rc="); // "подключение не удалось"
        Serial.print(client.state());
        Serial.println(" try again in 5 seconds");
    }
}
}
}

```

Функция первоначальных настроек, запускается единожды, при запуске платы.

```

void setup() {
    Serial.begin(115200); //открываем последовательный порт, задаем скорость передачи
данных 115200 бод
    pinMode(D4, INPUT); //переводим контакт для DHT22 в режим приема данных
    servo.attach(D8); //привязываем сервопривод к контакту D8
    pinMode(D5, INPUT_PULLUP); //переводим контакт для датчик движения на приём

    Переводим контакты для нагрузки в режим на выход.
    pinMode(output5, OUTPUT);
    pinMode(output4, OUTPUT);
    pinMode(output16, OUTPUT);

    Переводим контакты для нагрузки в состояние включено, что при подключении к реле
выключит нагрузку.
    digitalWrite(output5, HIGH);
    digitalWrite(output4, HIGH);
    digitalWrite(output16, HIGH);

    Вызываем функцию подключения к беспроводной сети Wi-Fi.
    setup_wifi();

    Задаем MQTT сервер (брокер), к которому подключается клиент.
    client.setServer(mqtt_server, 1883);
    Задаем функцию, которая будет читать сообщения из топиков.
    client.setCallback(callback);

    Назначаем потокам функции и задаем интервал срабатывания в мсек.
    mainnightThread.onRun(mainnight);
    mainnightThread.setInterval(10000);

    nasosThread.onRun(nasos);
    nasosThread.setInterval(5000);

    reconnectThread.onRun(reconnect); // назначаем потоку задачу
    reconnectThread.setInterval(5000); // задаём интервал срабатывания, мсек

```



```
datchikiThread.onRun(datchiki); // назначаем потоку задачу  
datchikiThread.setInterval(20000); // задаём интервал срабатывания, мсек
```

Запускаем внутренний http-сервер.

```
server.begin();  
Serial.println("HTTP server started");
```

Настроили программную точку доступа для создания Wi-Fi сети, подтвердив SSID, пароль, IP-адрес, маску IP-подсети и IP-шлюз.

```
WiFi.softAP(ssid_ap, password_ap);  
WiFi.softAPConfig(local_ip, gateway, subnet);  
}
```

Далее следует функция, которая постоянно повторяется во время работы платы.

```
void loop() {
```

проверяем подключен ли клиент, если нет, то вызываем функцию подключения.

```
if (!client.connected()) {  
    reconnect();  
}  
if(!client.loop()){  
    client.connect("espmain");  
}
```

Если пришло время считывания данных с датчиков, то выполняем функцию datchiki

```
if (datchikiThread.shouldRun()) {  
    datchikiThread.run();  
}
```

Считываем данные с датчика освещенности.

```
int svet = digitalRead(PIN_PHOTO_SENSOR);
```

Далее проверяем значение включен ли режим автоматической работы.

```
if (tmp_auato == "on") {
```

Если автоматический режим включен, то следуем по коду ниже.

Далее проверяем значение датчика освещенности, что определит какой сценарий будет выполняться, для дня или ночи.

```
if (svet > 0) { //если svet больше 0, значит ночь
```

Далее проверяем псевдопараллельный поток, если наступило время запуска, то запускаем функцию mainnight, в которой происходит управление окном и обогревателем.

```
if (mainnightThread.shouldRun()) {  
    mainnightThread.run();  
}
```

Следующим шагом идет считывание данных с датчика движения и установка состояния света. Для включения/выключения света также можно было задать поток и время работы, но так как у нас датчик движения выполнен в виде модуля, то он имеет регулятор, задающий время на которое свет будет включен. Этого времени хватит, но при желании можно использовать поток и задавать время самому.

```
int valueDvizhenie = digitalRead(D5); //считываем данные датчика движения  
if (valueDvizhenie == 1) { //проверяем есть ли сигнал с датчика движения  
    Serial.println("EST dvizhenia");  
    digitalWrite(output4, LOW); //включаем свет на реле  
    output4State = "on"; //запоминаем состояние света на «включен»  
} else {  
    Serial.println("NET dvizhenia");  
    digitalWrite(output4, HIGH);  
    output4State = "off";  
}
```

Далее проверяем настало ли время для запуска функции отвечающей за полив.

```
if (nasosThread.shouldRun()) {  
    nasosThread.run();  
}  
}  
else {
```

Сценарий если на улице день. Также есть две задачи потока для управления поливом и поддержании температуры. Также проверка состояния света и выключение.

```
if (mainnightThread.shouldRun()) {  
    mainnightThread.run();  
}  
  
if (output4State = "on") {  
    digitalWrite(output4, HIGH);  
    output4State = "off";  
}  
  
if (nasosThread.shouldRun()) {  
    nasosThread.run();  
}  
}  
}
```

Если для автоматической работы теплицы достаточно только включить плату, то для управления в ручном режиме необходим веб-интерфейс, с помощью которого пользователь будет взаимодействовать с теплицей. В случае обрыва связи и недоступности основного узла управления, реализуем веб-интерфейс управления на самой плате, для подключения напрямую без использования протокола MQTT.

Включаем ожидание входящих клиентов.

```
WiFiClient client = server.available();
```

Если подключился новый клиент

```
if (client) {
```

То пишем сообщение в терминал.

```
Serial.println("New Client.");
```

Создаем строку для хранения входящих данных от клиента.

```
String currentLine = "";
```

Задаем цикл, если клиент подключен.

```
while (client.connected()) {
```

Если есть байты для чтения с клиента.

```
if (client.available()) {
```

Читаем байт, затем

```
char c = client.read();
```

Пишем в терминал.

```
Serial.write(c);
```

```
header += c;
```

Если байт является символом новой строки.

```
if (c == '\n') {
```

Если текущая строка пуста, получаем два символа новой строки подряд. Это конец клиентского HTTP-запроса, поэтому отправляем ответ.

```
if (currentLine.length() == 0) {
```

Заголовки HTTP всегда начинаются с кода ответа (например, HTTP / 1.1 200 OK) и типа содержимого, чтобы клиент знал, что происходит, затем пустая строка.

```
client.println("HTTP/1.1 200 OK");
```

```
client.println("Content-type:text/html");
```

```
client.println("Connection: close");
```

```
client.println();
```

Включаем и выключаем нагрузку в соответствии с нажатой пользователем кнопкой в веб-интерфейсе. Проверяем GET запрос, выводим в терминал состояние контакта, меняем в переменной состояние контакта.

```
if (header.indexOf("GET /5/on") >= 0) {
```

```
Serial.println("GPIO 5 on");
```

```
output5State = "on";
```

```
digitalWrite(output5, LOW);
```

```
} else if (header.indexOf("GET /5/off") >= 0) {
```

```
Serial.println("GPIO 5 off");
```

```
output5State = "off";
```

```
digitalWrite(output5, HIGH);
```

```
} else if (header.indexOf("GET /4/on") >= 0) {
```

```
Serial.println("GPIO 4 on");
```

```
output4State = "on";
```

```
digitalWrite(output4, LOW);
```

```
} else if (header.indexOf("GET /4/off") >= 0) {
```

```
Serial.println("GPIO 4 off");
```

```

output4State = "off";
digitalWrite(output4, HIGH);
} else if (header.indexOf("GET /16/on") >= 0) {
  Serial.println("GPIO 16 on");
  output16State = "on";
  digitalWrite(output16, LOW);
} else if (header.indexOf("GET /16/off") >= 0) {
  Serial.println("GPIO 16 off");
  output16State = "off";
  digitalWrite(output16, HIGH);
} else if (header.indexOf("GET /15/on") >= 0) {
  Serial.println("GPIO 15 on");
  output15State = "on";
  servo.write(75);
} else if (header.indexOf("GET /15/off") >= 0) {
  Serial.println("GPIO 15 off");
  output15State = "off";
  servo.write(0);
} else if (header.indexOf("GET /auto/on") >= 0) {
  Serial.println("AUTO on");
  tmp_auto = "on";
} else if (header.indexOf("GET /auto/off") >= 0) {
  Serial.println("AUTO off");
  tmp_auto = "off";
}

```

Создаем HTML страничку для показа клиенту

```

client.println("<!DOCTYPE html><html>");
client.println("<head><meta          name=\"viewport\"          charset=\"utf-8\"
content=\"width=device-width, initial-scale=1\">");
client.println("<link rel=\"icon\" href=\"data:;\">");

```

CSS стилизация кнопок.

```

client.println("<style>html { font-family: Helvetica; display: inline-block; margin: 0px
auto; text-align: center;}");

```

```

client.println(".button { background-color: #195B6A; border: none; color: white;
padding: 16px 40px;");
client.println("text-decoration: none; font-size: 30px; margin: 2px; cursor: pointer;}");
client.println(".button2 {background-color: #77878A;}</style></head>");

```

Тело страницы, которое будет видеть клиент.

```
client.println("<body><h1>ESP8266 Web Server</h1>");
```

Задаем отображение корректного состояния ON/OFF кнопки режима авто.

```
client.println("<h3>Режим AUTO - " + tmp_auto + "</h3>");
```

Далее проверяем состояние кнопки, и в соответствии со значением переменной отображаем необходимую кнопку.

```

if (tmp_auto == "off") {
    client.println("<p><a      href=\"/auto/on\"><button      class=\"button\">AUTO
ON</button></a></p>");
} else {
    client.println("<p><a href=\"/auto/off\"><button class=\"button button2\">AUTO
OFF</button></a></p>");
}

```

Выводим информацию с датчиков и состояние нагрузки.

```

client.println("<p>Температура = ");
client.println(temperature);
client.println(" C ");
client.println("Влажность = ");
client.println(humidity);
client.println(" %</p>");
client.println("<p>Полив " + output5State + " | Обогрев " + output16State + " | Окно "
+ output15State + " | Свет " + output4State + "</p>");
client.println("</p>");

```

Если автоматический режим выключен, то дополнительно отобразить кнопки управления нагрузкой.

```

if (tmp_auto == "off") {

    client.println("<h3>Полив - State " + output5State + "</h3>");

```

```

        if (output5State == "off") {
            client.println("<p><a href=\""/5/on\"><button class=\"button\"> ON </button> </a>
</p>");
        } else {
            client.println("<p><a href=\""/5/off\"><button class=\"button button2\"> OFF
</button> </a> </p>");
        }

        client.println("<h3>Свет - State " + output4State + "</h3>");
        if (output4State == "off") {
            client.println("<p><a href=\""/4/on\"><button class=\"button\"> ON </button> </a>
</p>");
        } else {
            client.println("<p><a href=\""/4/off\"><button class=\"button button2\"> OFF
</button> </a> </p>");
        }

        client.println("<h3>Обогреватель - State " + output16State + "</h3>");
        if (output16State == "off") {
            client.println("<p><a href=\""/16/on\"><button class=\"button\"> ON </button> </a>
</p>");
        } else {
            client.println("<p><a href=\""/16/off\"><button class=\"button button2\"> OFF
</button> </a> </p>");
        }

        client.println("<h3>Окно - State " + output15State + "</h3>");
        if (output15State == "off") {
            client.println("<p><a href=\""/15/on\"><button class=\"button\"> ON </button> </a>
</p>");
        } else {
            client.println("<p><a href=\""/15/off\"><button class=\"button button2\"> OFF
</button> </a> </p>");
        }
    }
}

```

```
client.println("</body></html>");
```

HTTP-ответ заканчивается другой пустой строкой.

```
client.println();
```

Выходим из цикла.

```
break;
```

```
} else {
```

Если получаем новую строку, очищаем `currentLine`.

```
currentLine = "";
```

```
}
```

Если есть что-то еще, кроме символа возврата каретки,

```
} else if (c != '\r') {
```

Добавляем его в конец `currentLine`.

```
currentLine += c;
```

```
}
```

```
}
```

```
}
```

Очищаем переменную заголовка.

```
header = "";
```

Закрываем соединение.

```
client.stop();
```

```
Serial.println("Client disconnected.");
```

```
Serial.println("");
```

```
}
```

```
}
```

Функция, отвечающая за считывание данных с датчиков и их публикацию. Эта функция вызывается псевдопараллельным потоком, который создан ранее.

```
void datchiki() {
```

Считываем температуру и влажность с датчика DHT22 повторно, для актуализации данных.

```
int err = SimpleDHTErrSuccess;
```

```
if ((err = dht22.read2(&temperature, &humidity, NULL)) != SimpleDHTErrSuccess) {
```

```
Serial.print("Read DHT22 failed, err="); Serial.println(err);
```

```
}
```


Необходимо перевести вещественные числа в символьный тип данных для их публикации.

```
dtostrf(temperature, 6, 2, temperatureTemp); //перевод из float в char для публикации  
dtostrf(humidity, 6, 2, humidityTemp);
```

Публикуем данные о температуре и влажности.

```
client.publish("/espmain/temperature", temperatureTemp);  
client.publish("/espmain/humidity", humidityTemp);  
}
```

Далее создаем функцию для получения данных с датчика DHT22 и в зависимости от них, будет осуществляться управление нагрузкой. Эта функция вызывается псевдопараллельным потоком, который создан ранее.

```
void mainnight() {
```

Считываем температуру и влажность с датчика DHT22 повторно, для актуализации данных.

```
int err = SimpleDHTErrSuccess;  
if ((err = dht22.read2(&temperature, &humidity, NULL)) != SimpleDHTErrSuccess) {  
    Serial.print("Read DHT22 failed, err="); Serial.println(err); delay(2000);  
}
```

Выводим в терминал, температуру и влажность воздуха.

```
Serial.print("Sample OK: ");  
Serial.print((float)temperature); Serial.print(" *C, ");  
Serial.print((float)humidity); Serial.println(" RH%");
```

Далее в зависимости от температуры задаем состояние окна и обогревателя, для поддержания необходимой температуры.

```
if (temperature < 20.0) { //если температура меньше 20 градусов, то  
    digitalWrite(output16, LOW); //включить реле - обогреватель  
    output16State = "on";  
    if (output15State == "on") { //если окно открыто - закрыть  
        Serial.println("Okno zakryto");  
        servo.write(0); //ставим вал сервопривода под 0 градусов  
        output15State = "off"; //записываем в переменную новое состояние сервопривода  
    }
```

```

}
else if (temperature > 20.0 && temperature < 30.0) {
    digitalWrite(output16, HIGH); //выключить реле обогреватель
    output16State = "off";
    if (output15State == "on") { //если окно открыто - закрыть
        Serial.println("Okno zakryto");
        servo.write(0); //ставим вал под 0 градусов
        output15State = "off";
    }
    Serial.println("Norm temperatura");
}
else if (temperature > 30.0) {
    digitalWrite(output16, HIGH); //выключить реле обогреватель
    output16State = "off";
    if (output15State == "off") { //если окно закрыто - открыть
        Serial.println("Okno otkryto");
        servo.write(75); //ставим вал под 75 градусов
        output15State = "on";
    }
}
else {
    Если вдруг произойдет сбой чтения датчика DHT22, то будет режим по умолчанию.
    digitalWrite(output16, HIGH); //выключить реле обогреватель
    output16State = "off";
    if (output15State == "on") { //закрыть окно
        Serial.println("Okno zakryto");
        servo.write(0);
        output15State = "off";
    }
}
}

```

Создаем функцию, которая будет автоматически управлять поливом. Эта функция вызывается псевдопараллельным потоком, который создан ранее.

```
void nasos() {
```

Подключаем датчик влажности почвы к аналоговому входу платы.

```
int tmp_pochva = analogRead(pochva_pin); //считываем данные с датчика влажности  
почвы
```

```
tmp_pochva = tmp_pochva / 10; //переводим полученную цифру в диапазоне от 0 до 100
```

```
Serial.print("Vlazhnost pochvy ");
```

```
Serial.println(tmp_pochva);
```

В зависимости от полученных значений включаем насос или отключаем.

```
if (tmp_pochva < 50) {
```

```
    digitalWrite(output5, LOW); // включаем насос
```

```
    output5State = "on";
```

```
} else {
```

```
    digitalWrite(output5, HIGH); // выключаем полив
```

```
    output5State = "off";
```

```
}
```

```
}
```

Листинг основного файла шаблона base.html

```
<!DOCTYPE html>
<head>
  <meta http-equiv="content-type" content="text/html; charset=utf-8" />
  <meta name="description" content="" />
  <meta name="keywords" content="" />

  <meta http-equiv="refresh" >

  <title>RPi Web Server</title>
  <!-- Последний сжатый и скомпилированный JavaScript -->
  <script          src="https://code.jquery.com/jquery-3.1.1.min.js"          integrity="sha256-
hVVnYaiADRT02PzUGmuLJr8BLUSjGIZsDYGmIJLv2b8=" crossorigin="anonymous"> </script>
  <script src="https://cdnjs.cloudflare.com/ajax/libs/d3/3.5.6/d3.min.js"> </script>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <script          type="text/javascript"
src="//cdnjs.cloudflare.com/ajax/libs/socket.io/1.3.6/socket.io.min.js"> </script>
  <link href="http://fonts.googleapis.com/css?family=Open+Sans" rel="stylesheet" type="text/css" />
  <link href="http://fonts.googleapis.com/css?family=Kreon" rel="stylesheet" type="text/css" />

  <link rel="stylesheet" type="text/css" href="{{url_for('static',filename='style.css')}}" >

</head>
<body>
  <div id="header">
    <div id="logo">
      <a          href="/">          <span          class="cms">  </span> </a>
    </div>
    <div id="menu">
      <ul>
        <li class="first active"> <a href="/">Main</a> </li>
        <li > <a href="/panel">Panel</a> </li>
```

```

        </ul>
        <br class="clearfix" />
    </div>
</div>
<div id="wrapper">
    <div id="page">
        <div id="sidebar">
            <div class="side-box">
                <h3>Main menu</h3>
                <ul class="list">
                    <li class="first"><a href="/panel">Панель
управления</a></li>
                    <li><a href="/stat/">Статистика</a></li>
                    <li class="last"><a href="/info">Контакты</a></li>
                </ul>
            </div>
        </div>
        <div id="content">
            <div class="box">
                {% block content %}
                {% endblock %}
            </div>
            <br class="clearfix" />
        </div>
        <br class="clearfix" />
    </div>
    <div id="page-bottom">
        <div id="page-bottom-sidebar">
            <h3>Contacts</h3>
            <ul class="list">
                <li class="first">Mobile: +7-961-771-12-31</li>
                <li>Discord: Kelsin</li>
                <li class="last">Email: mr.kelsin@mail.ru</li>
            </ul>
        </div>
    </div>

```

```
<div id="page-bottom-content">
    <h3>About</h3>
    <p>Автоматизированная система управления тепличным хозяйством на
базе микроконтроллеров</p>
</div>
<br class="clearfix" />
</div>
</div>
<div id="footer">
    <a href="/">Kelsin AA</a>2020</a>
</div>
</body>
</html>
```

Листинг файла panel.html с кнопками управления теплицами

```
{% extends 'base.html' %}

{% block content %}
<h3 align="center">Control panel esp8266 over MQTT</h3>

<div class="admin-panel">
  <div class="item-admin">
    <h4 align="center">teplica № 1</h4>
    <p>Updated in <span id="readingsUpdated"></span><br>
    Temperature: <span id="temperature"></span>°C<br>
    Humidity: <span id="humidity"></span>%</p>

    {{ pins["auto_main"].name }}
    {% if pins["auto_main"].state == 'True' %}
      is currently <strong>on</strong></h5><div class="row"><div class="col-md-2">
        <a href="/{{ pins["auto_main"].topic }}/0" class="btn btn-block btn-lg btn-default"
role="button">Turn off</a></div></div>
    {% else %}
      is currently <strong>off</strong></h5><div class="row"><div class="col-md-2">
        <a href="/{{ pins["auto_main"].topic }}/1" class="btn btn-block btn-lg btn-primary"
role="button">Turn on</a></div></div>

    {% for pin in pins %}
    <h5>
      {% if pins[pin].board == "espmain" %}
        {% if pins[pin].name!="Auto" %}
          {{ pins[pin].name }}
          {% if pins[pin].state == 'True' %}
            is currently <strong>on</strong></h5><div class="row"><div class="col-md-2">
              <a href="/espmain/{{pin}}/0" class="btn btn-block btn-lg btn-default"
role="button">Turn off</a></div></div>
          {% else %}
            is currently <strong>off</strong></h5><div class="row"><div class="col-md-2">
              <a href="/espmain/{{pin}}/1" class="btn btn-block btn-lg btn-primary"
```

```

role="button">Turn on</a></div></div>

    {% endif %}
    {% endif %}
    {% endif %}
    {% endfor %}
    {% endif %}
</div>

<div class="item-admin">
    <h4 align="center">teplica № 2</h4>
    <p>(updating <span id="readingsUpdated"></span>)<br>
    Temperature: <span id="pin1esponse"></span>°C<br>
    Humidity: <span id="pin2esponse"></span>%</p>
    {{ pins["autoesponse"].name }}
    {% if pins["autoesponse"].state == 'True' %}
        is currently <strong>on</strong></h5><div class="row"><div class="col-md-2">
            <a href="/{{ pins["autoesponse"].topic }}/0" class="btn btn-block btn-lg btn-default"
role="button">Turn off</a></div></div>
        {% else %}
            is currently <strong>off</strong></h5><div class="row"><div class="col-md-2">
                <a href="/{{ pins["autoesponse"].topic }}/1" class="btn btn-block btn-lg btn-primary"
role="button">Turn on</a></div></div>
            {% for pin in pins %}
                <h5>
                {% if pins[pin].board == "esponse" %}
                    {% if pins[pin].name!="autoesponse" %}
                        {{ pins[pin].name }}
                        {% if pins[pin].state == 'True' %}
                            is currently <strong>on</strong></h5><div class="row"><div class="col-md-2">
                                <a href="/espmain/{{pin}}/0" class="btn btn-block btn-lg btn-default"
role="button">Turn off</a></div></div>
                                {% else %}
                                    is currently <strong>off</strong></h5><div class="row"><div class="col-md-2">
                                        <a href="/espmain/{{pin}}/1" class="btn btn-block btn-lg btn-primary"

```



```
role="button">Turn on</a></div></div>
```

```
{% endif %}
```

```
{% endif %}
```

```
{% endif %}
```

```
{% endfor %}
```

```
{% endif %}
```

```
</div>
```

```
<div class="item-admin">
```

```
<h4 align="center">teplica № 3</h4>
```

```
<h5>soon...</h5>
```

```
</div>
```

```
</div>
```

```
<br class="clearfix">
```

```
{% endblock content %}
```